

A Unified Propagation Framework for Reliability, Scheduling and Flow Analysis in Directed Acyclic Process Networks

by

[Temitope Ohiani]

A thesis presented in partial fulfilment of the requirements for the
degree of

Doctor of Philosophy

Department of Civil Engineering
University of Strathclyde

[Month Year]

Supervised by: [Edoardo Pattelli]

Co-supervised by: [Caroline Pyke]

Declaration of Authenticity and Author's Rights

This thesis is the result of the author's original research. It has been composed by the author and has not been previously submitted for examination which has led to the award of a degree.

The copyright of this thesis belongs to the author under the terms of the United Kingdom Copyright Acts as qualified by University of Strathclyde Regulation 3.50. Due acknowledgement must always be made of the use of any material contained in, or derived from, this thesis.

Signed: _____

Date: _____

Abstract

Acknowledgements

[Your acknowledgements text here. Typically includes thanks to: - Supervisors
- Funding bodies (EPSRC, industrial sponsors, etc.) - Industry collaborators
at Sellafield/NDA - Research group colleagues - Family and friends - Technical
support staff]

Contents

Declaration of Authenticity and Author's Rights	2
Abstract	3
Acknowledgements	4
Nomenclature	15
1 Introduction	16
1.1 Motivation	17
1.2 Objectives	17
1.3 Contributions	19
1.4 Thesis Structure	20
1.5 Publications	21
1.6 Summary	21
2 Existing Works	22
2.1 The Methodological Landscape	23
2.2 Uncertainty in Network Analysis	24
2.3 Multi-Analysis Methods for Complex Infrastructure	25
2.4 Availability of Engineering Analysis Software	27
2.5 Julia as a Language for Network Modelling and Analysis	29
2.6 Summary	30
3 Complex Processes as Directed Acyclic Graphs	31
3.1 Introduction	32
3.2 From Process to Graph	32
3.3 Node Roles	33
3.4 The Formal Object	34
3.5 Information on the Network	35
3.6 Assumptions and Limits	36
3.7 Three Questions	36

4	Input Processing Module	37
4.1	Introduction	38
4.2	File Parsers	39
4.2.1	Network File Parser	39
4.2.2	MetaData Parser	40
4.3	IPF's Graph Object	42
4.3.1	Topological Sorting in DAGs	42
4.3.2	Extended Network Properties	43
4.3.3	The Unified Graph Object	45
4.3.4	Why not Julia's Native Graph Object?	45
5	Sub-graph Decomposition: Diamond Processing Module	49
5.1	Introduction	50
5.1.1	Decomposition in Complex Networks	50
5.2	A Generic Decomposition Module	51
5.3	The Diamond Subgraph	52
5.4	The Identification Algorithm	54
5.4.1	Shared ancestry and the earliest covering fork	55
5.4.2	Recursive flow	56
5.4.3	Subgraph construction	57
5.4.4	Context-aware identity	58
5.5	Structural Correctness	59
5.5.1	Global completeness	59
5.5.2	Determinism	60
5.6	The Unique Diamond Object	61
5.7	Worked Examples	62
5.7.1	The induced diamond	62
5.7.2	A factorised join	62
5.7.3	A nested decomposition	62
5.8	Summary	65
6	Information As Probabilities: Probability Propagation Toolkit	66
6.1	Introduction	67
6.1.1	Computing network reliability	67
6.2	The Belief of a Node	69
6.3	Propagation in a Single Pass	70
6.4	Reconvergence Breaks the Local Update	71
6.5	Conditioning at a Join	72
6.6	Supernodes and Progressive Decomposition	74
6.6.1	A multi-level worked example	76

6.7	Validation	78
6.8	Complexity Analysis	78
6.9	Propagating Uncertain Probabilities	80
6.9.1	Interval inputs are propagated exactly	81
6.10	Probability-Box Propagation	82
6.10.1	The conditioning combination is not a convolution	82
6.10.2	Two dependence bounds, one proved and one open	83
6.10.3	What the bounds are worth	84
6.10.4	The cost of the distribution	85
6.11	Case Study: A Medical Drone Logistics Network	85
6.11.1	Inputs, traced to their source	86
6.11.2	Three designs and the redundancy boundary	86
6.11.3	What the propagated bounds say	88
6.12	Summary	90
7	Information As Resources: Capacity Flow Toolkit	91
7.1	Introduction	92
7.2	Mathematical Model and Network Flow Semantics	93
7.3	Available Modules	98
7.4	Base Flow Module	99
7.4.1	Max-Flow Min-Cut Solvers	99
7.4.2	Push-Relabel Solver	101
7.5	Network Flow Diagnostics	101
7.5.1	Min-Cut Utility Module	101
7.5.2	Structure Utility Module	103
7.5.3	Flow Sensitivity Module	104
7.5.4	Failure Impact Module	106
7.5.5	Parametric Threshold Module	107
7.6	Exactness, Validation, and Scope	108
7.6.1	Post-solve validation	108
7.6.2	Scope of results	109
7.6.3	Computational Complexity	109
7.7	Network Flow Output Structure	110
7.8	Case Study: Synthetic Benchmark Network	113
7.8.1	Benchmark Network Design	113
7.8.2	Comparative Results (Config A vs Config B)	114
8	Information As Schedule and Cost: Critical Path Toolkit	124
8.1	Introduction	125
8.2	Scheduling Analysis on Networks	125

8.3	Analysis Modes	126
8.4	The Two Kernels	127
8.5	Interval Analysis	128
8.6	Exact Interval Floats: the Domination Split	129
8.7	Case Studies	131
8.7.1	One network, four questions	132
8.7.2	The most reliable route	134
8.7.3	The boundary	136
8.8	Genericity	136
8.9	Summary	136
9	Package Implementation in Julia	138
10	The No-Code Web Interface	140
10.1	Introduction	141
10.2	A Local Client and a Local Server	142
10.3	The Contract	143
10.4	Data Sovereignty	144
10.5	The Interface	144
10.6	Summary	147
11	An End-to-End Framework Application: An Integrated Case Study	148
12	Conclusions	150
	Appendices	152
A	Case-Study Network Specifications	153
B	Full Numerical Results	155
	Bibliography	157

List of Figures

1.1	The framework as a layered toolkit. Contribution 1 is the network representation and its decomposition; Contribution 2 is the shared propagation algorithms; Contribution 3 is the interval and probability-box support each toolkit actually carries, full support in reliability, interval only in schedule, neither yet in flow; Contribution 4 is the interface; Contribution 5 is the package the interface is built on.	18
3.1	A small treatment process and its model. Components become nodes, dependencies become directed edges, and the duplicated treatment stage becomes a pair of parallel routes that later reconverge.	33
3.2	Node roles on a small network. Double borders mark the source and the sink, the bold outline marks the fork, and the shaded node is the join where the redundant pair of routes reconverges alongside an independent supply.	34
5.1	Induced diamonds. Every diamond path originates from the single fork and the subgraph carries one diamond join.	52
5.2	Non-induced diamonds. In (b) two diamonds overlap through the shared fork g , and in (c) parent p is itself the fork of the inner pattern.	53
5.3	Edge isolation at a nested join	58
5.4	A factorised join. Two correlated groups meet at join 7, one through each fork, and source 8 enters from outside both. Bold edges mark the two constituent diamonds.	62
5.5	Frame 1. The network, with forks outlined in bold and joins shaded.	63
5.6	Frame 2. The maximal diamond at join 8, with fork 1 fixed (filled) and the span covering the whole network.	63
5.7	Frame 3. The recursive call at join 6 in context $E = \{1\}$, with fork 3 fixed, the inner diamond in bold, and edge $(1, 3)$ cut.	64

5.8	Frame 4. The emitted hierarchy of two maximal diamonds, with the inner diamond D_3 stored once and referenced by both.	65
6.1	The simple diamond. Source 1 feeds fork 2 (bold), whose two paths reconverge at join 5 (shaded).	71
6.2	The multi-level worked network. Sources are double-bordered, forks 1, 7 and 14 bold, joins 4, 6, 11, 12 and the target T shaded.	76
6.3	Maximum conditioning-set size against the logarithm of the re-ordered decision diagram's node count across the complexity-validation networks. Both quantities measure the same structural width, and neither method dominates.	80
6.4	Tightness envelope of the propagated probability-box bound on the benchmark grid: band width against component uncertainty width, across input-distribution shapes and reconvergence regimes. Soundness holds in every configuration, and only tightness varies, driven by regime and width rather than input shape.	84
6.5	Reliability map of the dense decentralised design: each facility coloured by the lower bound of its propagated reachability (left, darker is less certainly reachable) and by the width of its reliability band (right, darker is more uncertain). Hubs, drawn as diamonds, are assumed always available. Uncertainty concentrates at island and far-northern facilities, and the worst-served facility, Islay Hospital, is bounded at $[0.56, 0.72]$	89
7.1	Minimal single-source single-sink capacitated DAG network.	93
7.2	Graph reduction of a multi-source multi-sink capacitated DAG to an equivalent single-source single-sink max-flow problem.	95
7.3	Node-splitting transformation.	96
7.4	Runtime workflow for the flow-capacity toolkit with available modules	98
7.5	Single-edge sensitivity in Config A and cross-configuration rank comparison (A vs B).	118
7.6	Flagship benchmark DAG (43 nodes, 107 edges, 8 layers). Node colours: blue = sources, green = hubs, red = constrained nodes, orange = bypass nodes, purple = trunks, brown = secondary, pink = combiner, grey = sinks.	121
7.7	Sensitivity ranking for non-zero ΔF^* values in Configurations A and B.	122
7.8	Degradation trajectory $F^*(\alpha)$ for Configurations A and B under uniform capacity scaling.	122

7.9	Sink-flow comparison for edge-only and node-capacitated solves across Configurations A and B.	123
8.1	The running example. Node durations 1, 5, 2, 1; LongestPath is critical through nodes 1, 2, 4 and ShortestPath through 1, 3, 4. . .	128
8.2	Duration classes relative to node v . Shaded nodes are dominated by v (every complete route through them passes v), the unfilled node is incomparable to it, and double-bordered nodes form the bypass set H_v : they share routes with v and can also route around it.	130
8.3	The water network read for both extremes. The bold double-bordered chain sets the minimum release timeline of 45 units. The shaded stages and dashed edges form the tied-fastest route system, every path through which completes in 15 units. The two readings share no stage.	132
8.4	Exact float bounds on the water network with every duration at plus or minus 20 percent. Black bars are the nine possibly-critical stages, whose lower bounds reach zero. Grey bars are three of the 23 stages whose floats stay provably positive everywhere in the uncertainty box.	134
8.5	Best-route success across the 323 drone destinations. Left of the dashed line at 0.5 lie the 104 destinations that no single route serves reliably.	135
10.1	The local-first architecture. The browser front end drives the analysis service over HTTP on the local machine, and the service invokes the framework's modules, keeping sessions and the decomposition store as files beside the user's own.	142
10.2	The guided pipeline. Each step unlocks when its prerequisite data exists, and any unlocked step is reachable directly.	145
10.3	The network as the interface presents it, the interactive layered drawing and the structure dashboard derived by input processing.	146
10.4	Propagation results as the interface presents them, ranges shown as ranges, with scenarios of the same network compared side by side.	146

List of Tables

3.1	Notation established by the network model.	35
4.1	Input Processing Contracts by Analysis Class	42
4.2	Components of the Unified Graph Object	46
5.1	Fields of the unique diamond object	61
6.1	Conditioning trace of the worked network. w is the conditioning node's contextual belief, $\psi(1)$ and $\psi(0)$ the conditional signal probabilities the diamond's sub-runs return at the join, and $P(E)$ the group's recombined signal $w \psi(1) + (1 - w) \psi(0)$	77
6.2	Certified bound on the requirement-violation probability $P(b(v) \leq 0.95)$ at the benchmark grid's target node, from one propagation, against the Monte Carlo sample count required to match the certified precision under worst-case a-priori planning.	85
6.3	The three proxy designs: structure, conditioning width, and propagated interval reliability bounds. Every belief is contained in $[0, 1]$ with lower bound at most upper bound. The worst-served facility in both dense designs is Islay Hospital.	88
7.1	Max-flow solver comparison.	99
7.2	Per-task computational complexity for the flow capacity toolkit. T_s denotes the cost of a single max-flow solve under the chosen algorithm.	110
7.3	Unified output schema for network flow capacity analysis across all modules.	112
7.4	Configuration comparison summary.	114
7.5	Node-upgrade sweep (Configurations A and B give identical results): minimum single-node capacity increase required to restore sink_1 service under the node-capacitated solve.	116
7.6	Full ranked single-edge sensitivity results for Configuration A (all non-zero ΔF^* edges).	117

7.7	Full ranked single-edge sensitivity results for Configuration B (all non-zero ΔF^* edges).	117
7.8	Parametric threshold and marginal range comparison for key edges.	120
7.9	Min-cut enumeration for Config A.	120
8.1	The shipped modes. Every row carries a defined backward semantics; the margin column names what the backward pass reports.	126
8.2	LongestPath on the water network: completion 45 along the critical chain, with the second chain behind it.	133
8.3	Accumulation on the water network under a budget of 528 units at outlet 27 (current total 480). Allowance is headroom divided by path multiplicity.	133
8.4	The five best-served destinations by single-route success. At the other end of the scale, 104 of the 323 destinations fall below 0.5 on even their best route.	135

List of Algorithms

1	FindIterationSetsWithClosure	44
2	Recursive diamond identification (skeleton)	55
3	Ford-Fulkerson Min-Cut Max-Flow	100
4	Push-Relabel Min-Cut Max-Flow	101

Nomenclature

Abbreviations

IPA	Information Propagation Algorithm
IPA	Information Propagation Framework
DAG	Directed Acyclic Graph
MC-MC	Min-Cut Max-Flow
DFS	Depth First Search
BFS	Breath First Search
BP	Belief Propagation
CPM	Critical Path Method
MC	Monte Carlo
NDA	Nuclear Decommissioning Authority
p-box	Probability Box

Chapter 1

Introduction

1.1 Motivation

A water distribution network, a power grid, a plant process line and a construction schedule are frequently the same kind of directed dependency structure, asked three different questions: does the system keep functioning when a component fails, how much can it carry, and when does a dependent sequence of activities finish. Reliability computation, flow analysis and schedule analysis answer these three questions separately, each with its own methods, and each typically consuming a single precise number for what it does not know exactly about its inputs, a component failure probability, an edge capacity, an activity duration.

No tool or method combines all three of these analyses on one network with any of them propagating interval or probability-box uncertainty natively. Huang, Huang and Lin’s exact project-reliability method pairs reliability with schedule but not flow, computing the probability that a project completes within both a time and a budget constraint from discrete, point-valued activity states [1]. Tong’s Georgia Tech dissertation pairs reliability with flow but not schedule, using a point-valued probability propagation method throughout [2]. Running several kinds of analysis against one shared network object is itself not new: `pandapower`, an open-source power-system tool, runs five different analyses, power flow, optimal power flow, state estimation, topological search and short-circuit calculation, against one shared graph of buses and lines [3]. None of these three, and nothing else found in a systematic search, combines reachability reliability, capacitated flow and activity scheduling on one network representation with native imprecision propagation across the combination. Chapter 2 surveys this landscape in full.

This thesis develops a framework that closes that gap: one network representation supporting all three analyses, native interval and probability-box propagation wherever the underlying computation supports it, and delivery through a local, no-code interface rather than a script. Figure 1.1 sketches the framework this way, top to bottom: the network representation and its decomposition strategy, the propagation algorithms the three toolkits share a common structure through, the point at which each toolkit’s imprecision support actually stops, reliability carrying intervals and probability boxes, schedule carrying intervals only, flow carrying neither yet, and the package and interface the whole framework is delivered through.

1.2 Objectives

The objectives of this thesis are summarised as follows:

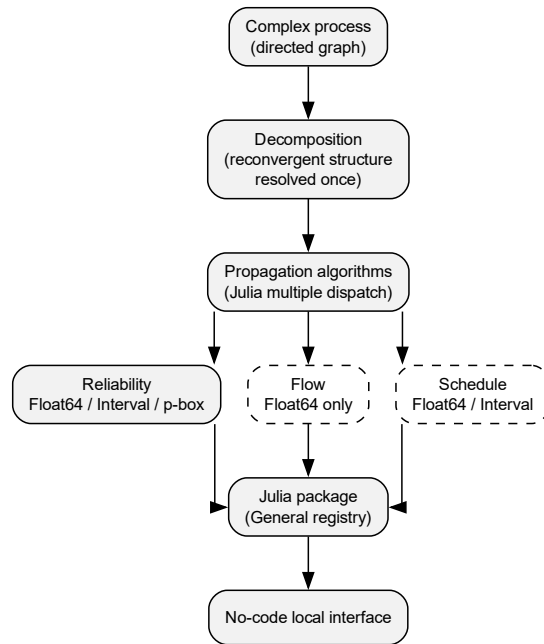


Figure 1.1: The framework as a layered toolkit. Contribution 1 is the network representation and its decomposition; Contribution 2 is the shared propagation algorithms; Contribution 3 is the interval and probability-box support each toolkit actually carries, full support in reliability, interval only in schedule, neither yet in flow; Contribution 4 is the interface; Contribution 5 is the package the interface is built on.

- To develop a single directed-graph representation of a complex process, general enough that reachability reliability, capacitated flow and activity scheduling are each a reading of the same object rather than three separate models.
- To develop a decomposition strategy that resolves reconvergent structure, the redundancy that makes each of the three analyses expensive, once per query rather than once per analysis.
- To develop propagation algorithms, sharing one code path specialised by Julia’s multiple dispatch, that serve reliability, flow and schedule analysis alike without three independently maintained implementations.
- To extend those algorithms to intervals and probability boxes wherever the underlying computation supports it, and to state plainly, not elide, the domains and the specific value types where it currently does not.
- To deliver the framework as a documented, tested Julia package, published to the Julia General package registry.
- To deliver the framework through a local, no-code interface, built on that same package, that asks its user for files and clicks rather than code.

1.3 Contributions

The main contributions of this thesis are summarised as follows:

- **Contribution 1:** a unified network representation and a decomposition strategy for it, developed together, so that a reconvergent structure identified once is available to every analysis run against the network rather than rediscovered per analysis.
- **Contribution 2:** propagation algorithms, generic across value type through Julia’s multiple dispatch, that compute reliability, flow and schedule analysis from one shared code structure rather than three independently maintained ones.
- **Contribution 3:** native propagation of interval and probability-box uncertainty through the reliability computation, and of interval uncertainty, not yet probability-box, through the schedule computation. The flow computation shares neither and remains point-valued. Each of these three boundaries is stated plainly here, not written around.

- **Contribution 4:** a local, no-code interface to the framework, so that running an analysis does not require writing or reading code.
- **Contribution 5:** the framework delivered as a ready-to-use Julia package, published to the Julia General package registry, and used directly as the server backend behind Contribution 4’s own interface rather than as a separate research prototype alongside it.

1.4 Thesis Structure

Chapter 2 surveys the literature this thesis positions itself against: the separate literatures for reliability, flow and schedule analysis, how each represents what it does not know precisely, the closest existing tools and methods that combine more than one of the three analyses, and the languages and software delivery patterns already established in adjacent engineering fields.

Chapter 3 develops the network representation itself, a directed graph general enough to support all three readings of the same structure.

Chapter 4 describes how a network definition is parsed, validated and turned into that graph object.

Chapter 5 develops the decomposition strategy that resolves reconvergent structure once, the shared foundation every analysis toolkit that follows depends on.

Chapter 6 develops the reliability toolkit: exact source-to-node reachability, computed by message passing, generic across point-valued, interval and probability-box inputs.

Chapter 7 develops the flow toolkit: exact capacitated flow, and states its current point-valued boundary plainly.

Chapter 8 develops the schedule toolkit: the critical path method extended to interval durations, generic across the same value types as the reliability toolkit.

Chapter 9 discusses the framework’s Julia-specific implementation, including the server backend the framework’s own interface is built on, and the framework’s delivery as a package in the Julia General registry, without repeating the algorithmic content already developed in the chapters above.

Chapter 10 develops the no-code local interface, positioned against the delivery patterns Chapter 2 surveys.

Chapter 11 applies the complete framework to a single system, running reliability, flow and schedule analysis on it through the interface Chapter 10 develops, and closes the thesis.

1.5 Publications

According to the results of the research reported in this thesis, the following papers have been published, submitted, or are in preparation:

- Ohiani, T. and Patelli, E. (2023). “The Information Propagation Method for Efficient Network Reliability Analysis.” *2023 7th International Conference on System Reliability and Safety (ICSRS)*, 580–584.
- Ohiani, T. and Patelli, E. Journal paper submitted to *Reliability Engineering & System Safety* (in preparation).
- Conference paper on the framework’s no-code interface (planned).
- Conference paper for JuliaCon (planned).

1.6 Summary

Reliability, flow and schedule analysis are usually computed separately, each on its own network model, each with its own answer to what it does not know precisely about its inputs. No existing tool, method or doctoral thesis combines all three on one network representation with native imprecision propagation across the combination, and none delivers that combination without asking its user to write code. This thesis develops a framework that does both, delivered as a Julia package and through a local, no-code interface built on it.

Chapter 2

Existing Works

Infrastructure and process networks are routinely analysed for three separate properties: whether the system keeps functioning when a component fails, how much it can carry, and when a dependent sequence of activities finishes. A water distribution network, a power grid, a plant process line and a construction schedule are frequently the same kind of directed dependency structure examined three different ways, and the three literatures built around these properties, reliability computation, capacity and flow analysis, and project scheduling, developed largely apart from one another, each with its own vocabulary and its own notion of what counts as an exact answer.

All three properties reduce to a computation over the same kind of object, a directed graph whose nodes and edges can fail, vary, or take time to traverse. Reliability computation returns the probability that at least one path still delivers supply from source to demand once every component is subject to failure. Flow analysis returns the greatest throughput the network delivers given the maximum capacity each edge and node sustains, and where it bottlenecks. Scheduling returns the earliest completion time of a dependent sequence of activities and which of them carry no room to slip. Three different quantities, a probability, a capacity and a duration, computed over structures the three literatures largely built their own methods for without citing one another's.

2.1 The Methodological Landscape

A reliability block diagram arranges components in series, parallel, and k -out-of- n groups whose system reliability has a closed form [4], and fault and event trees widen the reachable logic by building the failure mechanism explicitly as a tree of gates [5], both efficient exactly as far as the failure logic reduces to a tree or a series-parallel structure. Path and cut-set decomposition reaches arbitrary topologies exactly by splitting the network into subproblems [6, 7], at a combinatorial cost that grows with the same redundancy that makes the analysis worth doing, and Monte Carlo simulation escapes structure entirely in exchange for exactness, at a sample cost that grows demanding when failures are rare [8]. Binary decision diagrams compile the same Boolean structure into a canonical form on which reliability is a single traversal [9, 10], and cutset conditioning and the junction-tree algorithm reach the same computation from probabilistic inference instead, treating reachability as marginal inference on a directed graphical model [11, 12]. Every one of these methods consumes a point-valued component probability.

Series-parallel reduction collapses independent stretches pair by pair [13], factoring conditions on a single component to split the graph into independent

cases [14], and polygon-to-chain transformation widens what can be collapsed in one step beyond a plain series or parallel pair [15]. Treewidth-bounded tree decomposition generalises the same idea to graphs no sequence of series-parallel reductions can fully collapse, bounding the cost of exact computation by a structural width of the graph rather than its raw size [16], and the hash-consed representations binary decision diagrams also depend on store a repeated substructure once rather than rebuilding it [17].

A dependent sequence of activities poses a structurally similar problem under a different algebra. The critical path method computes the longest path through a project network and the float every other activity carries, a computation whose float calculus dates to the late 1950s [18].

Flow along capacitated edges is the fourth reading, and in its deterministic form it is the oldest of the four, dating to the max-flow-min-cut results of the 1950s [19]. What deterministic flow analysis does not address is what happens once the capacities themselves are uncertain.

2.2 Uncertainty in Network Analysis

A component's failure probability, an edge's capacity, and an activity's duration are rarely known exactly. Each is typically estimated from sparse failure data, elicited from expert judgement, or quoted as a manufacturer's tolerance, and representing that further layer of not-knowing is itself a choice with several competing answers.

Fuzzy set theory represents such a quantity as a possibility distribution rather than a probability distribution, assigning each possible value a graded degree of membership rather than a frequency [20]. Dempster-Shafer evidence theory assigns belief mass to sets of possible values directly, combining evidence from independent sources without requiring a single probability assignment over individual outcomes [21]. Imprecise probability represents the same not-knowing as a bounded set of probability distributions rather than one distribution or one possibility function: an interval bounds an unknown probability between two fixed values, and a probability box bounds an entire unknown distribution between two fixed ones [22, 23].

Reliability computation's exact methods, from reliability block diagrams through junction trees, consume a single point-valued probability throughout; the imprecise-probability treatment above is the direct answer to that limitation.

Flow under uncertainty takes forms that do not coincide with each other. Robust optimisation treats an uncertain capacity adversarially rather than probabilistically: a capacity is drawn from an uncertainty set rather than assigned

a distribution, and the network is evaluated against its worst case within that set. The approach has its own complexity landscape, the robust maximum-flow problem is solvable in polynomial time, while the corresponding robust minimum-cut problem is NP-hard [24]. No probability measure enters this formulation at any point. The stochastic-flow network reliability problem, founded by Doulliez and Jamouille, holds capacities as point-valued random variables instead and asks for the probability that the network can deliver a stated demand: each arc’s capacity is a discrete random variable, and network reliability comes from decomposing the space of feasible capacity configurations [25]. The method family that follows, state-space decomposition, boundary-point enumeration, and sum-of-disjoint-products techniques for combining them, mirrors path and cut-set reliability computation closely enough that the same formalism travels into project scheduling directly: an activity’s duration and cost stand in for an arc’s capacity states, precisely the modelling move behind the exact multi-state project reliability method discussed in Section 2.3 [1].

Duration uncertainty in scheduling followed a similar split. PERT replaced each fixed duration with a three-point estimate and a distributional assumption almost as soon as the critical path method itself appeared [26]. An interval rather than a distribution over each duration is harder still: deciding whether an activity is possibly critical is NP-complete in general [27].

Across reliability, flow, and schedule, the same handful of representations recur, point-valued distributions, robust or adversarial bounds, and imprecise probability, but no single representation is used consistently across all three; each domain settled on its own answer largely independently of the other two.

2.3 Multi-Analysis Methods for Complex Infrastructure

Reasoning about a complex infrastructure system rarely stops at one question, and running more than one kind of analysis against a single shared network representation is an established pattern in engineering software, not unique to any one discipline. `pandapower`, an open-source power-system analysis tool, runs power flow, optimal power flow, state estimation, topological graph search, and short-circuit calculation against one explicit graph of buses, lines and transformers, a diversity of analysis paradigm matched by commercial power-system suites such as DlgSILENT PowerFactory and PSS/E [3]. The ArcGIS Utility Network models power, gas, and water networks as an explicit graph of features connected through topology, and runs eight distinct trace types, connectivity, isolation, sub-

network discovery, loop detection among them, against that one shared topology [28]. Neither tool’s analyses are this thesis’s reachability reliability, capacitated flow, or activity scheduling, and neither propagates interval or probability-box uncertainty.

Whether an existing tool or method combines reachability reliability, capacitated flow, and activity scheduling specifically, on one network representation, is a narrower question. The search for it turned up three distinct partial answers among tools and methods, plus one comparable piece of doctoral research, and none of the four meets it fully.

Commercial reliability software has, in practice, already paired reliability with flow. Several of the reliability block diagram and fault tree suites still in active use, ReliaSoft’s BlockSim, Isograph’s Availability Workbench, and ITEM Software’s ITEM ToolKit among them, bundle a throughput or process-flow module alongside their core reliability engine, modelling multi-flow-type capacity, bottlenecks and resource allocation on the same diagrams their reliability analyses use [29–31]. None of their public documentation describes a project-scheduling or critical-path module, and all three represent component life and failure with classical distributions, fitted from data or specified by the analyst, rather than intervals or probability boxes.

Open-source infrastructure simulation platforms have, from the opposite direction, paired flow with something schedule-shaped. InfraRisk integrates power-flow, water-hydraulic and traffic-flow simulators for interdependent power, water and road networks under its own banner of resilience analysis, and its recovery module sequences repair-crew actions with real start and end times computed from travel time and repair duration, either by heuristic prioritisation or by model-predictive-control optimisation [32]. That sequencing is a logistics schedule rather than a precedence-network critical-path computation, since it optimises the order and timing of repair actions against a performance-loss metric rather than the float of a fixed activity network. Its reliability side is likewise not a source-to-node reachability computation. The hazard module derives point component-failure probabilities from exposure and intensity to drive Monte-Carlo-style disaster scenarios, and the platform reports its results as simulated performance loss rather than an exact reliability figure.

A third direction pairs reliability with schedule directly, inside project-network reliability itself. Modelling a project’s activities as the arcs of a multi-state network and asking for the probability that the project completes within both a time and a budget constraint is an established line of work. Huang, Huang and Lin’s decomposition method computes this exact project reliability, splitting the space of feasible completion-time and budget combinations into subsets via a project’s

upper and lower boundary points and evaluating each by inclusion-exclusion, an approach the authors validate against full enumeration on a real thirteen-activity manufacturing project [1]. The method returns a project reliability of 0.906 for one tested time-budget pair on that project and traces how the figure moves as the constraints are relaxed. It has no notion of a physical arc capacity carrying a transportable flow, activity duration and cost standing in its place instead, and every input is a discrete state with a fixed probability rather than an interval or a probability box.

Doctoral research comes closer than any commercial or open-source tool to combining two of the three. Tong’s Georgia Tech dissertation develops new methods for infrastructure flow networks and investigates their reliability in terms of both connectivity and flow capacity together, using a probability propagation method published the same year and belonging to the same message-passing lineage surveyed in Section 2.1 [2]. Schedule and critical-path analysis are absent from it, and the probability treatment throughout is point-valued rather than interval- or probability-box-based.

No candidate combines all three analyses on one network, and the four above do not even pair the same two twice: reliability and flow are paired in commercial software with no schedule dimension, flow and a form of scheduling are paired in open-source simulation platforms with no formal reliability computation, reliability and schedule are paired in project-network theory with no true flow network, and reliability and flow are paired again, this time exactly, in Tong’s dissertation, still without schedule. Every one of the four, without exception, represents its uncertain inputs precisely or adversarially, never as an interval or a probability box, and neither **pandapower** nor the ArcGIS Utility Network above changes that picture: both run several kinds of analysis on one network, but neither runs this thesis’s specific three, and neither propagates imprecision either.

2.4 Availability of Engineering Analysis Software

In engineering analysis, three patterns of software availability recur, browser-delivered simulation-as-a-service, licensed desktop installation, and web-based GIS, and none of them has been adopted by the no-code and low-code business-software movement.

The no-code and low-code software category defines itself around business application development throughout its own literature. A systematic review of 383 definitions across 306 publications from 2014 to 2025 characterises no-code platforms as visual, drag-and-drop environments that let a non-programmer build software without writing code, and low-code platforms as the same idea with

a professional-developer audience layered on top [33]. Mapping the identified platforms onto ten functional system classes, the same review names enterprise-resource-planning extensions, customer-relationship platforms, multi-experience development platforms, business-process automation and management suites, robotic process automation, document management systems, and integration-platform hubs as the categories the low-code and no-code movement actually occupies, illustrated with named commercial examples such as Mendix, Salesforce, UiPath, Bizagi and Zapier. Engineering or scientific analysis, reliability, flow, structural or otherwise, does not appear among them.

In engineering analysis, browser-delivered simulation-as-a-service is one established alternative. SimScale, launched in 2013, runs computational fluid dynamics, finite element, thermal and electromagnetic analysis entirely in the browser with no local installation [34], though its computation runs in the vendor’s cloud rather than on the user’s own machine. The licensed desktop GUI reliability suite is a second, older pattern: the same commercial software named in Section 2.3 solves the no-programming half of the delivery problem as a local install rather than a browser application, the reverse trade-off from SimScale’s.

Some web-based GIS platforms exist specifically for hazard and infrastructure risk assessment. A PRISMA-based systematic review of this literature, screening 1,775 articles and admitting 65, organises the field around themes including visualisation and user-interface design and decision-support systems [35]. Repetto et al.’s platform is a concrete member of it. It delivers wind-monitoring, forecasting and statistical risk mapping for ports, motorways, rail lines and industrial plants exposed to wind. Its interface is web-based GIS built on free and open geospatial software, run by port authorities and operators who never touch the underlying numerical wind and wave models themselves [36]. Most platforms in this category focus on monitoring and forecasting rather than a structural reliability or component-failure computation.

Local-first software keeps the usability of the modern web while storing and computing its data on the user’s own machine rather than a remote server [37], an architecture that matters most in exactly the industries whose proprietary system models cannot leave the premises.

SimScale is browser-based but not local; the desktop suites are local but not browser-based; the web-GIS platforms are browser-based but run on a remote server. No tool found in this search combines browser delivery, no programming, and local computation and data all at once.

2.5 Julia as a Language for Network Modelling and Analysis

MATLAB has offered built-in graph and network functions since the mid-2010s: shortest-path, maximum-flow, centrality, connected-component and spanning-tree computations all operate on its native `graph` and `digraph` classes [38]. R's equivalent general-purpose graph package is `igraph`, used for shortest paths, centrality, community detection and flow computation over one shared graph object [39], and R additionally carries a package purpose-built for structural reliability specifically, computing system and survival signatures over a network's topology [40], the kind of dedicated reliability tooling neither MATLAB's built-in functions nor Python's ecosystem, below, provide natively. Rust's graph ecosystem is newer and smaller, centred on `petgraph`, a general-purpose graph data-structure library with the same shortest-path, spanning-tree and traversal algorithms as the languages above [41].

Python's equivalent general-purpose graph package is `NetworkX`, whose own canonical paper presents it as covering simple graphs, directed graphs, and graphs with parallel edges, over a single native numeric type [42]. No single Python package purpose-built for network reliability computation, in the reliability block diagram, path-set, or decision-diagram sense this literature uses the term, was found in this search, the way R's `ReliabilityTheory` package is; the closest candidate, `pgmpy`, is a general Bayesian-network and probabilistic-graphical-model toolkit with belief propagation and structure learning built in, useful machinery for the same class of problem but not a package built to solve it directly. This is a gap in packaged tooling specifically, not in published research: network reliability analysis is itself an actively reviewed field with applications across communications, transportation, and electrical and gas distribution [43], and that research draws on whichever language a given group already works in without converging on one shared package the way `NetworkX` did for general graph algorithms.

Julia's own graph ecosystem, centred on `Graphs.jl` and its satellite packages for weighted graphs, property graphs and bindings to established external libraries, occupies the equivalent general-purpose position to `NetworkX`, `igraph` and `petgraph`, though no single paper describes that ecosystem the way Hagberg, Schult and Swart's paper describes `NetworkX`. None of the packages named above, in any of the five languages, carries a native notion of an interval- or probability-box-valued edge or node, which is not a limitation any of them sets out to address, since none is built for propagating uncertain quantities across a network in the first place.

Multiple dispatch, choosing which method of a generic function to run from

the runtime types of every argument rather than just the first, is presented in the canonical Julia paper as the mechanism that lets one generic function definition serve arbitrarily many numeric and non-numeric types without its author anticipating any of them in advance [44]. The same paper grounds Julia’s central design claim, that a dynamic language need not trade its ease of use for the performance a statically compiled one gets, in the same type system and dispatch mechanism. This is the mechanism this framework’s own propagation step leans on to specialise correctly whether the value flowing through it is a float, an interval, or a probability box, without the author of the propagation step anticipating any of the three in advance.

2.6 Summary

Reliability, flow, and schedule analysis grew as three separate literatures, each with its own exact methods and its own way of representing what it does not know precisely about its inputs. Running several kinds of analysis against one shared network object is not itself a new idea in engineering software, but no tool, method, or doctoral thesis found in this search combines reachability reliability, capacitated flow, and activity scheduling on one network representation, with any of them propagating imprecision natively across the combination. No engineering-analysis tool found in this search delivers that combination through an interface that asks its user for files and clicks rather than code, either. This thesis’s own framework is built to close that gap. It computes all three analyses from one network model, propagates interval uncertainty natively through each and probability-box uncertainty through reliability specifically, and reaches its user through a local, no-code interface rather than a script.

Chapter 3

Complex Processes as Directed Acyclic Graphs

3.1 Introduction

The systems this thesis serves are flow-oriented. Water is abstracted, treated and distributed. Power is generated, transformed and delivered. Materials move through processing stages towards demand points, and project activities complete in an order their dependencies dictate. In each case the behaviour of the whole is organised by a directed movement of something, supply, product, work or information, through a network of components that pass it on.

The framework rests on a single model of such systems, defined in this chapter and consumed unchanged by every module that follows. The model fixes what a system becomes when it enters the framework, what its components and their relationships mean, what a value attached to a component can stand for, and which assumptions the later analyses inherit. It closes with the three questions the model makes precise, which the three analysis toolkits of Chapters 6, 7 and 8 answer.

3.2 From Process to Graph

A system enters the model through two identifications. Components become nodes, and dependencies become directed edges. A component is whatever unit of the system can meaningfully carry a value of its own, a pump, a treatment stage, a substation, a task, and a dependency is the fact that one component feeds, supplies or precedes another. Direction is not an abstraction imposed on the system but a property read off it, because in a flow-oriented process the relation between two connected components is already asymmetric. Water leaves the intake and arrives at the pump, and the pump does not supply the intake.

The model further requires that the directed edges form no cycle, and this is the one modelling decision that needs defending rather than reading off. For a large class of the target systems acyclicity is simply true. Treatment trains, supply chains, assembly processes and project schedules move material or work strictly forward, and a dependency cycle in a schedule would be a planning error rather than a modelling choice. For other systems it is an orientation of a richer reality. A distribution pipe can carry water in either direction in principle, yet a network operating around its design flows has a dominant direction on every pipe, and orienting each edge by that dominant flow yields a directed acyclic snapshot of the operating regime. What the orientation gives up is the analysis of regimes far from the snapshot, flow reversals under unusual demand, recirculation loops, feedback control, and the model excludes these deliberately rather than approximating them. Figure 3.1 shows the mapping on a small treatment process,

whose deliberate duplication of the treatment stage already displays the structural phenomenon that Section 3.3 names.

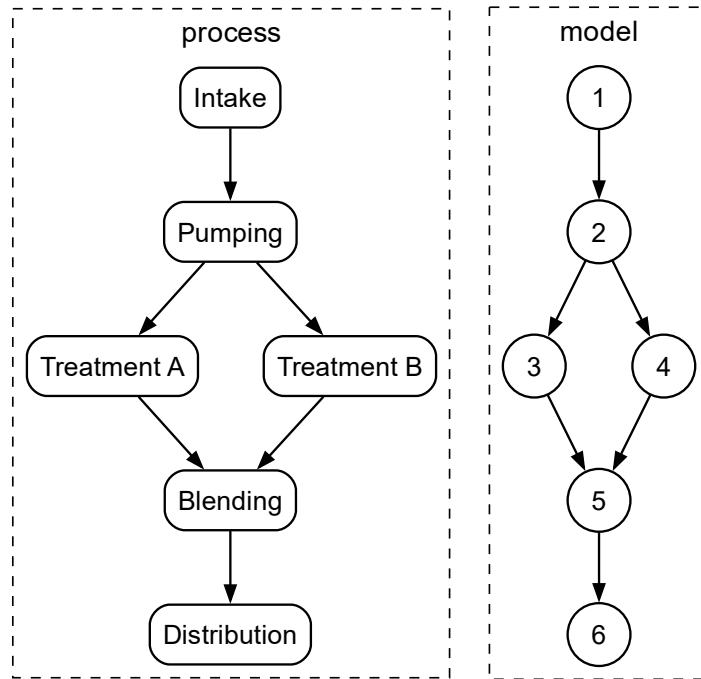


Figure 3.1: A small treatment process and its model. Components become nodes, dependencies become directed edges, and the duplicated treatment stage becomes a pair of parallel routes that later reconverge.

3.3 Node Roles

Position in the network gives every node a role, and each role has a physical reading. A node with no incoming edge is a source, the point where supply, material or work enters the system, an intake, a generator, the start of a project. A node with no outgoing edge is a sink, where the system delivers, a demand point, an outlet, a completed handover. A node that feeds two or more downstream components is a fork, and forks are how systems distribute and, above all, how they build in redundancy, because providing two routes to the same destination begins with one component supplying two. A node fed by two or more components is a join, where distributed flows gather, where assembled parts meet, and where redundant routes come back together.

The last of those readings deserves emphasis, because it is a fact about engineered systems rather than about graphs. Robust systems duplicate deliberately,

and duplicated routes must reconverge at the destination they protect, so a well-designed network is dense with paths that separate at a fork and meet again at a join while sharing the components upstream of the fork. Whether that sharing matters depends on the question being asked, and the chapters that ask the questions will return to it. Chapter 5 makes the pattern itself precise and Chapter 6 shows why, for one of the three readings of the model, ignoring it produces wrong answers. Here it is enough that reconvergence is a designed feature of the systems being modelled, visible already in Figure 3.2, not an accident of drawing.

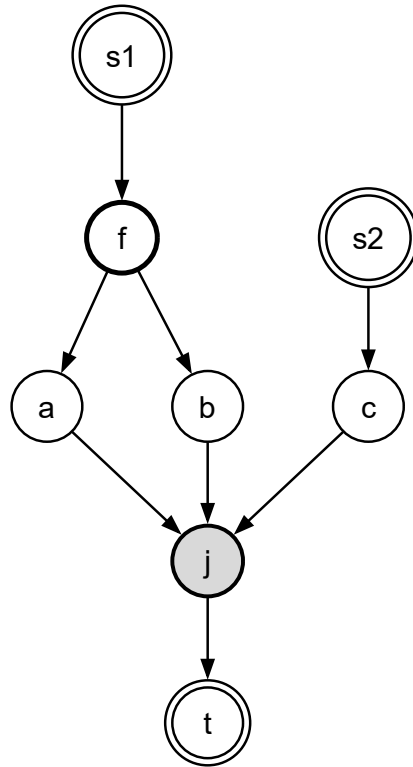


Figure 3.2: Node roles on a small network. Double borders mark the source and the sink, the bold outline marks the fork, and the shaded node is the join where the redundant pair of routes reconverges alongside an independent supply.

3.4 The Formal Object

Definition 3.1 (Network model). *A system is modelled as a finite directed acyclic graph $G = (V, E)$ with node set V and edge set $E \subseteq V \times V$. The sources $S \subseteq V$ are the nodes with no incoming edge and the sinks $T \subseteq V$ the nodes with no outgoing edge. For a node v , $\text{Pa}(v)$ denotes its parents, the nodes with an edge into v , and $\text{Ch}(v)$ its children.*

Definition 3.2 (Ancestry). *A node u is an ancestor of v when a directed path leads from u to v , and a descendant when one leads from v to u . The sets $\text{anc}(v)$ and $\text{desc}(v)$ collect them, and throughout the framework both are taken self-inclusively, so that $v \in \text{anc}(v)$ and $v \in \text{desc}(v)$.*

Definition 3.3 (Forks and joins). *The forks of G are $F = \{v : |\text{Ch}(v)| \geq 2\}$ and the joins are $J = \{v : |\text{Pa}(v)| \geq 2\}$.*

These definitions are the shared vocabulary of the thesis. The chapters that follow reference them rather than restate them, and Table 3.1 collects the notation in one place.

Table 3.1: Notation established by the network model.

Symbol	Meaning
$G = (V, E)$	the system as a finite DAG
S, T	sources (no incoming edge), sinks (no outgoing edge)
$\text{Pa}(v), \text{Ch}(v)$	parents and children of v
$\text{anc}(v), \text{desc}(v)$	ancestors and descendants, self-inclusive
F, J	fork nodes (fan-out ≥ 2), join nodes (fan-in ≥ 2)

3.5 Information on the Network

The model separates what the network is from what is asked of it. The topology of Definition 3.1 is fixed once per system, and an analysis attaches values to its nodes and edges whose meaning depends on the question. Under the probabilistic reading a node's value is the probability that the component operates and an edge's value the probability that the handover between its endpoints succeeds, so the network describes what survives failure. Under the schedule and cost reading the values are durations or costs, of the component's own processing and of the transfer along each dependency, so the network describes when things complete and what they accumulate. Under the capacity reading the values are throughput limits, and the network describes how much can be carried. Three readings of one topology are three different analyses, and keeping the topology common across them is what allows a single system model to answer questions that are usually asked of three separate models [45].

How well a value is known is a further axis, independent of what it means. A deterministic value asserts full knowledge, and is the right form for design constants and well-measured components. An interval asserts only bounds, and is the honest form when data is sparse, when a manufacturer quotes a tolerance,

or when experts will commit to a range but not a number. A probability box bounds a distribution rather than a value, and carries partial statistical knowledge, measurements enough to constrain a curve without fixing it. The framework propagates all three forms through the same analyses, so the choice among them reflects the state of knowledge about the system rather than the limits of the tool.

3.6 Assumptions and Limits

The model buys its tractability with four assumptions, and each is a limit on what the answers mean. Component behaviours are independent, so a common cause that fails several components together is not represented and must be modelled explicitly as a shared upstream node where it matters. Under the probabilistic reading a component is binary, operational or failed, and degraded intermediate states are outside the model. The topology is static over the analysis horizon, so reconfiguration during operation is a sequence of separate analyses rather than one. And the acyclicity of Section 3.2 prices out feedback and reversal, which is a property of the operating snapshot being analysed rather than of the physical assets. None of these assumptions is hidden in an algorithm, and every result in the later chapters is conditional on exactly this list.

3.7 Three Questions

The model makes three questions precise, and the thesis answers them in turn. Under the probabilistic reading the question is whether information survives the network, with what probability a node is reached from the sources despite component failures, and it is answered by the Probability Propagation Toolkit of Chapter 6. Under the capacity reading the question is how much the network can deliver, where the limits bind and what relieving them buys, answered by the Capacity Flow Toolkit of Chapter 7. Under the schedule and cost reading the question is when and at what accumulation the network completes, which dependencies set that outcome and which have room to move, answered by the Critical Path Toolkit of Chapter 8.

Before any of the three can be asked, the model itself must be built from the heterogeneous descriptions in which real systems arrive, which is the work of Chapter 4, and for the probabilistic question the reconvergent structure of Section 3.3 must be identified and organised, which is the work of Chapter 5. The two are preprocessing in the strict sense, performed once per system, and everything after them is a reading of the object this chapter has defined.

Chapter 4

Input Processing Module

4.1 Introduction

When it comes to adopting a tool like IPF, the first consideration will always be about dataset format and model compatibility. Understanding complex civil infrastructures often requires measuring multiple different metrics and objectives. The same system or process might get a different type of treatment across different teams within the same project.

Attribute-level information often gets split across silos of project objectives. Each silo usually only maintains the network parameters and behaviours specific to the operations concerned. Within the same infrastructure, one workflow may track fault/health signals while another tracks project related costs. For example, a fault detection objective is unlikely to track cost impact metrics or environmental load.

This could be because of scope restriction or lack of access to that additional data. Beyond scope of analysis, the stage of the system's lifecycle might affect the kind of data available, as planning and design needs evolve alongside the tools used as the system matures.

The actual tools used across infrastructure operations often implement different schemas and export conventions which may introduce more formatting inconsistencies. Sometimes the dataset has been purpose built to fit a specific tool or standard. Despite this, from a purely structural perspective, the network model is simply a collection of edges describing some topology, process, or flow of activities. Regardless of variation in source format, any DAG-based system model can generally be reduced to a node list and an edge list, in which nodes represent point features such as sources, junctions, assets, or demand points, and edges represent the relationships between these assets. This is consistent with widely used infrastructure data formats, such as GraphML, EPANET .inp, and MAT-POWER .m, which all reduce to node and edge records beneath their respective schemas.

However, when system characteristics and behaviours come into play, the structure is not so clear-cut. The relationships may carry information about the system's behaviour or analysis touchpoints such as resource capacity. This work takes a split perspective of network structure vs analysis inputs as different layers of the same network model. The initial dataset may combine these, but the goal of this one-time preprocessing step is to parse different formats into what the framework needs irrespective of source schema.

The Input Processing Module is the interface between heterogeneous raw inputs that describe the system's characteristics and the standardised network model used throughout the framework. This module is therefore necessarily the

sole mandatory entry point, without which no downstream algorithm can be validly invoked. This is implemented as a two-part transformation: parsing different raw input file format, and constructing the standardised graph object the framework operates on.

4.2 File Parsers

4.2.1 Network File Parser

Nodes in the physical system typically have descriptive identifiers that reflect their role in the network (Pump_A, Substation_North). However, integers are more efficient in terms of memory footprint while allowing faster indexing during the repeated lookups and set operations performed throughout the framework.

The network parser exposes a reader function that expects a .Edges or .CSV network structure file containing either an integer-keyed edge list or an adjacency matrix. Although these integers are not required to be contiguous, the user is responsible for mapping their domain-specific identifiers to unique integers before parsing the network into the framework.

The module uses a staged detection flow to automatically detect file types when called. Given file path, the reader function always prefers detection via file extension. This allows for .edges files to immediately trigger the edge list reader. On the other hand, a .csv file is ambiguous since it could contain either an adjacency matrix or a column-formatted edge list. In this case, the next detection stage is lightweight content sniffing that scans headers for common source-destination keywords that indicate an edge list.

If no matching header with expected descriptors is present, the next stage scans the file content for the integer-delimiter-integer pattern that usually indicates an edge list. The last edge list indicator checks the shape of the table itself where a two-column Nx2 shape is treated as an edge list, otherwise the file defaults to adjacency matrix parsing.

When an adjacency matrix is detected instead, the reader first streams the table row by row to build an effective edge list. As the edges are streamed in from either file type, self-loops are rejected immediately and cycle-forming edges are detected during ingestion, since both violate the acyclicity expectation of the target DAG model.

To keep the ingestion process linear in the number of edges, the reader function builds both the incoming and outgoing index directions in place in order to avoid a second pass that would otherwise be needed to construct the reverse adjacency. During the stream, for every valid edge discovered, both nodes and

their directional relationships are recorded in the same pass. Once the stream completes, the parser returns the framework’s minimal graph object.

4.2.2 MetaData Parser

Given a validated base graph object, the framework supports both node-level and edge-level information depending on the algorithm invoked. The analysis-specific input files are expected to hold the network parameters that drive each downstream algorithm. Three classes of propagation characteristics are supported: critical path, flow capacity, and probabilistic information. The sources of this information are usually decoupled from network modelling. Instead they tend to be observed externally and managed separately (such as manufacturer specs and historic data).

In large networks, embedding these behavioral properties into the network structure leads to a bloated file that can be harder to maintain. For multi-metric evaluation on the same network, the user would be forced to maintain duplicate topology files per analysis variant even when the network structure does not change.

To this end, regardless of analysis type, all algorithm inputs are required as .JSON files. JSON is a human-readable, platform-agnostic format that maps naturally to the key-value shape for describing component attributes. Where flat formats like CSV have a rigid tabular shape that makes them a good fit for describing the graph, JSON’s flexibility allows for a more expressive representation of system characteristics. Intervals, p-boxes, and other nested data types can be easily described by data owners without the need to invent column conventions or force fit flattened values on inherently nested properties. Since it also requires no specialist tools to construct or edit, as long as the user understands the node and edges mapping, then no special understanding of graph theory or modelling tools is required to contribute values.

Schema Conventions and Type Dispatch

Although metadata labels are useful for documentation and reporting, the parser’s execution is driven by parseable values, key shape, and algebraic compatibility with the specified analysis type. To determine the appropriate parser for different algorithm’s inputs, any JSON file must declare a `data_type` (scalar, interval, p-box) as well as an `analysis_type` (flow capacity, critical path, probability).

Irrespective of the analysis types, the rest of the schema contract is simple: values must be mappable to valid node identifiers and edge identifiers from the parsed DAG, and values must be parseable into the declared type domain. This

convention aims to ensure that uncertainty deserialisation is concentrated in one place rather than spread across multiple readers, while keeping the JSON interface flexible to different naming conventions.

Critical Path inputs describe node and edge weights in terms of critical-path style propagation. Since the framework is deliberately unit-agnostic for this analysis class, the weights may represent durations, costs, risk scores, environmental impact, or any other accumulative quantity, as long as the interpretation is consistent within a single run. The parser does not discriminate by unit-specific labels such as hours or risk points; it is instead driven by runtime operator functions. In addition to the node-value and edge-value maps, the input expects three operator functions to be specified: a combination function, a propagation function, and a node function, which are discussed further in the critical path methodology chapter. These operators are what define the analysis semantics at runtime and may be left unspecified, in which case the framework applies classical critical-path defaults.

Flow Capacity inputs describe throughput constraints across the network. Edge and node capacities may be supplied in the same JSON file or split across separate files; the parser infers the value mapping based on key structure. Edge capacities are the main requirement, with optional node capacities. Unlike the other two analysis classes, capacity in this work is not uncertainty-polymorphic and only handles fixed values. To model unconstrained capacities, the metadata reader supports standard infinity tokens including `Inf`, `+Inf`, `Infinity`, and ∞ . Given this, numeric soundness of capacity inputs is validated upfront, with non-negative and non-NaN constraints enforced at parse time and any malformed or invalid input rejected early.

Probabilistic Inputs describe uncertainty attached to nodes and edges, typically for reliability and probability propagation. Node priors capture component-level uncertainty and edge probabilities capture link-level transmission or failure likelihoods. All probability values, whether scalar, interval, or p-box, must remain within $[0,1]$; the framework rejects values outside this range at parse time. This is the most expressive of the three analysis classes, supporting all three data types. When a non-scalar data type is specified, the values contained must match the expected shape. For instance, an interval is expected as either a delimited tuple or nested properties keyed by upper and lower bounds. P-boxes, in contrast, must follow a tagged serialization contract and declare a `construction_type`. The p-box representation, including the construction type, is expected to mirror the constructor families supported by `ProbabilityBoundsAnalysis.jl`.

Table 4.1: Input Processing Contracts by Analysis Class

Layer / Class	Contract Shape	Value Domain	Validation / Dispatch
Network topology	Edge list or adjacency matrix over integer node IDs	Integer node IDs; directed edges	Format auto-detected; self-loops and cycles rejected at parse
Probability propagation	Node map and edge map keyed to DAG identifiers	Float64, Interval, pbox	Values within $[0,1]$; key shape separates node and edge maps; <code>data_type</code> drives deserialisation
Flow capacity	Edge capacity map (required), node capacity map (optional)	Float64 only	Non-negative, non-NaN enforced at parse; Inf tokens accepted (Inf, +Inf, ∞)
Critical path	Node-value and edge-value maps	Float64, Interval	Unit-agnostic; semantics set by runtime operators (combination, propagation, node); classical defaults applied if unspecified

4.3 IPF's Graph Object

4.3.1 Topological Sorting in DAGs

For the directed acyclic graph $G = (V, E)$, the topological sort T is a linear ordering of nodes where every node appears exactly once, and for every edge $u \rightarrow v$, node u must appear before v . This order can equivalently be expressed in layered sets such that each layer groups nodes that are at the same depth from the source nodes and do not have dependencies within the group.

Theorem 4.1 (IPF's Convergence Guarantee). *Given a DAG $G = (V, E)$, an information propagation algorithm will always converge in exactly $|L|$ iterations, where L is the topologically sorted layers of G .*

Proof. We prove by induction on the number of nodes $|V| = n$ that for any DAG $G = (V, E)$ there exists a valid topological sort T and an equivalent layered topological sort $L' = \{L_1, L_2, \dots, L_k\}$.

- *Base case:* If G contains only one node, then the topological sorting T is simply the node itself, and L contains a single set containing the node. Thus, $|L| = 1$ and the single layer contains only this source node.
- *Inductive Assumption:* Assume that for any DAG G with $n - 1$ nodes, there exists a valid topological sort T of G and a layered sort $L = \{L_1, \dots, L_{k-1}\}$.

- *Inductive Step:*

- First subset: Since G is a DAG, at least one source node must exist. We group all the source nodes as L_1 .
- Reduction: Removing the nodes in L_1 and all their outgoing edges from G transforms G into the reduced graph G' .
- Reintegration: By the inductive assumption, G' has a valid layered topological sort $L' = \{L_2, L_3, \dots, L_k\}$. Therefore, prepending L_1 gives $L = \{L_1, L_2, \dots, L_k\}$, which must be a valid layered topological sort of G .
- Next subset: With L_1 removed, a new set of nodes becomes the sources in the reduced graph G' , forming the next layer L_2 in the ordered collection.
- Further Reduction: Removing the nodes in L_2 and their outgoing edges further reduces G' . Repeating this process until no nodes remain results in an ordered collection of subsets of nodes $\{L_1, L_2, \dots, L_k\}$ where each set L_i is the set of source nodes identified in each iteration and k is the number of reductions required until G' has no nodes.

From this construction, for any node $v \in L_i$, all parents $p \in \text{Pa}(v)$ must lie in earlier layers L_j with $j < i$. That is, when processing layer L_i , all parent dependencies are already resolved, so each node is processed exactly once without revisiting. Therefore an algorithm on a DAG that processes nodes in this order is guaranteed to converge in exactly $|L|$ iterations. $\square$

This guarantee matters specifically because every analysis in this framework is an exact analytical method without iterative convergence loops. Given this, declaring a fixed processing order once allows for reuse across the framework. Furthermore, because nodes within the same processing layer share no dependencies, the layers provide a natural parallelisation boundary for any algorithm that supports it.

4.3.2 Extended Network Properties

The graph builder extends the parsed minimal object into IPF's complete graph object. Beyond the iteration sets just established, downstream methods also rely on cheap ancestry and reachability lookups, which the neighbourhood dictionaries alone do not provide. In large networks, computing these additional properties can be expensive, so the builder constructs them once during the same topological traversal.

In this work, the iteration sets L of G are considered one such property and are defined within the unified graph object. The input module computes this property of the network by implementing a depth-partitioned variant of Kahn’s algorithm as shown in Algorithm 1.

Algorithm 1 FindIterationSetsWithClosure

Require: Directed acyclic graph (acyclicity validated upstream by parser) as edge list E , outgoing index Out , incoming index In

Ensure: iteration_sets , ancestors , descendants

```

1: if  $E = \emptyset$  then
2:   return  $\langle \rangle, \emptyset, \emptyset$ 
3: end if
4: compute  $\text{in\_degree}[v]$  for every node  $v$ 
5: initialise  $\text{ancestors}[x] \leftarrow \{x\}$  and  $\text{descendants}[x] \leftarrow \emptyset$  for every node  $x$ 
6: enqueue all source nodes (those with  $\text{in\_degree} = 0$ ) into queue  $Q$ 
7:  $\text{iteration\_sets} \leftarrow \langle \rangle$ 
8: while  $Q$  not empty do
9:    $\ell \leftarrow |Q|$   $\triangleright$  snapshot queue size = one topological level
10:   $\text{current\_set} \leftarrow \emptyset$ 
11:  for  $i = 1$  to  $\ell$  do
12:     $x \leftarrow \text{dequeue}(Q)$ 
13:    add  $x$  to  $\text{current\_set}$ 
14:    for all  $y \in \text{Out}[x]$  do
15:       $\text{ancestors}[y] \leftarrow \text{ancestors}[y] \cup \text{ancestors}[x]$ 
16:      propagate  $y$  as a descendant of every node in  $\text{ancestors}[x]$ 
17:       $\text{in\_degree}[y] \leftarrow \text{in\_degree}[y] - 1$ 
18:      if  $\text{in\_degree}[y] = 0$  then
19:        enqueue  $y$  into  $Q$ 
20:      end if
21:    end for
22:  end for
23:  append  $\text{current\_set}$  to  $\text{iteration\_sets}$ 
24: end while
25: return  $\text{iteration\_sets}, \text{ancestors}, \text{descendants}$ 
    
```

While layering itself is $\mathcal{O}(|V| + |E|)$, tracking the full ancestor and descendant closure within the same pass introduces additional set-propagation cost that depends on the graph’s transitive density. This is a deliberate space-for-time trade-off, justified by the type of set operations the framework performs repeatedly. In the worst case, each node’s closure could contain every other node, giving $\mathcal{O}(n^2)$ total space.

In return, any reachability query reduces to a single set membership test in $\mathcal{O}(1)$, rather than incurring an on-demand $\mathcal{O}(|V| + |E|)$ traversal per query.

Since both sets are populated within the same Kahn’s pass, space is the only additional cost incurred beyond the topological sort itself.

4.3.3 The Unified Graph Object

The framework's unified graph object is a named tuple assembled at the end of input processing, combining the validated outputs of the parser and the graph builder into a single canonical representation. This object is the handoff point from input processing to analysis. For any downstream algorithm (probability propagation, capacity analysis, critical path, diamond decomposition) to be invoked, all arguments can be derived from this tuple. The components of the unified object are split between the shared graph specific fields and the optional analysis-input fields. This is to allow for the same instance of a network to be evaluated under different scenarios even for the same analysis type.

As shown in Table 4.2, the data structure choices throughout reflect the operations the framework actually performs at runtime.

Adjacency indices like the `outgoing_index` and `incoming_index` are stored as `Dict{Int64, Set{Int64}}` because unlike Vectors, Julia's dictionary type gives $O(1)$ average-case lookup without requiring contiguous integer node identifiers. On the other hand, the Set-typed values used in fields like `fork_nodes` and `join_nodes` were chosen to efficiently handle order-insensitive properties that need membership, union, and cardinality operations performed on them.

Tuples are stack-allocated and immutable in Julia so the `Tuple{Int64, Int64}` edge list carries no per-edge heap allocation. Additionally, tuples in Julia are hashed by value so the same `(source, target)` pair maps directly to its entry in any edge-keyed dictionary without auxiliary indexing needed. Iteration sets are stored as `Vector{Set{Int64}}`, since the outer vector position ensures the sets are ordered, but internally each set holds its level members in a container with no positional ordering.

4.3.4 Why not Julia's Native Graph Object?

The Julia language natively supports graph modelling and operations through the `Graphs.jl` package. Component-level information can be attached through complementary packages such as `MetaGraphs.jl` and `NamedGraphs.jl`, all of which are widely used and well-maintained.

At instantiation, IPF immediately validates input data directly from a file path. The input processing module's parser was designed to automatically handle file variation beyond extension type such as excluding header rows and comment lines.

By contrast, building a digraph in `Graphs.jl` with the same input would need to strip these manually because it can only be constructed from a Julia typed matrix or an edge iterator. Since `Graphs.jl` does not have a DAG specific constructor,

Table 4.2: Components of the Unified Graph Object

Component	Type	Role
<i>Base graph — built once, shared across all analyses</i>		
edgelist	Vector{Tuple{Int64, Int64}}	Ordered edge sequence; input to Kahn's in-degree counting
outgoing_index	Dict{Int64, Set{Int64}}	Forward adjacency: node $\rightarrow$ children
incoming_index	Dict{Int64, Set{Int64}}	Reverse adjacency: node $\rightarrow$ parents
source_nodes	Set{Int64}	Nodes with no incoming edges; DAG roots
sink_nodes	Set{Int64}	Nodes with no outgoing edges; DAG leaves
all_nodes	Vector{Int64}	Sorted list of all nodes in the graph
fork_nodes	Set{Int64}	Nodes with fan-out > 1
join_nodes	Set{Int64}	Nodes with fan-in > 1
iteration_sets	Vector{Set{Int64}}	Kahn BFS levels; dependency-safe processing order
ancestors	Dict{Int64, Set{Int64}}	Transitive upward closure (includes self)
descendants	Dict{Int64, Set{Int64}}	Transitive downward closure
<i>Analysis inputs — loaded per invocation, optional</i>		
node_priors	Dict{Int64, T}	Prior value per node; $T \in \{\text{Float64}, \text{Interval}, \text{pbox}\}$
edge_probabilities	Dict{Tuple{Int64, Int64}, T}	Conditional probability per edge
edge_capacities	Dict{Tuple{Int64, Int64}, Float64}	Flow capacity per edge
node_capacities	Dict{Int64, Float64}	Flow capacity per node
node_values	Dict{Int64, T}	CPM node-level weight; $T \in \{\text{Float64}, \text{Interval}\}$
edge_values	Dict{Tuple{Int64, Int64}, T}	CPM edge-level weight

the acyclicity check is no longer guaranteed by the parser and will have to be performed separately.

Although both approaches require an integer-keyed node set, without companion packages such as `NamedGraphs.jl`, `Graphs.jl` enforces a contiguous integer vertex set where our module does not.

Moreover, the iteration sets in the custom object are grouped by topological depth, unlike `Graphs.jl`'s flat ordering. While the flat order is technically a valid topological sort, it does not expose the level structure required for independent operations within a layer which would force all propagation sequences within the framework to always be sequential even when they can be parallelised.

IPF's custom object defines node classifications such as source, sink, fork, and join nodes directly. While `Graphs.jl` exposes degree information, these roles would have to be inferred every time they are needed. Similarly, since `Graphs.jl` does not expose a precomputed transitive closure, any reachability query would require a fresh traversal each time.

These derived properties can technically be added on top of a base `SimpleGraph` by extending it with `MetaGraphs.jl`, attaching iteration sets, closure dictionaries, and node classifications as graph-level properties. This is workable but stretches the package beyond its intended use, since `MetaGraphs.jl` was designed for node-level and edge-level attributes rather than whole-graph structural products. Either way, the framework still needs to compute these properties itself before storing them anywhere.

Additionally, `MetaGraphs.jl` stores properties in Symbol-keyed dictionaries with loosely typed values, which is intentional for flexibility but introduces type instability and triggers dynamic dispatch on every access. Because the input module is purpose-built for this framework, it can enforce strict typing throughout, which gives Julia's compiler more concrete information than a generic metadata dictionary. In the lookup-heavy set operations performed repeatedly during propagation, the custom structure's concrete field types let the compiler resolve these operations statically.

The motivation behind the custom object is the runtime contract: every downstream algorithm receives a validated DAG with iteration sets, closure, and node roles already computed and attached, accessed through one consistent interface. Building this contract requires the same custom logic regardless of whether the base layer is a `SimpleGraph` or a purpose-built object. Thus, using the packages would only redistribute that work into adapter code wrapping the package objects.

The framework's single canonical object also avoids the consistency risk of maintaining derived structures separately from the topology they describe.

Finally, as a general-purpose graph library, Graphs.jl ships with algorithms and utilities the framework does not use, which adds load-time and maintenance cost without operational benefit. More broadly, fewer external dependencies mean less installation friction and a smaller maintenance surface for users adopting the framework.

Chapter 5

Sub-graph Decomposition: Diamond Processing Module

5.1 Introduction

The Input Module returns a validated graph object that carries the network topology together with its layered iteration sets and ancestry closures. All analysis modules in the framework consume this object directly as is.

The Network Decomposition Module, when invoked, extends the graph object with the network’s decomposition structure. Like input processing, this is a network pre-processing step whose objective is to identify all instances of a specified subgraph pattern in the input DAG network.

5.1.1 Decomposition in Complex Networks

Real infrastructure networks are rarely trees. Their components connect to and depend on many others at once, and the interactions between them overlap rather than branch cleanly [45, 46]. Redundancy strategies are the obvious case: a robust network deliberately supplies more than one route to the same goal, so paths diverge and reconverge by design. Process networks and hierarchical flows behave the same way, with a component feeding several downstream dependencies while itself depending on shared upstream outputs. As the system grows, overlapping paths accumulate and the connectivity web thickens.

Graph-based analysis methods almost always do some form of decomposition, and for large networks with complicated connectivity it is a practical necessity [47]. Broadly, decomposition breaks a network into substructures that are easier to handle than the whole, and methods differ in what they do with those substructures. Some strategies aim to reduce: a substructure is replaced by a single equivalent element, so the network shrinks while the analysis stays exact. Other decomposition methods work by partition: the network is divided into units of work sized for the solver, and the pieces are processed in turn. Beyond either, decomposition can give a more compact representation of the network, storing a repeated substructure once rather than every time it appears, without losing the overall structure. Depending on how the decomposed units are defined, they can also support localised investigation, such as confining a sensitivity study to one part of the system without needing to run analysis against the full network.

There is no universal correct unit of decomposition; a decomposition method defines its own target substructure and partitions the network in those terms. In the classic series–parallel reduction, the reliability formula replaces each series or parallel pair with a single equivalent component and the network shrinks pair by pair [13]. The units here are the series and parallel pairs themselves. When the network is not series–parallel, that reducible structure is missing, and the factoring approach manufactures it: conditioning on a single component splits

the problem into the component-working and component-failed cases, and the method recurses on each until it becomes reducible [14, 48, 49]. Polygon-to-chain reductions extend the same idea, widening what can be collapsed in one step beyond the plain series and parallel pairs [15].

In dynamic programming the unit is the tree: a tree decomposition arranges the network into small overlapping bags of nodes connected in a tree, each interacting with the rest of the graph only through its neighbours, which is exactly the subproblem structure a dynamic program can process in a single sweep [50]. Working over bags rather than the whole graph is what makes exact reliability computation feasible on networks as large as metropolitan transit systems [16].

The framework’s default decomposition unit is the *diamond*: a fork-join subgraph where paths in the network fan out from a common node and fan back in at a node further downstream. The framework’s decomposition is partition-forward, that is, each identified diamond is returned as a localised graph object of the same form as the full network, and can be handed to any analysis module in the framework on its own. Secondly, a diamond can serve as a reductive unit, since a solved diamond can stand in for its subgraph as a single equivalent component.

5.2 A Generic Decomposition Module

The network decomposition module identifies and returns every instance of a subgraph pattern discovered in the input DAG, and what counts as the pattern is decided by three discriminator functions. One discriminates the nodes eligible to take part in a pattern at all. One decides what counts as the structure a pattern forms around. One decides when two paths into a node belong to the same pattern. The identification algorithm does the same work whatever the discriminators say.

A discriminator can be defined on any property in the unified graph object, or on any set of network characteristics mapped to a property in the unified graph. Each discriminator can be overridden or modified to fit the required definition. A user can adjust an existing discriminator (e.g. changing the threshold to 3 or more paths), extract a different characteristic of the network altogether (e.g. fan-in-then-fan-out instead of fan-out-then-fan-in), or discriminate on a characteristic that is not already a graph object property, mapped onto one during input processing (e.g. a component type supplied with the raw network, so that only one class of component can anchor a pattern).

In the Julia implementation of the framework, the eligibility discriminator is exposed directly as a runtime override, and the other two are compact internal

functions a replacement definition would swap.

While the module stands as a generic subgraph decomposition tool, the discussion in the rest of this chapter mainly focuses on the framework’s default target, the *diamond* subgraph.

5.3 The Diamond Subgraph

As introduced in Section 5.1.1, the decomposition unit in this work is the diamond subgraph. Diamond patterns are formed when paths diverge from a common fork and reconverge downstream with overlapping ancestries.

Consider a distribution system where two supply routes leave one substation and meet again at a demand point. Diamonds are exactly this diverge and reconverge pattern. An *induced* diamond is a diamond made of one fork and one diamond-forming join. Every one of its paths originates from that single fork, whether there are two of them or four, and the subgraph carries no second diamond join (Figure 5.1). Diamonds are not always induced, and non-diamond paths may still reach the same join node, as in Figure 5.1c. As network interconnectivity increases, so does the likelihood of diamond patterns and their complexity, with diamonds becoming more nested and overlapping each other’s paths (Figure 5.2).

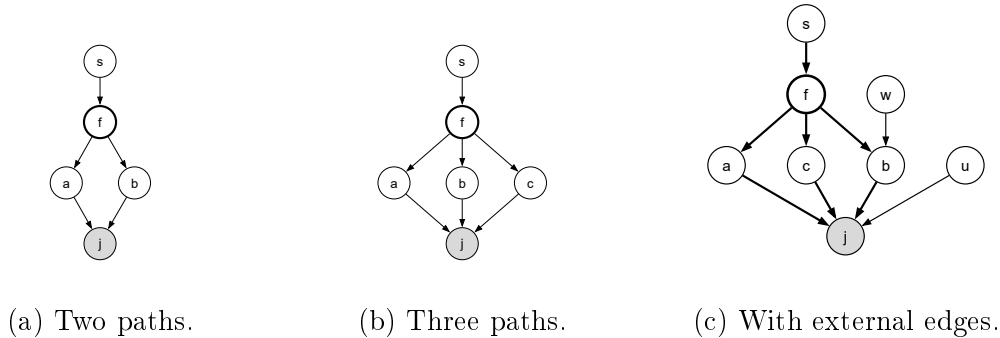


Figure 5.1: Induced diamonds. Every diamond path originates from the single fork and the subgraph carries one diamond join.

The decomposition module consumes the DAG $G = (V, E)$ and the iteration sets $L = \{L_1, \dots, L_k\}$ from the input module’s unified graph object. Given the iteration sets, the topological index $\text{topi} : V \rightarrow \{1, \dots, |V|\}$ follows by numbering nodes in increasing order across layers and in sorted node order within each layer, so every node receives a distinct integer and topi is a strict total order on V refining the layered order.

The diamond in the framework is thus defined by the three discriminators.

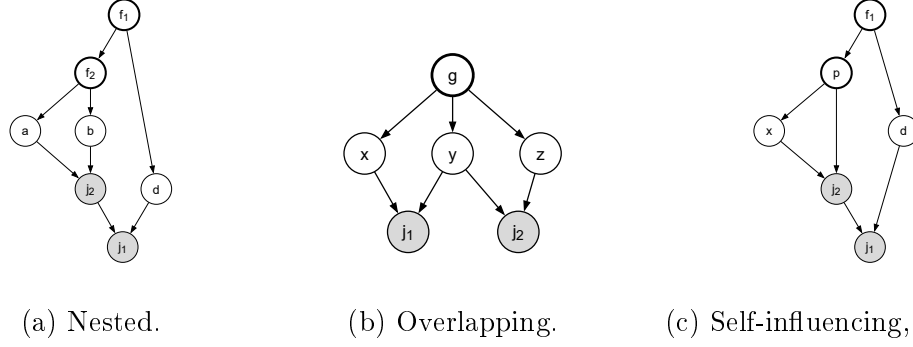


Figure 5.2: Non-induced diamonds. In (b) two diamonds overlap through the shared fork g , and in (c) parent p is itself the fork of the inner pattern.

- Eligibility criterion: the node's declared value must leave its reachability unsettled.
- Anchoring structure: the shared fork from which the reconvergent paths originate.
- Membership criterion: two parents belong to the same pattern when they share ancestry.

The eligibility criterion is the one discriminator not drawn from the diamond's definition, and `is_det` in the implementation is the only place identification reads declared values. Since IPF's decomposition module was built with probabilistic propagation in mind, the eligibility criterion excludes nodes whose prior probability is deterministic. That covers exactly two cases: any inactive node (i.e. unreachable with prior = 0) or a perfect source node (i.e. always active with prior = 1). The conjunction in the second case matters because a non-source node with prior = 1 may still be probabilistically uncertain depending on its incoming edges. Therefore excluding every node valued 0 or 1 would incorrectly disable every diamond in a probabilistic DAG where all node priors happen to be 1. For interval-valued and p-box-valued networks the two cases apply only to degenerate values, an interval or p-box collapsed exactly onto 0 or 1.

Definition 5.1 (Diamond join). *A join node is a diamond join when an unsettled fork lies in the shared ancestry of two of its parents, or one unsettled parent is itself an ancestor of another, so that a diamond forms there. A diamond join can itself be a fork of a different diamond elsewhere in the network.*

Each parent arriving at a diamond join carries its *influencing set*, its ancestry under the self-inclusive closure, the nodes with a path to it and the parent itself. A diamond join can also receive paths that originate outside the fan-out, and these form its non-influencing set.

Definition 5.2 (Non-influencing set). *The non-influencing set of a diamond join is the set of its parents whose influencing sets meet no other parent's. Their edges enter the join from outside every reconvergent pattern there and are carried alongside the join's diamonds as independent contributions.*

Given this, a diamond is defined formally.

Definition 5.3 (Diamond). *A diamond D is a subgraph of G whose relevant node set $R \subseteq V$ spans a diamond join, its correlated parents, and everything upstream of them, together with a set $C \subseteq V$ of fixed nodes that isolate the pattern.*

IPF returns two views of the network's diamond composition.

Unique diamonds are the complete set of distinct diamonds found in the network, each stored once, in `unique_subgraphs`. Any node can participate in any number of unique diamonds, whether as a fan-out point, a fan-in point, or an intermediate node along the paths.

The maximal diamond is the network-level view of a diamond join. It is the full subgraph the join's reconvergence forms, comprising the diamond of every group of the join's parents that forms one, spanning the join, the groups, and everything upstream of them, together with the edges of the join's non-influencing set. Every diamond join carries exactly one maximal diamond, while a fork can participate in several, and every diamond found at deeper levels inside it is a **sub-diamond**, its edge list contained in the enclosing diamond's.

A unique diamond with no inner sub-diamond is itself an induced diamond, the smallest a diamond can be. Since diamonds nest and overlap, the unique diamonds are exactly the distinct maximal diamonds together with their inner sub-diamonds.

5.4 The Identification Algorithm

`identify_diamonds` is a recursive set-membership algorithm over the closures of the unified graph object. For every join node of G it identifies the join's maximal diamond, pairing the join with the earliest covering forks in its parents' shared ancestry, and it recurses inside each diamond it forms for the sub-diamonds. Each call works on a join v , a *fixed context* $E \subseteq V$ of nodes already fixed by enclosing diamonds, and the parents P of v in view. The recursion starts from $E = \emptyset$, and Algorithm 2 shows the skeleton.

Under a context each parent's influencing set shrinks to

$$\text{infl}(p, E) = \text{Anc}(p) \setminus E,$$

with $\text{Anc}(p)$ self-inclusive so that $p \in \text{infl}(p, E)$ whenever $p \notin E$. Two parents p, q of a join are *structurally correlated under E* when $\text{infl}(p, E) \cap \text{infl}(q, E) \neq \emptyset$, that is, when some node upstream of both is not held fixed. Similar to factoring based decomposition, extending the context with a node removes it from every influencing set.

Algorithm 2 Recursive diamond identification (skeleton)

Require: join node v , fixed context E , parents P of v in view

Ensure: the diamonds at v under E

- 1: partition P into groups by pairwise influencing-set intersection
 - 2: **for all** groups G with $|G| \geq 2$ **do**
 - 3: $f \leftarrow$ eligible covering fork of G with minimum topi
 - 4: **if** no eligible covering fork exists **then**
 - 5: members of G join the non-influencing set of v ; skip the group
 - 6: **end if**
 - 7: build diamond D for (v, G, f) under $B = E \cup \{f\}$
 - 8: reuse the stored entry if the identity key of D is already in `unique_subgraphs`
 - 9: **for all** join nodes j within D **do**
 - 10: recurse on $(j, B, \text{parents of } j)$, restricted to G when $j = v$
 - 11: **end for**
 - 12: **end for**
 - 13: singleton groups join the non-influencing set of v
-

5.4.1 Shared ancestry and the earliest covering fork

At a join v with parents P under context E , any two parents whose influencing sets intersect are merged into one group by union-find.

Theorem 5.4 (Grouping correctness). *Fix a join v , context E , and parent set P . Let H be the undirected graph on vertex set P with an edge between p and q if and only if $\text{infl}(p, E) \cap \text{infl}(q, E) \neq \emptyset$. Then `components`(P, E) returns exactly the connected components of H .*

Proof. The intersection $\text{infl}(p, E) \cap \text{infl}(q, E)$ is evaluated for every unordered pair of P , so the pairs that intersect are exactly the edges of H , and union-find merges exactly those pairs. Union-find over a vertex set and its edge relation returns the connected components, which for P under this edge relation is the component partition of H . $\square$

Theorem 5.5 (Group disjointness). *If $G_i \neq G_j$ are distinct groups returned by `components`(P, E), then*

$$\left(\bigcup_{p \in G_i} \text{infl}(p, E) \right) \cap \left(\bigcup_{q \in G_j} \text{infl}(q, E) \right) = \emptyset.$$

Proof. Let $p \in G_i$ and $q \in G_j$. By Theorem 5.4 they lie in different components of H , and since every pair was evaluated, $\text{infl}(p, E) \cap \text{infl}(q, E) = \emptyset$. The intersection of the two unions expands to the union of these pairwise intersections, each of which is empty. $\square$

Disjointness is a statement about the influencing sets of the diamond groups: each group decomposes separately. Whether separately computed values recombine correctly at the join is a different claim, which Chapter 6 makes and proves in its own terms. Parents in singleton groups fall into the join's non-influencing set.

Each group of two or more is then searched for its eligible covering forks, those lying in the shared ancestry of at least two members and not already in the fixed context, with a member that is an ancestor of another counting as its own cover. The diamond anchors on the earliest of them, the covering fork of minimum topological index, which leaves the largest part of the group's shared structure intact downstream for reuse, and the strict total order makes the choice unique. A group whose shared ancestry offers no eligible covering fork forms no diamond, and its members join the non-influencing set.

5.4.2 Recursive flow

Theorem 5.6 (Termination). *The recursion of Algorithm 2 terminates after at most $|V|$ nested context extensions along any recursive path.*

Proof. Along any path of the recursion, each call either returns without recursing, which happens when every group at the join in view is a singleton or offers no covering fork, or fixes a fork f and recurses with $B = E \cup \{f\}$. Covering forks are never already in E (Section 5.4.1), so $f \notin E$ and $|V \setminus B| = |V \setminus E| - 1$ strictly. The quantity $|V \setminus E|$ is a non-negative integer that strictly decreases along the recursive path, and from its initial value $|V \setminus \emptyset| = |V|$ it admits at most $|V|$ decrements. $\square$

Fixing fork f for group G at join v yields the diamond with boundary $B = E \cup \{f\}$. Paths within the diamond can share further unfixed ancestry, so the recursion runs again at every join within it, the diamond join itself included, under the extended context. Sub-diamonds form inside maximal diamonds this way until no covering fork and join pair remains anywhere.

5.4.3 Subgraph construction

The diamond for group G at join v under boundary B spans

$$R = \{v\} \cup G \cup \bigcup_{p \in G} \text{Anc}(p),$$

the join, the group, and everything upstream of the group, with induced edge list

$$\mathcal{E}_D = \{ (u, w) \in E : u, w \in R, w \notin B, (w \neq v \text{ or } u \in G) \}.$$

Write $V(\mathcal{E}_D)$ for the set of its endpoints. The first two clauses keep the edge list on the relevant set R . The third removes every edge into a fixed node, whose upstream structure is already settled, while its outgoing edges stay and act as local sources of the diamond. The fourth admits an edge into the diamond join only from the group, closing a failure mode the first three cannot see.

Lemma 5.7 (Edge isolation). *Let D be constructed for group G at join v under boundary B as above. Then $(u, w) \in \mathcal{E}_D$ if and only if $(u, w) \in E$, $u, w \in R$, $w \notin B$, and $(w \neq v \text{ or } u \in G)$. Consequently every directed path within $\mathcal{E}_D$ that terminates at v has its final edge originating in G , and no node of B appears on any path within $\mathcal{E}_D$ except as its first node.*

Proof. The membership characterisation restates the filter defining $\mathcal{E}_D$. For the path properties, induct on path length. A length-one path into v is a single edge $(u, v) \in \mathcal{E}_D$, which the fourth clause forces to have $u \in G$. Any longer path has a final edge satisfying the same property, and each preceding edge (u, w) has $w \notin B$ and both endpoints in R by the first three clauses, so no node after the first lies in B and no edge of the path enters from outside R . $\square$

Consider a nested level at which an enclosing diamond has already fixed a fork r , so $r \in E$, and suppose r is both an ancestor of some group member p and a direct parent of the join v . Partitioning handles r correctly, since fixing removed r from every influencing set, and in the minimal case of r a source with no free ancestry its influencing set is empty, landing it in a singleton group whose contribution to v enters separately.

The span R , built from the raw closure $\text{Anc}(p)$, contains r no matter what the context says, so partitioning and span silently diverge wherever $E \neq \emptyset$. The edge (r, v) then has both endpoints in R and a head outside B , so the first three clauses admit it, and a diamond meant to capture v as reached through G alone quietly gains a second unrelated entry into its own join. The fourth clause blocks exactly this edge. Figure 5.3 shows the minimal instance.

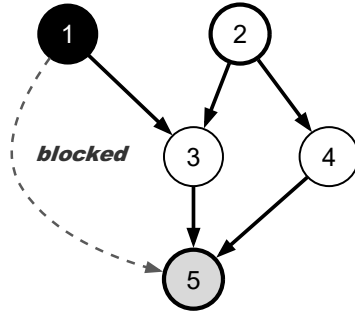


Figure 5.3: Edge isolation at join 5 in context $E = \{1\}$. Node 1 (filled) was fixed by an enclosing diamond and is excluded from the group $G = \{3, 4\}$ with fork 2. Its direct edge $(1, 5)$, drawn dashed, is admitted by the first three filter clauses but blocked by the fourth. Bold edges form the nested diamond’s edge list.

5.4.4 Context-aware identity

A diamond’s edge list alone cannot place it in the nesting hierarchy.

Proposition 5.8. (i) *There exist graphs and contexts $E_A = \emptyset$ and $E_B = \{g\}$, with g a source fork among the endpoints of $\mathcal{E}_D$, under which the construction of Section 5.4.3 anchors the same fork f and yields the same edge list $\mathcal{E}_D$, while one diamond is maximal and the other a sub-diamond. The pair $(\mathcal{E}_D, \{f\})$ therefore conflates distinct diamonds.* (ii) *Under $\text{hkey}(D) = (\mathcal{E}_D, \{f\} \cup (E \cap V(\mathcal{E}_D)))$, equal keys imply the same diamond join, the same group, the same edge list, and the same set C , with the two fixed contexts agreeing on every node of $V(\mathcal{E}_D)$ and every stored field equal.*

For (i), take a join v whose group G anchors on fork f , and a source fork g that is an ancestor of exactly one member of G and keeps one edge whose head lies outside both boundaries. Coverage of one excludes g from the covering-fork search, so the search offers the same forks in both cases and f is anchored in both, and removing g from a single influencing set cannot split the group, since the correlation runs through f . The cut into g removes nothing, as a source has no incoming edges, and g ’s retained edge passes the filter in both cases, so $g \in V(\mathcal{E}_D)$. The group, the fork, the span, and the edge list therefore coincide, while the diamond is maximal under E_A and a sub-diamond of the one that fixed g under E_B . The keys differ, since $E_A \cap V(\mathcal{E}_D) = \emptyset$ and $g \in E_B \cap V(\mathcal{E}_D)$.

For (ii), equal keys give equal $\mathcal{E}_D$ and equal C , since both are hashed. The diamond join is recoverable as the unique endpoint of $\mathcal{E}_D$ that every other endpoint precedes in the closure, which acyclicity makes unique, and the group as the in-neighbourhood of the join in $\mathcal{E}_D$, which the fourth clause admits from the group

alone. Every stored field is computed from $\mathcal{E}_D$, $V(\mathcal{E}_D)$, C , and the global closures, so equal inputs give equal fields, and $B \cap V(\mathcal{E}_D) = (\{f\} \cap V(\mathcal{E}_D)) \cup (E \cap V(\mathcal{E}_D)) = C \cap V(\mathcal{E}_D)$ in both constructions. The two fixed contexts can still disagree away from the diamond, but only narrowly. A node outside the span is never consulted, since every membership test in the recursion concerns a parent of a join within the diamond or one of its ancestors, all inside the span by transitivity. A span node x in one fixed context and not the other has no incoming edge, since such an edge would have its tail in the span and would pass the filter where x is free, forcing $x \in V(\mathcal{E}_D)$, and each outgoing edge of x inside the span is absent from the shared $\mathcal{E}_D$, so its head lies in each construction's boundary or it is a direct edge into the join excluded by the fourth clause. Every path from such a node towards the diamond therefore begins at a node in both fixed contexts or at an excluded edge.

Since a diamond cannot be unique by its edge list and fork alone, every unique diamond is keyed by

$$\text{hkey}(D) = (\mathcal{E}_D, \{f\} \cup (E \cap V(\mathcal{E}_D))),$$

where $\mathcal{E}_D$ is the diamond's edge list, f its covering fork, and $E \cap V(\mathcal{E}_D)$ the fixed context restricted to the diamond's endpoints.

Before a new entry is built, the key is looked up in `unique_subgraphs`, and a structurally identical diamond already resolved anywhere in the recursion is referenced rather than rebuilt. This is hash-consing [17, 51], a table keyed by structural identity so that equal structures are stored once and identity comparison becomes a key comparison, sometimes used in BDD algorithms. Without the store the recursion is a tree and every call site is distinct. With it, structurally identical call sites collapse into shared entries and the representation becomes a DAG. Diamonds with the same identity share one stored entry regardless of the outer context, since Proposition 5.8 bounds whatever influence a node outside the identity carries towards the diamond, and the shared entry meets the same fixed context within.

5.5 Structural Correctness

5.5.1 Global completeness

Theorem 5.9 (Global completeness). *Let v be a join node of G with parent set P , and suppose there exist distinct $p, q \in P$ and an unsettled node a such that either a is a fork node in $\text{Anc}(p) \cap \text{Anc}(q)$, or $p \in \text{Anc}(q)$ (the asymmetric case,*

taking $a = p$). Then the top-level pass of *identify_diamonds* stores a maximal diamond for v . Conversely, a join admitting no such pair receives none.

At the top level $E = \emptyset$, so in either case $a \in \text{infl}(p, \emptyset) \cap \text{infl}(q, \emptyset)$ and the intersection is non-empty. By Theorem 5.4, p and q lie in a common group G with $|G| \geq 2$. In the covering-fork search, a counts once for each member it covers. In the symmetric case a is a fork in both ancestries and covers p and q . In the asymmetric case $a = p$ is necessarily a fork, since it has an edge to v and a path to q that cannot pass through v , and with the self-inclusive closure it covers itself as well as q . Either way the coverage reaches 2 and the set of covering forks is non-empty. A non-empty finite set has a topi-minimal element, so the search anchors some fork, not necessarily a itself, Algorithm 2 constructs a diamond for G , and the top-level pass records it under v . For the converse, suppose no such pair exists. Any covering fork the search admits would yield one, since a symmetric cover is a fork in two members' ancestries, an asymmetric cover is a parent in another member's ancestry, and the search admits only unsettled nodes. Every group therefore offers no covering fork, whether its members are unrelated or related only through settled structure, and the algorithm constructs nothing at v . That is the correct outcome, because whatever shared ancestry remains at v is already settled.

5.5.2 Determinism

Theorem 5.10 (Determinism). *For a fixed input graph and iteration sets (G, L) , join v , context E , and admissible parent set, the diamonds returned by Algorithm 2 are uniquely determined, independent of set or dictionary iteration order, thread scheduling, or any other accident of execution.*

By induction on recursion depth, it suffices that each step of one call is a function of its arguments alone. This holds for the partition step because, by Theorem 5.4, its output is the connected-component partition of H , a graph invariant that does not depend on the order in which pairs are examined or merges performed. It holds for the selection step because the set of covering forks is itself a function of the arguments, and either it is empty, in which case the group deterministically produces no diamond, or the diamond anchors on its minimum under topi, which a strict total order makes unique. The boundary $B = E \cup \{f\}$ and span R are then determined, hence so is $\mathcal{E}_D$, and hence so are the joins passed to the recursive calls, which are deterministic by the inductive hypothesis. The store adds one further mechanism, reuse of an earlier entry on a key hit. Proposition 5.8(ii) makes equal keys carry the same join, group, edge list, and stored fields, and confines any residual difference between merged call

sites to nodes outside the identity, so reuse introduces no dependence on visit order.

This guarantee is similar to the canonicity guarantee of reduced ordered binary decision diagrams [9]. While BDD canonicity depends on an externally supplied variable ordering, *topi* is derived from a property of the input graph.

5.6 The Unique Diamond Object

The algorithm builds each unique diamond as a unified graph object of its own. The unique diamond object carries the diamond’s characteristics together with its subgraph in the form of the unified graph object, so a diamond can be treated as a standalone DAG network for any other type of analysis through the framework, as Table 5.1 shows.

Table 5.1: Fields of the unique diamond object

Field	Role
<code>sub_outgoing_index</code>	Forward adjacency of the diamond
<code>sub_incoming_index</code>	Reverse adjacency of the diamond
<code>sub_sources</code>	Local sources
<code>sub_fork_nodes</code>	Local forks
<code>sub_join_nodes</code>	Local joins
<code>sub_ancestors</code>	Ancestor closure restricted to the diamond’s nodes
<code>sub_descendants</code>	Descendant closure restricted to the diamond’s nodes
<code>sub_iteration_sets</code>	Layered order restricted to the diamond’s nodes
<code>sub_node_priors</code>	Value map restricted to the diamond’s nodes
<code>sub_diamond_structures</code>	For each join within this diamond, its sub-diamonds, held by their keys into <code>unique_subgraphs</code> , and its non-influencing set
<code>is_maximal</code>	Whether the diamond is maximal
<code>diamond</code>	The span R , the set C , and the edge list $\mathcal{E}_D$

Invoking the decomposition module on the unified graph object from the input module extends the object with the field `unique_subgraphs`, holding the network’s unique subgraphs. For this framework, the propagation algorithm of Chapter 6 reduces solved diamonds to supernodes.

5.7 Worked Examples

5.7.1 The induced diamond

Figure 5.1a is the minimal pattern, a source s feeding a fork f , two internally disjoint paths, and a join j where they meet. At j both parents' influencing sets contain f , so the parents form one group. Fork f is the only covering fork and is fixed, and the diamond spans the whole figure with $C = \{f\}$.

5.7.2 A factorised join

The nine-edge network of Figure 5.4 has forks 1 and 2 and a single join at node 7, which source 8 also feeds directly. The parents split into two correlated groups, $\{3, 4\}$ through fork 1 and $\{5, 6\}$ through fork 2, with parent 8 related to neither. Two constituent diamonds arrive at join 7, one anchored on each fork, and node 8 rides along in the non-influencing set. Together they are the join's maximal diamond.

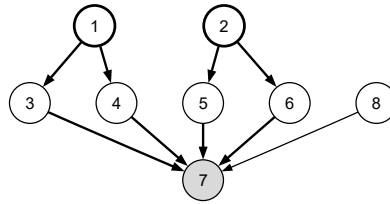


Figure 5.4: A factorised join. Two correlated groups meet at join 7, one through each fork, and source 8 enters from outside both. Bold edges mark the two constituent diamonds.

5.7.3 A nested decomposition

The eight-node network of Figure 5.5 has forks 1 and 3 and joins 6 and 8. An outer reconvergence runs through fork 1 and meets at join 8, and on one of its branches an inner reconvergence runs through fork 3 and meets at join 6. The decomposition below is the machine-checked output of the implementation on this edge list, and the examples of Figures 5.3 and 5.4 were verified in the same run.

Join 8, top level. The parents are $\{2, 7\}$, with $\text{infl}(2, \emptyset) = \{1, 2\}$ and $\text{infl}(7, \emptyset) = \{1, 3, 4, 5, 6, 7\}$. The sets intersect in $\{1\}$, so the parents form one group. Fork 1 is the only covering fork of the pair and is fixed, and the maximal diamond D_1 spans all nine edges with $C = \{1\}$ (Figure 5.6).

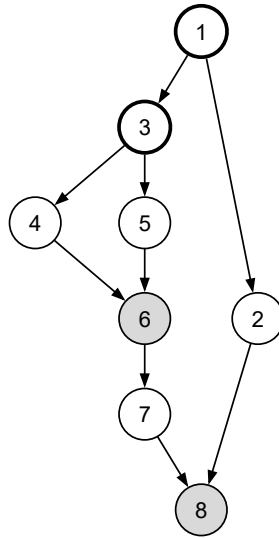


Figure 5.5: Frame 1. The network, with forks outlined in bold and joins shaded.

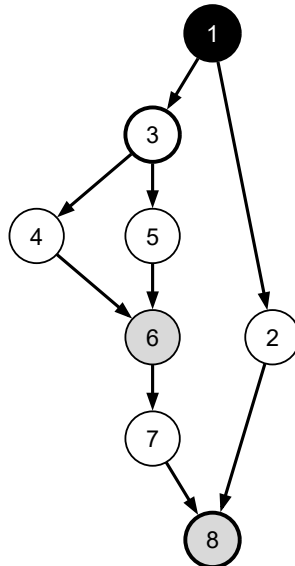


Figure 5.6: Frame 2. The maximal diamond at join 8, with fork 1 fixed (filled) and the span covering the whole network.

Join 6, inside D_1 . Under the boundary $B = \{1\}$, join 6 has parents $\{4, 5\}$ with $\text{infl}(4, \{1\}) = \{3, 4\}$ and $\text{infl}(5, \{1\}) = \{3, 5\}$. The parents are still correlated through fork 3, so fixing 1 did not resolve the inner reconvergence. The recursive call fixes fork 3 and produces the inner diamond D_3 with $C = \{3\}$ and $\mathcal{E}_D = \{(3, 4), (3, 5), (4, 6), (5, 6)\}$. The edge $(1, 3)$ is cut by the boundary clause because its head is fixed at this level (Figure 5.7).

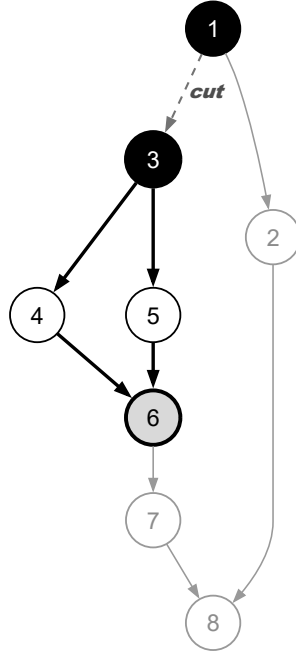


Figure 5.7: Frame 3. The recursive call at join 6 in context $E = \{1\}$, with fork 3 fixed, the inner diamond in bold, and edge $(1, 3)$ cut.

Join 6, top level. Join 6 is also a join of the network itself, so the top-level pass visits it in its own right. With $E = \emptyset$, forks 1 and 3 both cover parents 4 and 5, and the earliest covering fork rule fixes fork 1 rather than fork 3, which a reader tracing by eye might expect. The result is a second maximal diamond D_2 spanning $\{(1, 3), (3, 4), (3, 5), (4, 6), (5, 6)\}$ with $C = \{1\}$, and the recursion inside it, now in context $\{1\}$, again produces exactly D_3 .

The emitted hierarchy. In total the module returns two maximal diamonds and three unique diamonds (Figure 5.8). D_1 sits at join 8 and D_2 at join 6, each with $C = \{1\}$. The inner D_3 at join 6 with $C = \{3\}$ is stored once and referenced from the substructure tables of both D_1 and D_2 under the same key. Two different enclosing recursions posed the same sub-problem, with the same edge list, the same fork, and contexts agreeing on the span, and Proposition 5.8(ii) is the reason the single shared entry is correct.

The trace also confirms the edge-isolation case of Section 5.4.3 end to end. In

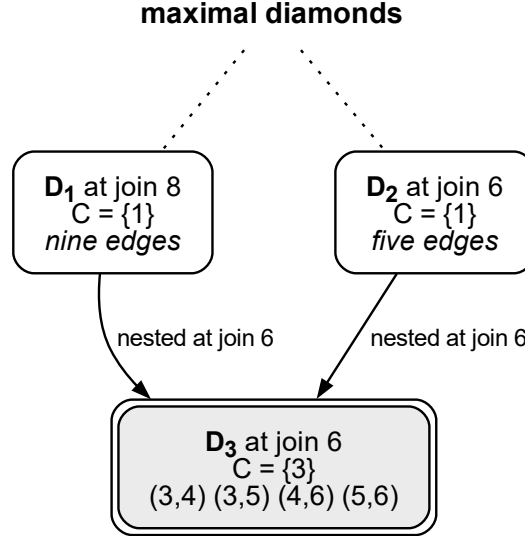


Figure 5.8: Frame 4. The emitted hierarchy of two maximal diamonds, with the inner diamond D_3 stored once and referenced by both.

the network of Figure 5.3, the nested diamond at join 5 under $E = \{1\}$ is emitted with $C = \{2\} \cup (\{1\} \cap V(\mathcal{E}_D)) = \{1, 2\}$, node 1 staying among the edge list's endpoints through its surviving edge $(1, 3)$, which is the context-aware identity visible in the output. Node 1 is recorded in the nested diamond's non-influencing set, and the edge $(1, 5)$ appears nowhere in its edge list. At the top level node 1 is a member of the group itself, so the maximal diamond at join 5 carries $(1, 5)$ legitimately.

5.8 Summary

The module extends the graph object of Chapter 4 with `unique_subgraphs`, storing every diamond found at any level exactly once, each entry a graph object of the same form as the network it came from, with the maximal diamonds flagged. The decomposition terminates within $|V|$ nested steps and is canonical for a given input (Theorems 5.6 and 5.10). It groups a join's parents exactly by shared unfixed ancestry and produces a diamond wherever unsettled reconvergence exists and only there (Theorems 5.4–5.9). Nothing outside a diamond's span enters its edge list (Lemma 5.7), and the context-aware identity keeps apart the diamonds that must be kept apart (Proposition 5.8).

Chapter 6

Information As Probabilities: Probability Propagation Toolkit

6.1 Introduction

Under the probabilistic reading of the network model, the value on a node is the probability that the component operates and the value on an edge the probability that the handover between its endpoints succeeds. The question this reading makes precise is whether information survives the network: with what probability is each node reached from the sources despite component and handover failures. The Probability Propagation Toolkit answers that question exactly, for every node of the network at once, and its propagation algorithm is the framework's namesake, the Information Propagation Algorithm.

The toolkit consumes the unified graph object of Chapter 4 and, wherever the network reconverges, the decomposition structure of Chapter 5. It is the analysis the decomposition module was built for, because reconvergent paths are precisely where local probability calculations stop being valid. Chapter 5 surveyed decomposition as a structural strategy. The concern here is the probability computation itself, and a short account of how that computation has been attempted places the toolkit among its alternatives.

6.1.1 Computing network reliability

The earliest reliability formulations are structural. A reliability block diagram arranges components in series, parallel, and k -out-of- n groups whose system reliability has a closed form [4], which is efficient exactly as far as the system is built from those groups. A network whose routes diverge and reconverge with shared upstream components is not, so the redundancy that motivates the analysis is the structure the formalism cannot hold. Fault and event trees widen the reachable logic by building the failure mechanism explicitly as a tree of gates [5], at the price of manual construction and logical expressions that grow rapidly with the number of interacting pathways, and the tree is a model of one failure event rather than of the network, so every source-to-node question needs its own tree.

Graph-theoretic methods compute on the network directly. The reliability of reaching a node is the probability of a union of path events, so minimal path and cut sets, suitably disjointed, give exact answers, and recursive decomposition methods improve on raw enumeration by splitting the network into subproblems [6, 7]. Their shared difficulty is that path and cut collections grow combinatorially with exactly the redundancy and reconvergence that make the analysis worth doing. Monte Carlo simulation escapes structure entirely and handles arbitrary networks with confidence intervals in exchange for exactness, with sample counts that grow demanding when failures are rare or answers must be tight [8].

Binary decision diagrams brought a different discipline to the exact compu-

tation [9, 10]. Shannon decomposition compiles the Boolean structure function into a canonical decision graph on which reliability is a single traversal, and the approach handles systems with thousands of variables. Its cost is concentrated in the diagram’s size, which depends critically on the variable ordering, and finding a good ordering is itself intractable in general [52]. Reconvergent fan-out is the diagram’s hard case, forcing intermediate nodes to remember which shared ancestors have been decided, and on densely reconvergent networks the construction can exhaust memory under one ordering while remaining compact under another.

The probabilistic-inference literature reaches the same problem from another side. Source-to-node reachability is marginal inference on a directed graphical model, and the exact inference methods are conditioning and clustering: cutset conditioning enumerates the states of a node set whose fixing renders the rest of the structure singly connected [11], and the junction-tree algorithm clusters the network into a tree of overlapping bags [12]. Both are exact and both cost exponential in a structural width of the graph, the cutset size for one and the treewidth for the other. Message passing makes the same inference local and scalable when exactness is relaxed, and reliability-specific variants such as the directed probability propagation method track marginal and pairwise probabilities to approximate the dependency structure [53], gaining scale at the price of approximation error and pairwise bookkeeping that grows with the dependent interactions.

One limitation runs across every exact method above: all of them consume point-valued component reliabilities. In practice failure probabilities are estimated from sparse data, elicited from experts, or quoted as manufacturer bounds, and the literature on imprecise probability represents such knowledge as intervals or as probability boxes [22, 23], bounds on the distribution of the uncertain quantity. None of the exact machinery propagates these forms natively. An interval query against a decision diagram means re-evaluating at chosen input corners, and no analytic route to a distributional bound exists at all.

The toolkit of this chapter is message passing kept exact. Local closed-form updates carry the computation wherever the structure is independent, every reconvergence is resolved by conditioning on the diamonds Chapter 5 identified, and solved diamonds reduce to single equivalent components so that no dependence is ever paid for twice. The cost of exactness lands in the same width-governed class as the decision diagram and the junction tree, which Section 6.8 makes precise and measures. What separates the toolkit from that class is the gap just named: one propagation mechanism, generic over the three value forms of Chapter 4, returning exact beliefs for point inputs, exact ranges for interval inputs, and guaranteed distributional bounds for probability boxes.

6.2 The Belief of a Node

Every component of the network can fail. A node v carries its prior π_v , the probability that the component operates, and an edge (u, v) carries its probability ρ_{uv} , the probability that the handover from u to v succeeds, both attached to the graph object during input processing. Writing X_v and Y_{uv} for the corresponding indicator variables, the independence assumption of Chapter 3 makes all of them mutually independent.

Information reaches a node when some path delivers it, and whether it does is itself an indicator variable. The reachability of v is

$$R_v = X_v \wedge \left(v \in S \vee \bigvee_{u \in \text{Pa}(v)} (R_u \wedge Y_{uv}) \right), \quad (6.1)$$

where S is the source set and $\text{Pa}(v)$ the parents of v . The toolkit computes the *belief* of every node,

$$b(v) = P(R_v = 1), \quad (6.2)$$

the probability that v operates and is reached from at least one source. Unrolling the recursion writes R_v as a disjunction over all source-to- v paths of the conjunction of each path's node and edge indicators, so the belief is equally the probability that at least one path is viable end to end.

Two properties of this quantity govern the whole chapter. Computing $b(v)$ subsumes two-terminal network reliability and is therefore #P-hard [54, 55], so no exact method escapes exponential cost on every network, and the realistic aim is exactness at a cost set by the structure of the particular network rather than by its size. The second property is that R_v is a monotone Boolean function of the component indicators, since restoring a failed component can only create source-to-node connectivity, never destroy it.

Proposition 6.1 (Monotonicity). *$b(v)$ is non-decreasing in every node prior and every edge probability.*

Proof. Realise each indicator from its own independent uniform variable, $X_w = \mathbf{1}\{U_w \leq \pi_w\}$ and $Y_{uw} = \mathbf{1}\{U_{uw} \leq \rho_{uw}\}$. Raising any single π_w or ρ_{uw} can switch its indicator from 0 to 1 but never from 1 to 0, and R_v is monotone in every indicator, so on every realisation R_v can only switch from 0 to 1. Hence $P(R_v = 1)$ cannot decrease. $\square$

6.3 Propagation in a Single Pass

A source is reached exactly when it operates, so its belief is its prior. Every other node receives information through its parents. The *signal* through a parent p of v is the event A_p that a viable path reaches p and the handover to v succeeds, with probability

$$S_p = b(p) \rho_{pv}, \quad (6.3)$$

and v is reached when at least one signal arrives, so

$$b(v) = \pi_v P\left(\bigcup_{p \in \text{Pa}(v)} A_p\right). \quad (6.4)$$

When the signal events are mutually independent the union has the closed form

$$b(v) = \pi_v \left(1 - \prod_{p \in \text{Pa}(v)} (1 - b(p) \rho_{pv})\right), \quad (6.5)$$

which for a single parent collapses to the chain rule $b(v) = \pi_v b(p) \rho_{pv}$.

The iteration sets of Chapter 4 order the computation so that every parent's belief exists before it is consumed. The algorithm initialises the sources, then processes each layer in turn, applying the local update once per node, and Chapter 4's convergence guarantee makes the single forward pass complete in exactly $|L|$ layer steps. There is no iterative refinement to converge and no sampling to average, because every update is an exact probability calculation.

Equation (6.5) evaluates the union as a product of complements rather than by the alternating inclusion-exclusion sum. Under independence the two are identical, but in the product form each signal probability occurs exactly once, which keeps the evaluation numerically stable and, once values become intervals rather than numbers, avoids the artificial widening that repeated occurrences of a variable inflict on interval arithmetic (Section 6.9.1).

Everything so far rests on one hypothesis. The update of Eq. (6.5) is exact precisely when the signal events are independent, and they are independent exactly when the paths feeding the parents share no component upstream. In a tree that is always true. In the systems this framework serves it is false by design, because Chapter 3 established that engineered redundancy is deliberate reconvergence, with duplicated routes sharing the components upstream of the fork that created them.

6.4 Reconvergence Breaks the Local Update

Figure 6.1 shows the smallest network on which the local update fails, a diamond in the sense of Chapter 5. Source 1 has prior 1, every other node has prior 0.9, and every edge has probability 0.9. Every worked number in this chapter is the machine-checked output of the implementation on the stated inputs, verified in the same run against an independent exact computation, exhaustive enumeration of all component states here and exhaustive path enumeration with inclusion-exclusion for the larger example of Section 6.6.1.

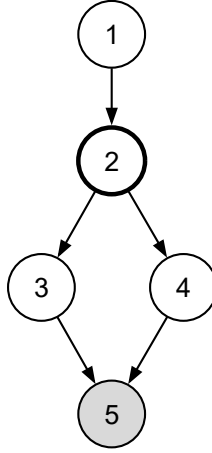


Figure 6.1: The simple diamond. Source 1 feeds fork 2 (bold), whose two paths reconverge at join 5 (shaded).

The forward pass runs cleanly until the join. The fork’s belief is the chain $b(2) = 1 \times 0.9 \times 0.9 = 0.81$, and each intermediate node follows as $b(3) = b(4) = 0.81 \times 0.9 \times 0.9 = 0.6561$, so each signal into the join carries $S = 0.6561 \times 0.9 = 0.59049$. Treating the two signals as independent, Eq. (6.5) gives

$$b(5) = 0.9 (1 - (1 - 0.59049)^2) = 0.749071,$$

while exhaustive enumeration over all component states gives $b(5) = 0.675462$. The local update overestimates the join’s belief by eleven per cent of its true value, and it does so because both signal events contain the fork’s reachability. The two events are positively dependent, their intersection is larger than the product of their probabilities, and a union rule that understates the intersection overstates the union. On a network built deliberately out of redundancy the error is not a corner case but the norm, and it always flatters the system, reporting reliability the structure does not have.

Conditioning on the shared fork repairs the calculation. Given $R_2 = 1$ the two paths share nothing further, so each transmits independently with probability $0.9 \times 0.9 \times 0.9 = 0.729$ and the join receives a signal with probability $1 - (1 - 0.729)^2 = 0.926559$. Given $R_2 = 0$ nothing arrives, since every path into the join runs through the fork. Weighting the two cases by $P(R_2 = 1) = b(2) = 0.81$,

$$P(\text{signal at } 5) = 0.81 \times 0.926559 = 0.750513, \quad b(5) = 0.9 \times 0.750513 = 0.675462,$$

the exact value. Two conditioned sub-problems replace an enumeration over 2^{10} component states, because conditioning is applied only to the one variable whose sharing created the dependence. That economy, exactness at the price of the dependence actually present, is the algorithm's entire design, and on real networks, where diamonds nest, overlap, and arrive several to a join, delivering it takes the machinery the diamond decomposition exists to provide.

6.5 Conditioning at a Join

The example used two facts without naming them, that a belief can be split over the states of other nodes and that fixing the right node makes the parents' signals independent. A third governs the joins the example was too small to show, where several groups of parents arrive sharing nothing between them and each group's contribution must recombine with the others. All three hold in general, and together they justify the algorithm's treatment of any join. The structural objects come from Chapter 5, whose decomposition partitions each join's parents into groups of shared unfixed ancestry, anchors each group's diamond, and sets aside the non-influencing parents. A diamond arrives carrying its set C of fixed nodes, and since the propagation resolves the diamond by conditioning on the reachability states of exactly those nodes, this chapter calls C the *conditioning set*.

Lemma 6.2 (Conditional invariance). *For any set $A \subseteq V$ and any node v ,*

$$b(v) = \sum_c P(R_A = c) P(R_v = 1 \mid R_A = c),$$

where R_A is the vector of reachability indicators of A and the sum runs over its states $c \in \{0, 1\}^{|A|}$. Every term is well defined because each R_u is a deterministic function of the mutually independent component indicators.

Proof. The events $\{R_A = c\}$ partition the sample space, and the display is the law of total probability over that partition. $\square$

Lemma 6.3 (Separator sufficiency). *Let v be a join evaluated within a network in which every node of a fixed set C enters only as a local source whose state is pinned, and suppose the influencing sets $\text{infl}(u, C) = \text{anc}(u) \setminus C$ of v 's parents within that network are pairwise disjoint. Then, conditional on $R_C = c$, the parents' signal events are mutually independent, and*

$$b(v \mid R_C = c) = \pi_v \left(1 - \prod_{u \in \text{Pa}(v)} (1 - P(R_u = 1 \mid R_C = c) \rho_{uv}) \right).$$

Proof. Every path into a parent u either starts at a node of C , whose pinned state replaces everything upstream of it, or runs through components of $\text{infl}(u, C)$ alone. Conditional on $R_C = c$, the signal event through u is therefore a function of the constant c , of the indicators of the nodes of $\text{infl}(u, C)$, of the edges among them, and of the edge (u, v) . The influencing sets are pairwise disjoint, so these indicator sets are pairwise disjoint and mutually independent, and functions of disjoint sets of independent variables are independent. The display is Eq. (6.5) applied conditionally. $\square$

The hypotheses are not assumptions, because Chapter 5 discharges both at every site where the propagation applies the independent update. Its edge isolation lemma guarantees that a fixed node appears in a diamond's edge list only as a local source, its outgoing edges kept and its upstream structure cut, and its grouping theorem certifies disjoint influencing sets, with the recursion extending the conditioning one covering fork at a time until the parents in view are disjoint in exactly this sense. The conditional beliefs the display consumes are the *contextual beliefs*, the values each node holds within the sub-network currently being evaluated, which are conditional on the ambient states by construction because they were computed there.

The remaining question is the one Chapter 5 deferred, whether the contributions of separately decomposed groups recombine correctly at the join. Its group disjointness theorem gives distinct groups pairwise disjoint influencing sets under the current context, and disjoint influence is exactly independence.

Lemma 6.4 (Recombination across groups). *Fix a join v and a context E whose nodes' states are given, and let $G_1, \dots, G_m$ be the parent groups of the decomposition's partition at v , together with the singleton groups forming the non-influencing set. Write E_k for the event that some parent in G_k delivers a signal to v . The events $E_1, \dots, E_m$ are mutually independent, so*

$$P(\text{signal at } v) = 1 - \prod_{k=1}^m (1 - P(E_k)),$$

and each group is resolved on its own conditioning set alone.

Proof. Each E_k is a function of the indicators of the nodes in $\bigcup_{p \in G_k} \text{infl}(p, E)$, of the edges among them, and of the edges from G_k into v . By group disjointness the node sets are pairwise disjoint, hence so are the edge sets they induce, and the given context enters every E_k only as fixed constants. The events are therefore functions of pairwise disjoint sets of mutually independent indicators, hence mutually independent, and the display is the union rule for independent events. $\square$

The lemma is also where the non-influencing set of Chapter 5 earns its treatment. A non-influencing parent is a singleton group, its influencing set meeting no other's, so its signal enters the product as one more independent factor at the cost of a single multiplication, with no conditioning at all. A join can therefore gather several diamonds and several plain parents, and the recombination stays a one-line product over independent contributions.

Recombination across groups is where the decomposition pays for itself. A join whose m groups carry conditioning sets $C_1, \dots, C_m$ is resolved in $\sum_k 2^{|C_k|}$ conditioned sub-problems, where conditioning on all the groups jointly would take $2^{\sum_k |C_k|}$. The gap is widest at high fan-in over independent structure. On an adversarial family in which k independent reconvergent branches meet at one join, the implementation resolves exactly $2k + 1$ conditioned sub-problems for every k tested up to 16, linear where joint conditioning would be exponential.

What remains at a single group is the enumeration itself. Writing $C = \{a_1, \dots, a_m\}$ for the group's conditioning set, total probability gives

$$P(E_k) = \sum_{c \in \{0,1\}^m} \left(\prod_{i=1}^m b(a_i)^{c_i} (1 - b(a_i))^{1-c_i} \right) \psi(c), \quad (6.6)$$

where the weights are built from the conditioning nodes' contextual beliefs and $\psi(c)$ is the probability that the group delivers a signal to v given the state c . A conditioning state does not simplify the diamond to a formula. It poses a smaller instance of the whole problem, and the algorithm answers it accordingly.

6.6 Supernodes and Progressive Decomposition

A diamond arrives from Chapter 5 as a graph object of the same form as the network itself, and the propagation exploits that self-similarity directly. For each state c of the conditioning set, the algorithm poses the diamond as a network in its own right and runs on it the same single pass it runs on the whole. The local

sources of that sub-network take their values from the enclosing computation, a conditioning node pinned to its state and every other local source keeping its contextual belief. The join's own prior is deferred, set to 1 inside the sub-problem so that the sub-run returns the probability that a signal arrives, with the join's prior applied exactly once by the enclosing update. The value the sub-run returns at the join is $\psi(c)$.

The map $c \mapsto \psi(c)$ is all the enclosing computation ever needs from the diamond. Materialising that observation gives the algorithm its central reduction: the solved diamond is replaced by a *supernode*, a single equivalent component that delivers a signal to the join with probability $\psi(c)$ under each state of its conditioning set.

Lemma 6.5 (Supernode equivalence). *Let D be a diamond at join j with conditioning set C , solved in its context to the map ψ_D . Replacing D by a supernode that delivers a signal to j with probability $\psi_D(c)$ under each state c leaves the belief of every node downstream of j unchanged.*

Proof. Fix a state c . Conditional on $R_C = c$, the event that D delivers a signal to j is a function of the indicators internal to D , which are independent of every indicator outside D . The enclosing computation only ever conditions on the nodes of its own conditioning sets, each of which enters the sub-network containing D as a local source, upstream of D or beside it, so each such conditioning event is a function of indicators outside D and leaves $P(\text{signal at } j \mid R_C = c) = \psi_D(c)$ unchanged. A downstream belief depends on D only through the signal event at j , entering by Lemma 6.2 as the pair of the weights $P(R_C = c)$ and the conditionals $\psi_D(c)$, and the supernode preserves both. $\square$

The proof's independence step needs the enclosing computation to fix no node internal to D , and the decomposition guarantees exactly that. When an outer conditioning does reach inside a subgraph, the context-aware identity of Chapter 5 makes the inner structure a different diamond with its own stored entry, so the sub-problem solved is always the one whose context actually holds.

Nesting makes the reduction progressive. Within a state's sub-run, an inner join with unresolved shared ancestry carries its own diamond, recorded by the decomposition in the enclosing diamond's substructure table, and the sub-run resolves it by the same conditioning, from the inside out. Termination is inherited from Chapter 5, whose recursion bounds the nesting by strictly growing the fixed context, and exactness follows by induction on nesting depth, each level applying Lemmas 6.2 to 6.5 within its own sub-network, which satisfies the same model assumptions as the whole.

Two mechanisms keep the recursion from repeating itself. A conditioned sub-problem is identified by its diamond’s edge list together with the local source values its state assigns, and every solved sub-problem is stored under that identity, so a structurally identical sub-problem posed anywhere else in the recursion is answered from the store rather than recomputed. Separately, a conditioning node whose contextual belief is already pinned to 0 or 1 by the enclosing state carries no remaining uncertainty, so its enumeration collapses to the single state of non-zero weight. The collapse is exact because the discarded state has weight exactly $b = 0$ or $1 - b = 0$ in Eq. (6.6).

6.6.1 A multi-level worked example

The network of Figure 6.2 exercises everything at once: three sources, three forks, nested diamonds, a join fed by a diamond and a non-influencing parent together, and a target whose reconvergence encloses every other structure. Sources S_1 , S_2 , S_3 have prior 1, every other node prior 0.9, and every edge probability 0.9.

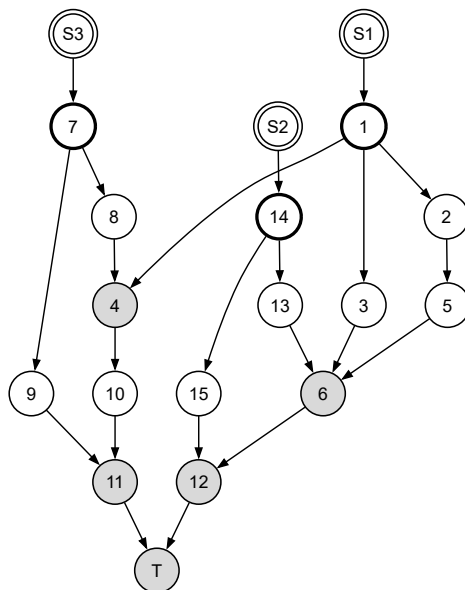


Figure 6.2: The multi-level worked network. Sources are double-bordered, forks 1, 7 and 14 bold, joins 4, 6, 11, 12 and the target T shaded.

The decomposition of Chapter 5 returns four maximal diamonds. D_1 at join 6 anchors on fork 1, covering the reconvergent parents 5 and 3, while parent 13 arrives from S_2 ’s side sharing nothing with them and lands in the join’s non-influencing set. D_2 at join 11 anchors on fork 7 and D_3 at join 12 on fork 14. D_4 at the target anchors on fork 1, because the target’s two parents, 12 and 11, share exactly node 1 in their unfixed ancestry, and within D_4 ’s span the reconvergences

at joins 11 and 12 recur as sub-diamonds. Join 4 receives parents 1 and 8 whose ancestries are disjoint, so it needs no diamond anywhere, and the plain update of Eq. (6.5) handles it at every level.

Table 6.1 traces the four resolutions in the terms of Sections 6.5 and 6.6. Each is a two-state enumeration whose weight is the conditioning node's contextual belief, here 0.81 in all four cases since forks 1, 7 and 14 each sit one perfect source, one edge and one prior away.

Table 6.1: Conditioning trace of the worked network. w is the conditioning node's contextual belief, $\psi(1)$ and $\psi(0)$ the conditional signal probabilities the diamond's sub-runs return at the join, and $P(E)$ the group's recombined signal $w\psi(1) + (1-w)\psi(0)$.

Diamond	Join	C	w	$\psi(1)$	$\psi(0)$	$P(E)$
D_1	6	{1}	0.81	0.889023	0	0.720108
D_2	11	{7}	0.81	0.893745	0.478297	0.814810
D_3	12	{14}	0.81	0.931860	0.583288	0.865631
D_4	T	{1}	0.81	0.913943	0.818038	0.895721

At join 6 the diamond's contribution recombines with the non-influencing parent by Lemma 6.4: parent 13 delivers its signal with probability $0.6561 \times 0.9 = 0.590490$ independently of the group, so the join receives a signal with probability $1 - (1 - 0.720108)(1 - 0.590490) = 0.885382$ and $b(6) = 0.9 \times 0.885382 = 0.796843$. The other three joins carry a single group each, so their beliefs follow directly from the table, $b(11) = 0.733329$, $b(12) = 0.779068$, and $b(T) = 0.806149$.

The dead states of D_2 , D_3 and D_4 show why the conditioning is a genuine weighting rather than an on-off switch. With fork 7 down, join 11 is still reached through node 1's side at probability 0.478297, with fork 14 down, join 12 is still reached through S_1 's side at 0.583288, and with fork 1 down the target still collects the S_2 and S_3 routes at 0.818038. Each of those values is itself the output of a full sub-network propagation in which the fork is a pinned local source, and inside D_4 's sub-runs the reconvergences at joins 11 and 12 are resolved by their own nested diamonds under the extended context.

The store is visible in the run, which poses twenty conditioned sub-problems of which sixteen are distinct. The four savings are the inner reconvergence at join 6, which D_3 's recursion meets again inside each of its states in exactly the form D_1 already solved, because node 14 lies outside that sub-problem's span, so the extended context is invisible to its identity and the store answers all four posings. The nested diamonds inside D_4 show the opposite case: node 1 lies inside their spans, its pinned state is part of their identity, and each is stored under both outer states, four entries apiece rather than two. The same subgraph is reused

where the context cannot reach it and kept apart where it can, which is the context-aware identity of Chapter 5 doing its work at propagation level. Every belief above agrees with exhaustive path enumeration under inclusion-exclusion to floating-point round-off.

6.7 Validation

Every guarantee this chapter states has been checked against implementations that share none of the propagation’s machinery. The exactness oracle is a reduced ordered binary decision diagram with dynamic variable reordering, built independently for each network, with exhaustive path enumeration where it is tractable and Monte Carlo estimation beyond. Across a corpus of 129 random and mutated DAGs, six topological families, larger random networks, and real and published benchmark networks, the propagated point beliefs agree with the oracle to a worst per-node deviation of 1.1×10^{-16} , under both perfect and imperfect component regimes. The propagated intervals coincide with the oracle’s corner evaluations on all 129 graphs to at worst 2.8×10^{-16} , and the probability-box bounds were checked against Monte Carlo ground truth across fifty configurations spanning topology, input shape, and uncertainty width, with zero soundness violations. The measured cost of the corpus runs, alongside the oracle’s, is the evidence base of the next section.

6.8 Complexity Analysis

The cost of the propagation is concentrated entirely in the diamonds, because everything outside them is the linear-time update of Eq. (6.5) applied once per node. A diamond with conditioning set C and edge list E_D is resolved by $2^{|C|}$ sub-runs, each a single pass over its edge list, so one resolution costs $2^{|C|} \cdot O(|E_D|)$, and the total work sums this over every sub-problem the recursion poses. The sum ranges over posings, not over unique diamonds. Nesting re-poses the same diamond under many enclosing states, as the worked example showed on a small scale, and on deeply nested networks the posing count exceeds the unique-diamond sum by orders of magnitude, so a budget computed over unique diamonds alone would understate the work.

Both refinements of Section 6.6 act directly on this sum. The store collapses the posings to the genuinely distinct sub-problems, which across the validation corpus is between one and five per cent of the unmemoised posing bound, with the reduction factor a property of each network’s nesting rather than a constant, so the posing bound is a sound a-priori budget, computable from the decomposition

before any propagation runs, and the realised cost is typically far below it. The factorisation of Lemma 6.4 attacks the exponents themselves, replacing one joint enumeration by a sum of small ones, and its measured effect on the adversarial fan-in family, exactly $2k + 1$ sub-problems against a joint 2^k , is the difference between linear and exponential in the fan-in.

What no refinement removes is the width. The governing parameter is the largest conditioning set the network forces, and the algorithm’s position among the exact methods of Section 6.1.1 is fixed by it. The propagation is a specialisation of cutset conditioning [11] to source-to-node reachability, with two departures from the classical scheme. Where cutset conditioning selects one global set whose enumeration renders the whole network singly connected, the propagation identifies a local separator at each reconvergence actually encountered and recurses, so the realised cost is a sum of local exponentials rather than one global exponential, and tree-shaped regions are crossed at linear cost without touching any enumeration. And where the classical scheme conditions jointly, the factorisation refuses to enumerate independent forks together. Neither departure changes the complexity class. The largest conditioning set is bounded by the size of the largest set of mutually entangled shared ancestors, a quantity of the same nature as the treewidth that governs the junction tree [12] and the pathwidth that governs a well-ordered decision diagram [9], and all three methods are exponential in their width because #P-hardness leaves no other shape available. No closed-form cost in the node count exists for any of them.

The measurements bear the placement out. Figure 6.3 plots the maximum conditioning set against the logarithm of the reordered decision diagram’s node count across the corpus: the two track each other, seven against $\log_2 290 \approx 8.2$ on a published sixteen-node benchmark grid [53], because both quantities measure the same structural width of the network. Realised work tells the same story with no winner. On some networks the propagation resolves fewer sub-problems than the diagram holds nodes and on others more, and no family in the corpus systematically favours either method for exact point-valued computation. Wall-clock comparisons move the crossover around without changing the conclusion. On a reconvergence-saturating adversarial family the propagation is faster up to moderate width and the diagram leads beyond it, on square meshes both grow exponentially with the diagram’s exponent smaller, and on the fan-in family the propagation’s linear sub-problem count wins outright. Where networks pass beyond the practical range, the two methods fail by different resources: on a reconvergence-dense Bayesian-network benchmark of 413 nodes and 265 joins, the decomposition’s identification exhausts memory while the diagram’s construction exhausts time, its reordering still compressing a diagram already past two and a

half million nodes when the budget expires. Every exact method pays the width somewhere, and what differs is the resource and the point of failure.

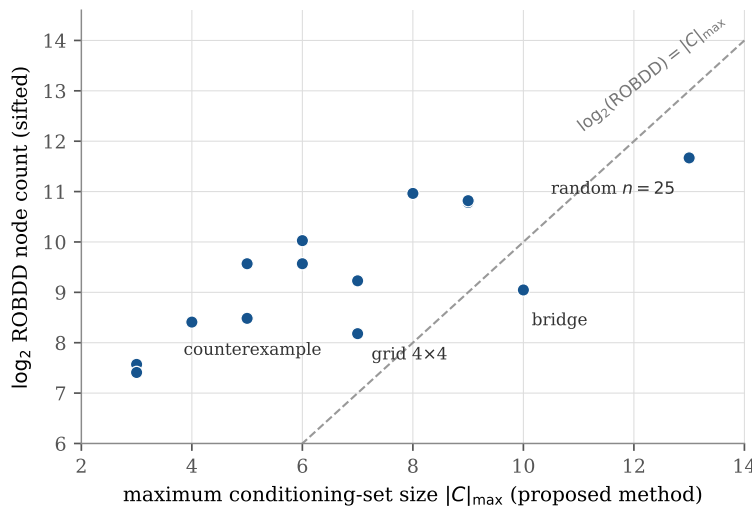


Figure 6.3: Maximum conditioning-set size against the logarithm of the reordered decision diagram’s node count across the complexity-validation networks. Both quantities measure the same structural width, and neither method dominates.

The honest conclusion is that for exact point-valued reliability the propagation is competitive with a well-ordered decision diagram and not superior to it. Its case does not rest on a smaller state space. It rests on the two chapters of capability that follow, which decision diagrams do not offer at all.

6.9 Propagating Uncertain Probabilities

The inputs of a reliability analysis are rarely known exactly. Failure probabilities come from sparse operational records, manufacturer tolerances, and expert judgement, and Chapter 3 gave the framework three value forms to carry those states of knowledge: a deterministic value asserting full knowledge, an interval asserting only bounds, and a probability box bounding a distribution. The usual fate of the honest forms is to be collapsed before the analysis begins, a point estimate chosen and the uncertainty deferred to an ad-hoc sensitivity study afterwards. The toolkit takes the opposite route and propagates the stated form itself, because everything the propagation does to a value is one of three operations, complement against one, product, and the weighted combinations of Eq. (6.6), and none of the machinery of Sections 6.5 and 6.6 requires the values to be numbers. The framework realises the propagation once, generically, over all three forms. What is guaranteed of the output differs by form, and the next two sections establish the guarantee for each uncertain form in turn: the exact

range for intervals, and sound distributional bounds for probability boxes.

6.9.1 Interval inputs are propagated exactly

An interval input asserts that a component’s reliability lies between two bounds and nothing more, and the correct output for a node is then the exact range of beliefs attainable over all inputs consistent with the stated bounds. The monotonicity of Section 6.2 makes that range available outright.

Proposition 6.6 (Corner exactness). *Let every node prior and edge probability be given as an interval, $\pi_v \in [\underline{\pi}_v, \bar{\pi}_v]$ and $\rho_{uv} \in [\underline{\rho}_{uv}, \bar{\rho}_{uv}]$. The exact range of $b(v)$ over all inputs consistent with the intervals is*

$$[b(v) \text{ at all lower endpoints, } b(v) \text{ at all upper endpoints}].$$

Proof. By Proposition 6.1, $b(v)$ is non-decreasing in every input coordinate, so over a box its minimum is attained where every coordinate sits at its lower endpoint and its maximum where every coordinate sits at its upper endpoint. $\square$

No dependency problem arises and no corner search is needed, two consequences worth pausing on because neither is typical of interval computations. A general function over an input box takes its extremes at unknown corners among the 2^n available, and naive interval arithmetic on it over-approximates whenever a variable occurs more than once. Here monotonicity names the two extreme corners outright, and the propagation is arranged so that the arithmetic between them stays exact. Its complement and product are exact interval operations, the product form of Eq. (6.5) presents each signal to the arithmetic exactly once, and the conditioning combination is evaluated at the endpoint values of each conditioning belief, which suffices because Eq. (6.6) is multilinear in each of them and a multilinear function attains its extremes over a box at its corners. Every step returns an enclosure of the true range, and across the full validation corpus the propagated interval coincides with the two corner propagations to machine precision, at worst 2.8×10^{-16} in either bound over all 129 graphs. On the worked network with every non-source prior and every edge widened to $[0.85, 0.95]$, the propagated belief at the target is $[0.634002, 0.931254]$ and agrees with the two corner runs exactly, to the last floating-point digit. Structural certainties are kept degenerate in such a widening, a perfect source stays $[1, 1]$, because degrading a certainty into an uncertainty does not perturb the problem, it changes which nodes can carry conditioning and poses a different and harder one.

The conditioning machinery matters as much here as for point values. An interval propagation that ignores reconvergent dependence stays sound but over-

widens, by up to 0.45 of the unit range on the corpus, wide enough to turn an informative bound into a vacuous one. Exactness of the range is a property of conditioning, not of interval arithmetic alone.

Carrying intervals is nearly free. The propagation costs a factor of about 1.2 over point propagation, a few extra floating-point operations per step, and for a one-shot query, producing a network’s belief range from scratch, it completed faster than the decision-diagram route to the same range, building the diagram once and evaluating its two corners, on every one of eight corpus families tested, by factors between 3.9 and 99.9. The claim is scoped to the one-shot cost, where the diagram’s construction counts against it, and a workload that amortises one diagram across many repeated queries on a fixed network is a different and untested scenario.

6.10 Probability-Box Propagation

A probability box is the strongest of the three input forms, a pair of bounding distribution functions carrying partial statistical knowledge of an uncertain reliability [22], and propagating it promises the strongest output: bounds on the entire distribution of a node’s belief, from which any decision quantity over that distribution can be bounded in turn. The promise comes with one genuinely new mathematical difficulty, and this section locates it, resolves it, and is explicit about what is proved and what is not.

6.10.1 The conditioning combination is not a convolution

The chain and union steps of the propagation raise no new issue for distributional values, because the quantities they combine are independent by the same structural discipline that makes the point computation exact, and p-box arithmetic under independence is standard [23]. The difficulty sits exactly at the conditioning combination. Taking the conditioning nodes one at a time, Eq. (6.6) is the convex combination

$$W \cdot A + (1 - W) \cdot B,$$

where W is the conditioning node’s contextual belief and A , B the conditional results of its two states. All three are now uncertain quantities, and two dependencies bind them. A and B are functions of the same upstream reliabilities, so they are dependent on each other, and the two coefficients are one random variable, not two. Convoluting the terms as if independent ignores both bindings, and the consequence is not mild conservatism but unsoundness: probability mass escapes the unit interval, by as much as 0.34 on the benchmark grid of Section 6.8.

A distributional result that assigns mass outside $[0, 1]$ bounds nothing, so the operator has to respect the combination’s structure.

The framework’s operator integrates over the weight’s own uncertainty. At each discretisation level of W ’s bounding functions it scales the two branch distributions by w and $1 - w$ and combines them under an explicit bound on their dependence, mixes the results across levels, and envelopes the mixture built on W ’s lower bounding function with that built on its upper. Applied one conditioning node at a time, the nested two-way combination reproduces the full mixture over all $2^{|C|}$ states. The output is finally intersected with $[0, 1]$, a projection that discards only values a probability cannot take and so can tighten but never invalidate a bound, and the projection is guarded, raising an error rather than clipping silently if the excursion ever exceeds floating-point round-off by orders of magnitude, so that a genuine soundness regression can never hide inside the projection.

6.10.2 Two dependence bounds, one proved and one open

The dependence bound between the branches is where provability and tightness part company, and the two must not be conflated with each other or with soundness itself. Sound means the computed pair of bounding functions contains the true distribution, and a sound result is never wrong. Tight means the pair is narrow, and a sound result can still be too wide to use. The choice of bound trades one against the other.

Bounding the branches by the Fréchet–Hoeffding inequalities is valid under any joint dependence whatsoever, so the resulting belief bounds inherit soundness from a classical theorem [23] with no proof burden here, at the price of width, and on hard cases the width can approach vacuity. The framework’s default blend instead restricts the dependence bound to non-negative dependence, on the ground that A and B are both non-decreasing functions of the shared upstream reliabilities and so cannot move against each other. The restricted blend is tighter and never vacuous in practice, and it has produced no soundness violation in any of the fifty validation configurations checked against Monte Carlo ground truth. But the motivating argument is not a proof. The dependence-bound literature offers a strict trichotomy, independence, a fully specified dependence, or no assumption at all, with no theorem for positive-but-otherwise-unknown dependence, and an attempted derivation through the natural ordering-of-dependence route meets a genuine obstruction rather than a technicality. Soundness of the restricted blend therefore stands as an open question, stated as such, and the framework keeps the distinction operational: the restricted blend is the default, and the Fréchet

blend is used wherever a claim must rest on proof alone.

6.10.3 What the bounds are worth

Soundness never varies across the validation sweep, and tightness does, so the honest characterisation of the output is an envelope rather than a single figure. Figure 6.4 sweeps input-distribution shapes, uncertainty widths, and reconvergence regimes on the benchmark grid: the bound band is near 0.18 of the unit range under high component reliability and weak reconvergence, degrades to about 0.70 under deep reconvergence with broadly uncertain components, and is driven by the regime and the input width rather than by the shape of the input distributions, with zero violations anywhere in the sweep. Deep reconvergence under broad uncertainty is precisely where the branch dependence is strongest and any distribution-free bound on it loosest, so the envelope’s shape is the mathematics showing through, not an artefact to engineer away.

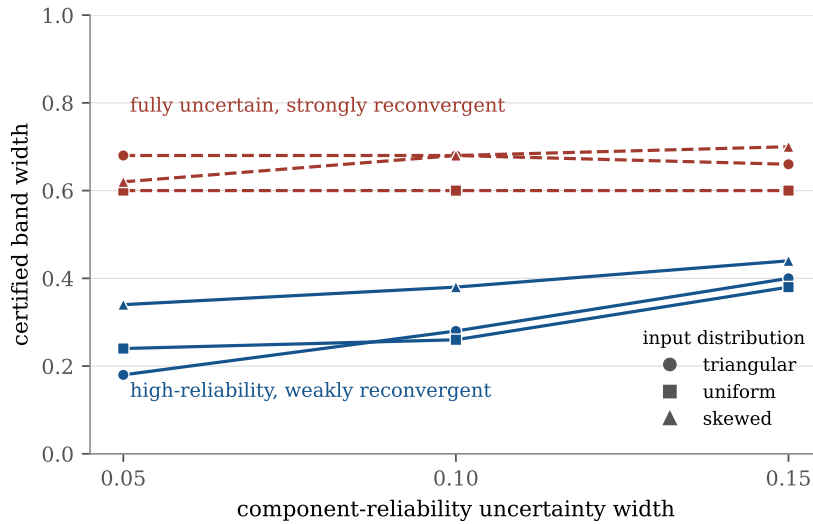


Figure 6.4: Tightness envelope of the propagated probability-box bound on the benchmark grid: band width against component uncertainty width, across input-distribution shapes and reconvergence regimes. Soundness holds in every configuration, and only tightness varies, driven by regime and width rather than input shape.

The practical payoff is a certificate no point-valued exact method produces. The output bounds the distribution of $b(v)$, so for a stated reliability requirement x^* it bounds $P(b(v) \leq x^*)$, the probability that the node fails its requirement, analytically and from a single propagation. Table 6.2 shows the certified band on that probability at a representative requirement of $x^* = 0.95$ on the benchmark grid, between 0.02 and 0.12 wide across the regimes tested. Matching that precision by simulation, planned conservatively without foreknowledge of the answer,

costs between 267 and 9,604 samples of the network’s uncertain inputs, and the result remains a statistical estimate rather than a bound. The planning figure is the honest one to quote, because the retrospective sample requirement collapses whenever the true probability sits near 0 or 1, correct statistics but knowable only in hindsight. A decision diagram has no analytic route to this quantity at all and would itself fall back on simulation.

Table 6.2: Certified bound on the requirement-violation probability $P(b(v) \leq 0.95)$ at the benchmark grid’s target node, from one propagation, against the Monte Carlo sample count required to match the certified precision under worst-case a-priori planning.

Regime	Uncertainty width	Certified band	Band width	MC samples
perfect components	0.05	[0.00, 0.02]	0.02	9,604
perfect components	0.10	[0.00, 0.10]	0.10	385
perfect components	0.15	[0.00, 0.12]	0.12	267
uncertain components	0.05–0.15	[0.98, 1.00]	0.02	9,604

6.10.4 The cost of the distribution

Distributional propagation is not cheap and is not claimed to be. Its cost is a dial, the discretisation level of the bounding functions, and grows roughly cubically with it, measured at 1.2, 6.0, 39 and 276 seconds at levels 25, 50, 100 and 200 on a fifteen-node reference network, in exchange for proportionally tighter bands. The bounds are sound at every level, so the level buys precision, never correctness, and levels between 25 and 100 are the practical band on networks of the scale the exact method suits. The conditioning depth itself is unchanged across all three value forms, so imprecision never relaxes the structural cost of Section 6.8, and a network too reconvergent for exact point propagation is equally beyond exact interval or distributional propagation. Where a network supports exact propagation but not the distributional dial, the exact interval range together with sampling is the practical recourse.

6.11 Case Study: A Medical Drone Logistics Network

The capabilities of this chapter come together on a real planning problem. Jones et al. designed a conceptual medical drone delivery network for Scotland [56], optimising station placement and network structure over real hospital, airport, and

candidate-station locations, with two drone types, a short-range vertical take-off type with a nominal range of 70 km and a long-range fixed-wing type with a nominal range of 700 km, trading off delivery time, capital cost, and resilience. Their own resilience analysis works with point probabilities. The question their framework structurally cannot answer, and this toolkit can, is how confidently each location in a candidate design can be called reachable from the supply hubs once the acknowledged uncertainty in the component availabilities is carried honestly through the network.

6.11.1 Inputs, traced to their source

Every input of the reliability model derives from the source study, with one clearly flagged extension. Location availability follows the study’s own resilience assumption, hub locations always available and every other location failing independently with probability 0.2. The hub availability is kept at exactly 1, and the non-hub availability becomes the interval $[0.75, 0.85]$ centred on the study’s figure, a deliberate choice because that figure is stated as an assumption rather than derived from data, and an interval input turns the whole analysis into a built-in sensitivity assessment of precisely that assumption. A connection exists, per drone type, exactly when the inter-location distance is within the type’s stated nominal range, the study’s own construction rule applied unchanged, and where both types serve a connection the two transmissions combine as independent alternatives.

The flagged extension concerns weather. The source study holds wind conditions constant in its own analysis and defers weather-driven range variation as an uncertainty that would need expert elicitation. The reliability model extends exactly that deferred point and nothing else: a connection operating comfortably within a drone’s range is treated as reliable, and a connection whose length falls in the final ten per cent of the range, an illustrative derating margin since the study quantifies none either, receives the vacuous interval $[0, 1]$, the honest statement that adverse weather may or may not close that particular route. Direction, finally, follows the supply logic. The study’s connections are undirected because a route can be flown either way, but the reliability question concerns supply reaching facilities from hubs, so the dependency graph is directed from the hub tier outward, matching the study’s own hub-and-spoke architecture.

6.11.2 Three designs and the redundancy boundary

The study’s actual optimised layouts are not published, so three configurations stand as proxies for the qualitative character of three of its described trade-off points: a fixed-wing-reliant centralised design serving remote locations through a

hub backbone, 217 locations and 263 connections, a densely interconnected short-range design, 242 locations and 1,753 connections, and a minimal-investment design retaining only pre-existing infrastructure, 230 locations and 1,648 connections. They are proxies for a described character, not reproductions of an unavailable result, and every conclusion below attaches to them as such.

Constructing the dense designs runs directly into the boundary this chapter has been pricing. Connecting every operationally plausible pair of locations produces a network whose identification alone did not complete, exhausting the memory of the analysis machine within minutes, so unrestricted connectivity sits beyond the practical range of exact analysis outright. The design parameter that controls the boundary is a genuine engineering quantity, the number of alternate routes provisioned per location, a redundancy choice and not a parameter of the method. Provisioning sixteen routes per location brings the conditioning width of the dense designs to 16 and 17, deliberately just inside the practical ceiling of around eighteen that earlier experience with the method located, and provisioning beyond sixteen changes neither the width nor the runtime materially, so the reconvergent structure of this network saturates there. The centralised design, tree-like by construction, carries width 10. Full exact interval propagation completes in under 25 seconds on the dense designs and well under a second on the centralised one.

The same boundary was measured for the decision-diagram alternative on the same networks, with identical inputs, and the two methods agree to floating-point precision everywhere both complete. At low redundancy both are comfortably fast, and the one-shot interval computation of the propagation is faster, by roughly a factor of fifty on the centralised design and fifteen on the minimal design provisioned at six routes. Raising the provision, the diagram construction completes up to twelve routes per location, at a build of some 123,000 nodes in 45 seconds, and did not complete within a documented budget from fourteen routes, under dynamic reordering, while the propagation's runtime stays near five seconds across the same sweep. The claim this supports is deliberately narrow. Both methods are governed by the same structural width, and on this one real network family the propagation's practical range, measured along a real redundancy parameter, extends further than the diagram's, with the crossover bracketed between twelve and fourteen routes rather than located.

Distributional propagation has its own, tighter boundary on the same family, exactly as Section 6.10.4 prices it. The full probability-box analysis completes in seconds on the centralised design and in minutes on the minimal design provisioned at six routes, and at eight routes it did not complete within an hour, so the distributional dial is affordable precisely on the sparser designs, and interval

propagation, exact and near-free, is what scales to the dense ones.

6.11.3 What the propagated bounds say

Table 6.3 collects the propagated reliability bounds, and they are informative in exactly the way the designs differ. In the centralised design every location’s band is narrow, at most 0.10 wide with a mean of 0.089: reachability essentially inherits the availability band assumed for the locations themselves, because a tree accumulates little dependence, but it also enjoys no protection, since any single-route failure disconnects service outright. The dense designs spread. Well-connected locations are reachable with near-certainty, and the worst-served facility, Islay Hospital, an island site, carries a reachability probability bounded between 0.56 and 0.72 in both dense designs. That interval is the honest answer to the planning question under the acknowledged input uncertainty, and it is a different kind of answer than a point analysis can give: a facility whose reachability is confidently 0.6 and one whose reachability is anywhere between 0.56 and 0.72 call for different interventions, better routes for the first, better component data before committing capital for the second, and no point-valued analysis of the same design can tell them apart. Figure 6.5 maps the dense decentralised design coloured by the lower reliability bound and by band width. The uncertainty concentrates at island and far-northern facilities, which is where additional redundancy, or better availability data, buys the most, and the map is the toolkit’s output translated back onto the geography the source study planned over.

Table 6.3: The three proxy designs: structure, conditioning width, and propagated interval reliability bounds. Every belief is contained in $[0, 1]$ with lower bound at most upper bound. The worst-served facility in both dense designs is Islay Hospital.

Design	$ V $	$ E $	width	mean band	max band	worst facility
fixed-wing centralised	217	263	10	0.089	0.100	[0.750, 0.850]
dense decentralised	242	1753	17	0.094	0.161	[0.562, 0.722]
concentrated minimal	230	1648	16	0.093	0.161	[0.562, 0.722]

The case study also returns something to the design process itself. More provisioned redundancy improves resilience and simultaneously raises the cost of verifying it exactly, and on this network that trade-off is now a measured curve rather than a caution: the width saturates at sixteen routes per location, exact verification is practical there, and the point at which the alternative exact method stops being practical sits measurably earlier. A designer choosing between these configurations can see, in the same analysis, which facilities are uncertain, why

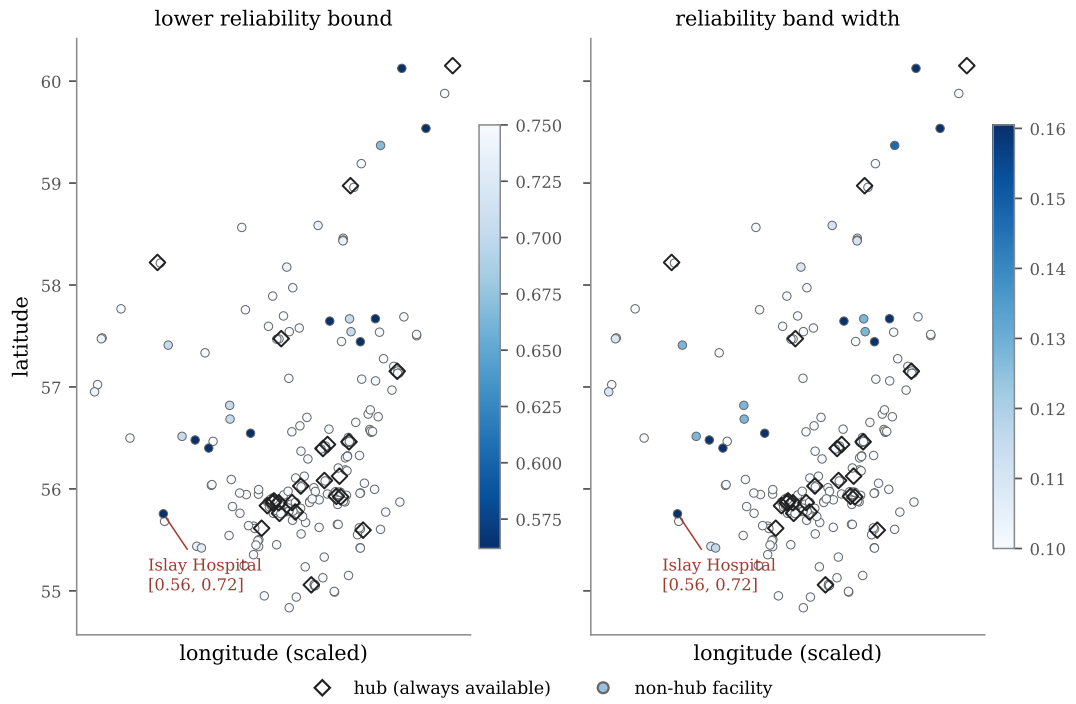


Figure 6.5: Reliability map of the dense decentralised design: each facility coloured by the lower bound of its propagated reachability (left, darker is less certainly reachable) and by the width of its reliability band (right, darker is more uncertain). Hubs, drawn as diamonds, are assumed always available. Uncertainty concentrates at island and far-northern facilities, and the worst-served facility, Islay Hospital, is bounded at $[0.56, 0.72]$.

they are uncertain, and what verifying a more redundant design will cost.

6.12 Summary

The toolkit computes the belief of every node, the probability that it operates and is reached from a source, in one pass over the layered order, applying the closed-form update wherever parent signals are independent and resolving every reconvergence by conditional enumeration on the diamonds of Chapter 5. Lemmas 6.2 to 6.5 carry the exactness argument: total probability splits a belief over conditioning states, the diamond’s fixed nodes make parents conditionally independent, disjoint groups recombine by the independent union, and a solved diamond stands in for its subgraph as a supernode, which discharges the two claims Chapter 5 deferred to this chapter. The cost is exponential only in the conditioning width the network forces, the same parameter that governs cutset conditioning, junction trees, and well-ordered decision diagrams, and for point-valued inputs the method is competitive with, not superior to, that established class. Its distinctive capability is that the same propagation carries the framework’s uncertain value forms natively, returning the exact belief range for interval inputs by monotonicity, and guaranteed distribution bounds for probability-box inputs through a conditioning operator built on the Fréchet–Hoeffding inequalities, certifying decision-relevant probabilities from a single propagation. On a real medical logistics network those capabilities produce what the chapter set out to produce, an honest reliability map under acknowledged input uncertainty, and a measured account of what exactness costs and where it ends.

Chapter 7

Information As Resources: Capacity Flow Toolkit

7.1 Introduction

The reachability toolkit focuses on network survivability and connectivity probability. It computes the probability that at least one active source-to-sink route survives random component failures in a given DAG network.

For some infrastructure reliability problems, this is a sufficient performance measure. In flow-carrying network models, however, the performance measure of the system also depends on the rate of resource flow rather than just the existence of an active path. This reachability module broadly aims to answer the network query of "is the network connected?" However, in networks that also model flow, knowing the likelihood of a signal arriving does not necessarily answer the query of "how much gets through".

Consider a resource distribution network where all source-to-sink paths satisfy the reachability survival threshold. Despite the network being considered connected, deliverable throughput at target nodes can still fall well short of demand if it enters a degraded capacity state. In flow-carrying networks, computing reachability alone cannot fully distinguish between a network with reachability but low capacity and one with reachability plus plenty of spare capacity at target nodes.

Using the fixed edge capacities stored on the extracted graph object in the InputProcessing step, the network flow toolkit consists of a collection of classical max-flow-based modules intended to support analysis of deliverable throughput, bottleneck structure, and the effect of structural or parametric degradation in capacitated DAG models. Specifically, given a capacitated directed acyclic graph as input, the toolkit is designed to address the following analysis objectives:

- Q1.** What maximum throughput can the network deliver under current capacities?
- Q2.** Which cut sets form the binding bottlenecks on that throughput?
- Q3.** Which component failures or capacity perturbations most reduce deliverable flow?
- Q4.** What degradation or upgrade threshold is required to maintain a target demand level?
- Q5.** How much edge and node redundancy exists in path-disjoint terms?
- Q6.** Which components are structural single points of failure?

7.2 Mathematical Model and Network Flow Semantics

Let

$$G = (V, E)$$

be a directed acyclic graph with edge-capacity function

$$c : E \rightarrow \mathbb{R}_{\geq 0},$$

where c_{uv} is the maximum admissible flow on edge $(u, v) \in E$. When node capacities are modeled, let

$$b : V \rightarrow \mathbb{R}_{\geq 0}.$$

Here b_v denotes the maximum admissible through-flow at node v . Define the flow variable

$$f : E \rightarrow \mathbb{R}_{\geq 0},$$

with $f_{uv} := f((u, v))$ for each edge $(u, v) \in E$. For node v , define

$$\delta^+(v) := \{(v, w) \in E\}, \quad \delta^-(v) := \{(u, v) \in E\}$$

as the outgoing and incoming edge sets, respectively. For a capacitated network model G with designated source and sink nodes $s, t \in V$, the primary objective is to compute the maximum deliverable throughput from s to t , subject to edge capacities c_{uv} for $(u, v) \in E$ and, when modelled, node through-flow bounds b_v for $v \in V$.

First consider a base-case network: a single-source single-sink capacitated DAG, shown in Fig. 7.1. Node s is the source, node t is the sink, and intermediate nodes relay flow under capacity constraints. The network performance measure must consider and quantify deliverable throughput from s to t , not just s - t path existence.

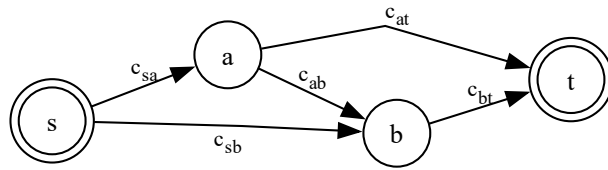


Figure 7.1: Minimal single-source single-sink capacitated DAG network.

For this base case, let $f : E \rightarrow \mathbb{R}_{\geq 0}$ and $F \in \mathbb{R}_{\geq 0}$ be decision variables, where F is the total delivered flow from s to t . The optimization model is

$$\max F$$

subject to

$$0 \leq f_{uv} \leq c_{uv}, \quad \forall (u, v) \in E, \quad (7.1)$$

$$\sum_{e \in \delta^-(v)} f_e - \sum_{e \in \delta^+(v)} f_e = 0, \quad \forall v \in V \setminus \{s, t\}, \quad (7.2)$$

$$\sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e = F, \quad (7.3)$$

$$\sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e = F. \quad (7.4)$$

Taking F^* to be the optimal (maximum feasible) value of F . Then if the sink node t has a required demand level D_t , the flow module quantifies service adequacy through the throughput margin

$$\Delta_t := F^* - D_t,$$

where $\Delta_t \geq 0$ means that the network can meet demand with residual margin, and $\Delta_t < 0$ indicates a throughput shortfall of $|\Delta_t|$.

Multi-Source Multi-Sink Reduction

Let the source set be $S \subseteq V$ and the sink set be $T \subseteq V$, with $|S| \geq 1$ and $|T| \geq 1$. Figure 7.2a shows the original multi-source multi-sink network.

By introducing a super-source s^* and a super-sink t^* , as shown in Fig. 7.2b, the reduced graph is

$$V' = V \cup \{s^*, t^*\},$$

$$E' = E \cup \{(s^*, s) : s \in S\} \cup \{(t, t^*) : t \in T\}.$$

The connector capacities are set to non-binding values (implemented as Inf in the solver):

$$c_{s^*s} = +\infty \quad \forall s \in S, \quad c_{tt^*} = +\infty \quad \forall t \in T.$$

Proposition 7.1 (Exactness of the super-terminal reduction). *The optimal value of the multi-source multi-sink max-flow problem on G is equal to the optimal value of the single-source single-sink max-flow problem on the reduced graph $G' = (V', E')$ with terminals s^* and t^* .*



(a) Original multi-source multi-sink network.

(b) Equivalent super-source/super-sink reduction.

Figure 7.2: Graph reduction of a multi-source multi-sink capacitated DAG to an equivalent single-source single-sink max-flow problem.

Proof. Let f be any feasible flow for the original multi-terminal problem with total value F .

We may define an augmented flow f' on G' by retaining the values of f on all original edges, setting

$$f'_{s^*s} := \sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e \quad \forall s \in S,$$

and

$$f'_{tt^*} := \sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e \quad \forall t \in T.$$

Since the connector edges are guaranteed to be non-binding, then all capacity constraints from original network remain satisfied. Naturally, flow conservation holds at every original transit node, while the net outflow from s^* and the net inflow to t^* are both equal to F . Therefore f' is a feasible s^*-t^* flow of value F .

Conversely, let f' be any feasible s^*-t^* flow on G' : then, restricting f' to the original edge set E gives a flow f on G . Again, conservation on all nodes in $V \setminus (S \cup T)$ is inherited directly from the reduced problem, and the super-terminal connectors account exactly for aggregate injection at the sources and aggregate withdrawal at the sinks. Therefore f is feasible for the original multi-terminal problem and has the same total value as f' . Since feasible flows map in both directions without changing objective value, the two optimal values are equal. $\square$

Thus, solving the reduced super-source/super-sink problem is equivalent to solving the original multi-terminal problem, which is:

$$\max F$$

subject to

$$0 \leq f_{uv} \leq c_{uv}, \quad \forall (u, v) \in E, \quad (7.5)$$

$$\sum_{e \in \delta^-(v)} f_e - \sum_{e \in \delta^+(v)} f_e = 0, \quad \forall v \in V \setminus (S \cup T), \quad (7.6)$$

$$\sum_{s \in S} \left(\sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e \right) = F, \quad (7.7)$$

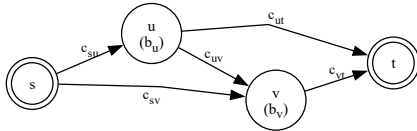
$$\sum_{t \in T} \left(\sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e \right) = F. \quad (7.8)$$

Node-Capacitated Extension via Node Splitting

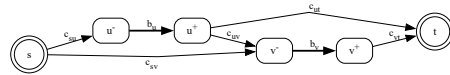
The Information Propagation Framework supports both node and edge network metadata. For capacity analysis, this allows for modelling explicit throughput constraints on nodes themselves. As shown in Figure 7.3, this work adopts a node-splitting approach where each constrained node $v \in V_c$ is replaced by an input node v^- and an output node v^+ , connected by an internal edge:

$$(v^-, v^+), \quad c_{v^--v^+} = b_v.$$

The original incoming edges of v are redirected to v^- , and every original outgoing edge is emitted from v^+ .



(a) Original network with edge and node capacities.



(b) Equivalent node-split edge-capacitated network.

Figure 7.3: Node-splitting transformation.

This extends the edge-capacitated flow model with

$$\sum_{e \in \delta^-(v)} f_e \leq b_v, \quad \forall v \in V_c, \quad (7.9)$$

$$\sum_{e \in \delta^+(v)} f_e \leq b_v, \quad \forall v \in V_c. \quad (7.10)$$

Here $V_c \subseteq V$ may include source and sink nodes when terminal throughput limits are part of the model.

The optimization model then becomes

$$\max F$$

subject to

$$0 \leq f_{uv} \leq c_{uv}, \quad \forall (u, v) \in E, \quad (7.11)$$

$$\sum_{e \in \delta^-(v)} f_e - \sum_{e \in \delta^+(v)} f_e = 0, \quad \forall v \in V \setminus \{s, t\}, \quad (7.12)$$

$$\sum_{e \in \delta^-(v)} f_e \leq b_v, \quad \forall v \in V_c, \quad (7.13)$$

$$\sum_{e \in \delta^+(v)} f_e \leq b_v, \quad \forall v \in V_c, \quad (7.14)$$

$$\sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e = F, \quad (7.15)$$

$$\sum_{e \in \delta^-(t)} f_e - \sum_{e \in \delta^+(t)} f_e = F. \quad (7.16)$$

7.3 Available Modules

Where required, the flow toolkit applies the super-terminal reduction automatically during the baseline flow solve, and the node-capacitated analysis invokes node splitting internally whenever node-capacity data are supplied. Public outputs are then remapped back to the original graph and original source/sink sets, so edge flows, sink allocations, and cut-side node sets remain interpretable in the user’s native network model.

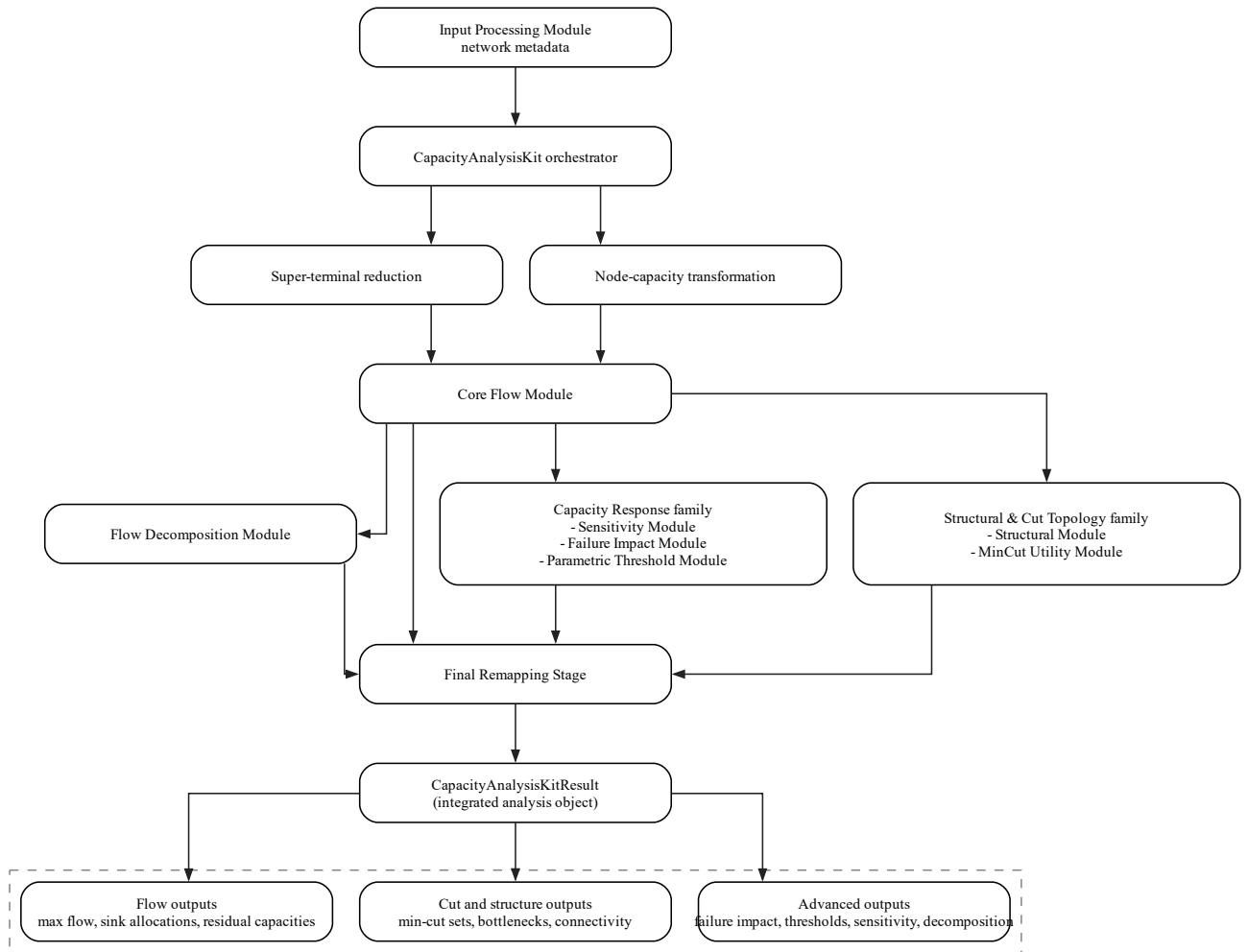


Figure 7.4: Runtime workflow for the flow-capacity toolkit with available modules

7.4 Base Flow Module

Motivating question: “How much flow can the network sustain, and what limits it?”

Inspired by Julia’s `Graphs.jl` and Python’s `NetworkX` flow analysis packages, the flow engine supports three solver variants to compute F^* for a given capacitated network G , summarised in Table 7.1.

Solver	Complexity	Best Fit
<code>:edmonds_karp</code>	$O(VE^2)$	Small networks
<code>:dinic</code>	$O(V^2E)$	Larger layered networks
<code>:push_relabel</code>	$O(V^2E) - O(V^3)^a$	Dense, high fan-out networks

^a Upper bound depends on node selection strategy; this solver does not enforce a highest-label invariant.

Table 7.1: Max-flow solver comparison.

7.4.1 Max-Flow Min-Cut Solvers

The intuition behind Ford-Fulkerson’s [19, 57] Max-Flow Min-Cut (MCMF) theorem is that given a capacitated network, the maximum permissible flow through the network is restricted by the smallest blocking bottleneck set of edges. That is, the maximum possible flow from $s \rightarrow t$ given G is equal to the sum of flow across the min-cut.

Definition 7.2 (Residual Graph). *Given a capacitated network $G = (V, E)$ with capacity function $c : E \rightarrow \mathbb{R}_{\geq 0}$ and a feasible flow f , the residual graph $G_f = (V, E_f)$ is defined over the same node set V with residual edge set*

$$E_f = \{(u, v) \in E : c_{uv} - f_{uv} > 0\} \cup \{(v, u) : (u, v) \in E, f_{uv} > 0\},$$

where the residual capacity on each edge is given by

$$r_{uv} = \begin{cases} c_{uv} - f_{uv} & (u, v) \in E \quad (\text{forward residual}), \\ f_{vu} & (v, u) \in E \quad (\text{backward residual}). \end{cases}$$

A forward residual edge (u, v) represents spare capacity on an original edge; a backward residual edge (v, u) represents flow on (u, v) that can be cancelled. An augmenting path P in G_f is then a directed path from s to t along which $r_{uv} > 0$ for every edge $(u, v) \in P$.

Algorithm 3 Ford-Fulkerson Min-Cut Max-Flow

```

1: Initialise residual graph  $G_f \leftarrow G$  with zero flow on all edges
2: while there exists an augmenting path  $P$  from source  $s$  to sink  $t$  in  $G_f$  do
3:    $\Delta \leftarrow \min_{e \in P} c_f(e)$  ▷ Bottleneck capacity along  $P$ 
4:   for each edge  $e = (u, v) \in P$  do
5:      $f(u, v) \leftarrow f(u, v) + \Delta$  ▷ Push flow forward
6:      $f(v, u) \leftarrow f(v, u) - \Delta$  ▷ Update reverse edge
7:   end for
8:   Update  $G_f$  to reflect new residual capacities
9: end while
10: Let  $A$  be the set of nodes reachable from  $s$  in  $G_f$ 
11: Min-cut =  $\{(u, v) \mid u \in A, v \notin A\}$ 
12: return Max-flow value  $|f|$ , min-cut set  $A$ 

```

As described in 3, the Ford-Fulkerson-based solvers work by iteratively finding and pushing flow along augmenting paths. Thus, most of the execution time is spent repeating four main lookups:

- Capacity on any edge $u-v$
- Current flow on any edge $u-v$
- Outgoing edges from any node u for graph search expansion and forward residual updates
- Incoming edges to any node u for backward residual updates and inflow queries

The graph object from the input processing module already stores this information as part of the network structure and therefore does not need to be recomputed on every call. Both Ford-Fulkerson-based solvers use this lookup structure but differ at step 2 in how the augmenting path is found.

Edmonds-Karp. The Edmonds-Karp solver uses a repeated breadth-first search (BFS) to find the shortest augmenting path in the residual graph. However, since each iteration requires an explicit path reconstruction via a stored parent map, the per-iteration overhead is higher than the Dinic variant, and the $O(VE^2)$ bound makes it poorly suited for larger and denser networks. It is therefore best reserved for smaller networks where simplicity and interpretability are preferred over performance.

Dinic. To avoid the one-path-at-a-time limitation of the Edmonds-Karp method, the Dinic solver alternates between level graph construction via BFS and a

blocking-flow depth-first search. The level graph restricts traversal to edges $u \rightarrow v$ where $\text{level}[v] = \text{level}[u] + 1$, reducing the search space to a layered subgraph per phase. Furthermore, to avoid the $O(E)$ per-path rescanning cost of naive DFS, this variant uses a pointer cursor $\text{ptr}[u]$ to track each node's position in its adjacency list, permanently skipping exhausted edges.

The Dinic solver is the best fit for most hierarchical DAG networks and is the framework's default when no user-specified option is selected. However, on very dense graphs with $O(V^2)$ edges and high structural redundancy, the per-phase BFS reconstruction becomes costly, and the blocking flow DFS must still handle a larger fraction of edges despite the pointer optimisation. In these cases, the Push-Relabel solver is recommended, as it operates locally without global path searches.

7.4.2 Push-Relabel Solver

Unlike the two path-based solvers, the Push-Relabel (PR) variant does not search for augmenting paths at all. This solver works by resolving excess flow locally from an initialised pre-flow. Here, instead of a residual graph per phase, a merged neighbourhood list is built once over the augmented network and shared across all push and relabel operations. The algorithm proceeds as follows:

Algorithm 4 Push-Relabel Min-Cut Max-Flow

```

1: height[ $s^*$ ]  $\leftarrow |V|$ ;   height[ $v$ ]  $\leftarrow 0 \forall v \neq s^*$ 
2: Push maximum admissible flow from  $s^*$  to all neighbours; initialise excess[ $v$ ]
3: while  $\exists$  active node  $u$  with excess[ $u$ ]  $> 0$  do
4:   if  $\exists$  neighbour  $v$  with  $r(u, v) > 0$  and height[ $u$ ] = height[ $v$ ] + 1 then
5:      $\delta \leftarrow \min(\text{excess}[u], r(u, v))$ 
6:     Push  $\delta$  units from  $u$  to  $v$ ; update excess[ $u$ ], excess[ $v$ ],  $r(u, v)$ ,  $r(v, u)$ 
7:   else
8:     height[ $u$ ]  $\leftarrow 1 + \min_{v: r(u, v) > 0} \text{height}[v]$   $\triangleright$  Relabel
9:   end if
10: end while
11: Traverse final flow state from  $s^*$  to identify min-cut partition
12: return Max-flow value  $|f|$ , min-cut set

```

7.5 Network Flow Diagnostics

7.5.1 Min-Cut Utility Module

Motivating question: “How constrained is the bottleneck, and in how many equivalent ways?”

The flow module solvers all compute the max flow F^* by constructing a representative valid min-cut S^* in the given network G . This output by itself neither indicates nor guarantees that S^* is the only possible min-cut set in G .

This distinction matters in practice for some engineering decisions. Infrastructure systems are designed with robustness and redundancy in mind. In these networks it is more common to expect multiple structurally distinct S/T partitions of V that all have the same max flow F^* value.

While the baseline max flow value directly addresses the first analysis objective, the utility module exploits the already-generated min-cut and residual graph to build the extended partition S^{**} and compute the full space of valid min-cuts.

Let $R_T \subseteq V$ be the set of nodes reachable via backward traversal from t^* in G_f .

Definition 7.3 (Extended Source Partition). *The extended source-side region is*

$$S^{**} = V \setminus R_T,$$

restricted to original graph nodes. Any node outside R_T cannot reach the sink under the current flow state, so absorbing it into the source partition never increases cut capacity. Consequently $S^ \subseteq S^{**}$.*

Definition 7.4 (Free Zone). *Given the extended partition, the free zone is*

$$\mathcal{F} = S^{**} \setminus S^*.$$

Nodes in $\mathcal{F}$ can be assigned to either side of a minimum cut without affecting its capacity. Every minimum cut of G under f^ takes the form $S = S^* \cup R$ for some $R \subseteq \mathcal{F}$.*

The free zone size $|\mathcal{F}|$ itself determines the degeneracy of the bottleneck structure. Specifically, when $|\mathcal{F}| = 0$ the minimum cut computed from the flow solver is unique and S^* is the only valid partition. For $|\mathcal{F}| > 0$ there are exactly $2^{|\mathcal{F}|}$ equivalent minimum cuts, meaning the cut boundary is not pinned to a single partition.

Both the extended partition S^{**} and free zone $\mathcal{F}$ allow for a more complete picture of the bottleneck edge structure beyond what the representative cut alone provides. It then follows that two distinct edge sets can be derived directly from from the lattice.

The set of edges appearing in at least one minimum cut is

$$E_{\text{some}} = \{(u, v) \in E : f_{uv}^* = c_{uv}, u \in S^{**}, v \notin S^*\},$$

and the set appearing in every minimum cut is

$$E_{\text{every}} = \{(u, v) \in E : f_{uv}^* = c_{uv}, u \in S^*, v \in T^{**}\},$$

where $T^{**} = V \setminus S^{**}$, and $E_{\text{every}} \subseteq E_{\text{some}}$ holds by construction.

Min-Cut Enumerator

Given the residual lattice, the enumerator function in this utility module iterates over subsets $R \subseteq \mathcal{F}$ in binary counting order. Since each subset maps to a distinct minimum cut $S^* \cup R$, every enumerated cut is guaranteed to be valid by lattice membership. The function exposes a `cut_limit` parameter such that: if $2^{|\mathcal{F}|} \leq \text{cut_limit}$ all minimum cuts are returned and `is_complete = true`; otherwise, the first `cut_limit` cuts are returned with `is_complete = false`. This can be necessary in highly symmetric networks, where $|\mathcal{F}|$ can be large, making full enumeration intractable.

7.5.2 Structure Utility Module

Motivating question: “Where is the network structurally fragile?”

Designed to inform routing and redundancy decisions, the structural module identifies components that are fragile purely due to their position in G , regardless of any capacity assignment.

Understanding the route-level structure of G is a prerequisite for any path-level fragility or flow-distribution analysis. To this end, the module exposes a base path enumerator function that performs a depth-first search on the topologically sorted iteration set derived from the `InputProcessingModule`. Since G is a DAG, exact enumeration is always finite; however, highly redundant networks can admit exponentially many routes, so a `path_limit` parameter bounds the number of enumerated paths and returns a representative sample rather than the complete set $\mathcal{P}$.

Flow Decomposition. Two networks can share identical topology and the same F^* while concentrating flow through entirely different routes. During design decisions where multiple configurations are evaluated and per-route visibility is needed, this decomposition exposes how f^* is distributed across $\mathcal{P}$.

Given an optimal flow f^* and a set of flow-agnostic paths $\mathcal{P}$, for each path $P = (v_0, v_1, \dots, v_k) \in \mathcal{P}$, the bottleneck flow contribution is the minimum flow on any edge along P :

$$\phi(P) = \min_i f_{v_i v_{i+1}}^*,$$

Thus, the edge meeting this minimum must be the path's local bottleneck. Paths are sorted in descending order of $\phi(P)$ as a ranked decomposition of f^* into per-route contributions. Where the min-cut module's ranking informs global capacity upgrade decisions, this ranking informs targeted interventions at the route level. In a network supplying two distinct demand nodes via separate routes, the min-cut module points to the tightest cut edge across the whole network, whereas the decomposition identifies independently which component is constraining delivery to each demand node.

Structural Single Point of Failure Nodes.

Definition 7.5 (Structural Single Point of Failure). *A node $v \in V \setminus (S \cup T)$ is a SPOF node if $F^*(G \setminus \{v\}) = 0$, where removing v also removes all incident edges. A SPOF node is one through which every source-to-sink path must pass; its removal alone disconnects the network regardless of any capacity assignment.*

The candidate set is restricted to nodes lying on at least one s - t path,

$$\mathcal{C} = (\text{fwd}(S) \cap \text{bwd}(T)) \setminus (S \cup T),$$

where $\text{fwd}(S)$ and $\text{bwd}(T)$ are the sets of nodes forward-reachable from S and backward-reachable from T in G respectively. Nodes outside $\mathcal{C}$ lie on no s - t path and cannot be SPOF nodes. For each $v \in \mathcal{C}$, a BFS is run on $G[V \setminus \{v\}]$; if no sink in T is reachable from any source in S , v is recorded as a SPOF node. The function requires one BFS per candidate and runs in $O(|\mathcal{C}| \cdot (|V| + |E|))$ time.

Noted that in a node-capacitated network, a node appearing in the min-cut is not necessarily a SPOF node in this structural sense. A capacity-constrained node enters the cut-space by way of its split edges $(v_{\text{in}}, v_{\text{out}})$ with capacity b_v and is not necessarily a structural choke-point in G . In these networks, a node can simultaneously be a flow-limiting bottleneck and a SPOF node.

7.5.3 Flow Sensitivity Module

Motivating question: “Which edges most constrain deliverable throughput, and by how much?”

At optimality, the flow f^* partitions the edge set into two classes: edges carrying strictly less than their capacity, $f_{uv}^* < c_{uv}$, and edges that are fully utilised,

$$f_{uv}^* = c_{uv}.$$

The latter are referred to as *saturated edges* in this work. By the max-flow min-cut theorem, every minimum cut set is made up only of saturated edges; therefore, any augmenting path from S to T must cross at least one saturated edge. Since the binding constraint lies elsewhere, *unsaturated edges* carry slack and are transparent to perturbation — increasing or decreasing their capacity by a small enough amount has little to no effect on F^* . The Flow Sensitivity module focuses only on saturated edges, since they collectively form the capacity-critical boundary of the network. This module exposes the following three functions.

Critical Edge Ranking. An obvious measure of an edge’s failure impact is how much the max flow drops when that edge is disconnected from the network, referred to as edge criticality in this module. The function re-runs the flow solver on a modified network for each candidate. For each saturated edge (u, v) , the critical impact is:

$$\Delta_{uv}^* = F^*(G, c) - F^*(G, c_0^{(uv)}),$$

where $c_0^{(uv)}$ denotes the capacity function with c_{uv} set to zero and all other capacities unchanged. The function returns the edges in descending order of Δ_{uv}^* .

Marginal Capacity Values. Similar to the critical edge ranker, the function is scoped to saturated edges. For each saturated edge (u, v) , it quantifies the incremental gain in F^* per unit of additional capacity,

$$\mu_{uv} = \frac{F^*(G, c_\delta^{(uv)}) - F^*(G, c)}{\delta},$$

where $c_\delta^{(uv)}$ increases c_{uv} by δ with all other capacities unchanged.

In highly redundant networks, bypass routes often exist that can absorb rerouted flow and upgrade priorities are not always clear. In these systems, an edge can rank high for failure impact but have a lower marginal value. The critical edge function identifies which edges are most damaging to lose while the marginal capacity function complements this by identifying which ones are most worth upgrading.

Marginal Range. Both Δ^* and μ are snapshot evaluations relative to the current capacity assignment c . Depending on the network structure, an edge with a low μ under the current inputs may become highly influential under different capacity values. This Marginal Range function evaluates the total range of influence a single edge (u, v) can exert on F^* :

$$\mathcal{B}_{uv} = F^*(G, c_\infty^{(uv)}) - F^*(G, c_0^{(uv)}),$$

where $c_{\infty}^{(uv)}$ sets $c_{uv} = \infty$ and $c_0^{(uv)}$ sets $c_{uv} = 0$, and all other capacities fixed. This function is analogous to the classical Birnbaum importance measure $I_B(i) = \partial h(\mathbf{p}) / \partial p_i$ (see [58]). However, the analogy is conceptual since $\mathcal{B}_{uv}$ is a deterministic throughput range, not a probability-weighted sensitivity.

7.5.4 Failure Impact Module

Motivating question: *“How is deliverable throughput affected when components fail or degrade?”*

The failure impact module models outright component failures and degraded operating states.

Single and Combined Edge Failures. For a single failed edge (u, v) , the throughput impact is:

$$\Delta_{uv} = F^*(G, c) - F^*\left(G, c_0^{(uv)}\right),$$

where $c_0^{(uv)}$ sets $c_{uv} = 0$ with all other capacities unchanged and candidate edges are restricted to E_{some} .

Real failure events are not always isolated to a single component. A storm or substation fault in a power network can take down multiple edges simultaneously. Since the interaction between affected edges may open or close re-routing options, the combined impact of k simultaneous failures is not in general the sum of their individual Δ_{uv} impacts.

The k -edge failure function generalises the single failure case to any set of k edges removed simultaneously. For a combination $\mathcal{K} \subseteq E_{\text{some}}$ with $|\mathcal{K}| = k$, the combined impact is:

$$\Delta_{\mathcal{K}} = F^*(G, c) - F^*\left(G, c_0^{(\mathcal{K})}\right),$$

where $c_0^{(\mathcal{K})}$ sets $c_{uv} = 0$ for all $(u, v) \in \mathcal{K}$. All $\binom{|E_{\text{some}}|}{k}$ combinations require one re-solve each. In practice, because $\binom{|E_{\text{some}}|}{k}$ grows rapidly with k , this function is most tractable for $k = 2$ or $k = 3$ depending on the size and density of the network. The function returns a complete ranking of k -edge failure scenarios by throughput impact.

Capacity Degradation Scenarios. Infrastructure components don’t always fail completely. Partial blockage or degraded components are more likely to present as a reduction in c_{uv} rather than complete loss. Additionally, a domain

expert working across multiple network states may want to compare the effect of varying input capacities and operating conditions. This function is intended to be used to efficiently compute the full F^* picture across all scenarios without needing to re-run the input processing or baseline solve steps for each. The degradation function accepts either an explicit override dictionary specifying new capacities for individual edges, or a uniform scale factor $\alpha \in [0, 1]$ applied to all finite capacities,

$$c'_{uv} = \alpha \cdot c_{uv}, \quad \forall (u, v) \in E.$$

7.5.5 Parametric Threshold Module

Motivating question: *“How much capacity change is needed to maintain or breach a target service level?”*

The previous modules treat edge capacities as fixed and analyse the network’s response to perturbations around them. The threshold module inverts this perspective. Given a required demand level $D_t \leq F^*$ to be met, it determines what capacity each edge must have for the network to meet that target delivery level. This makes it a planning and design tool rather than a diagnostic one.

This module exploits a specific structural property of F^* : it is piecewise linear in any single edge capacity c_e with all others fixed. Logically, each linear segment maps to a fixed min-cut partition S^* , over which the threshold satisfying $F^*(c_e) = D_t$ can be recovered analytically. Therefore, breakpoints must occur exactly where the min-cut partition shifts, i.e. where a change in c_e causes a different set of edges to become the binding bottleneck.

Degradation Threshold. For a target edge (u, v) and a required flow level D_t , the degradation threshold $c_{uv}^\dagger$ is the minimum capacity the edge must retain for F^* to remain at least D_t :

$$c_{uv}^\dagger = \inf \{c \in [0, c_{uv}] : F^*(G, c_c^{(uv)}) \geq D_t\},$$

where $c_c^{(uv)}$ denotes the capacity function with c_{uv} replaced by c . The degradation margin $c_{uv} - c_{uv}^\dagger$ is the slack available before the service level is breached. The module ranks edges by ascending degradation margin, so the most vulnerable edges appear first. A small margin can be interpreted as the edge is close to becoming a binding constraint on D_t , while a large margin means the edge can absorb substantial capacity loss without service impact.

The threshold is located by a partition-change recursion. Starting from the known endpoints $c_e = 0$ and $c_e = c_{uv}$, the algorithm evaluates F^* at the midpoint

and compares the resulting min-cut partition to those at the endpoints. If both endpoints share the same partition, the segment is linear and the threshold is solved analytically. If the partitions differ, the interval contains a breakpoint and the recursion descends into the sub-interval containing D_t . This continues until the threshold is isolated to a partition-constant segment, where it is computed exactly. The result is exactly capacity value at which F^* crosses D_t .

Two degenerate cases are handled explicitly. If $F^*(c_e = 0) \geq D_t$, the edge can degrade to zero without breaching the target. If $F^*(c_e = c_{uv}) < D_t$, the target is unachievable even at the current capacity, meaning the bottleneck lies entirely elsewhere and this edge cannot help.

Upgrade Threshold. If $F^* < D_t$ under the current capacity assignment, the upgrade threshold is the minimum capacity (u, v) must have for F^* to first reach D_t . Since the required capacity may exceed c_{uv} , the upper search bound is not known in advance. The algorithm uses a doubling search to bracket the threshold interval, repeatedly doubling the trial capacity until $F^* \geq D_t$, then applies the same partition-change recursion to locate it exactly.

An edge case arises when the edge is not on any minimum cut: expanding its capacity cannot raise F^* regardless of how large c_{uv} becomes. The doubling search detects this when F^* stops increasing before reaching D_t and returns `upgrade_ineffective = true`, indicating that reaching D_t requires addressing a different bottleneck.

When no explicit target is supplied, the module defaults to $D_t = 0.9 \cdot F^*$.

7.6 Exactness, Validation, and Scope

Exactness as used in this chapter is not the same as global combinatorial exhaustiveness. Where candidate-set pruning is applied, exactness describes the results on evaluated candidates only, not globally across all possible edge subsets or perturbation scenarios.

7.6.1 Post-solve validation

Every public solver call with validation enabled applies three post-solve consistency checks. (i) Capacity feasibility: all edge flows satisfy $0 \leq f(u, v) \leq c(u, v)$. (ii) Node-wise conservation: $\sum_{u:(u,v) \in E} f(u, v) = \sum_{w:(v,w) \in E} f(v, w)$ at all internal nodes. (iii) Max-flow/min-cut consistency: the solved flow value and the extracted cut capacity agree within numerical tolerance. These confirm primal feasibility and dual optimality consistency before derived analyses are interpreted.

7.6.2 Scope of results

Four main scope boundaries apply throughout this chapter. First, the sensitivity and failure-impact modules use candidate sets drawn from the baseline min-cut structure; results are exact on evaluated candidates, and global exhaustiveness over all edges or all combinatorial subsets requires explicit full-space evaluation. Second, the capacity toolkit does not compute $P(\text{performance} \geq T)$ or any probability-weighted reliability index. Third, all analyses operate on a single fixed-capacity instance; time-varying degradation and adaptive re-routing models are outside the scope of this layer. Finally, for complete throughput loss, minimum cuts are the minimal failure modes; for nonzero service targets, the relevant boundary is characterised by threshold margins $c_{uv} - c_{uv}^\dagger$.

7.6.3 Computational Complexity

The total computational cost of any derived analysis is the product of the per-solve cost of the chosen max-flow algorithm and the number of solver calls required. Repeated-solve tasks scale with solve count, which is the dominant cost driver in practice. Per-solve costs are summarised in Table 7.1.

Module	Function	Operation	Complexity
<i>Base Solver</i>			
Base	Edmonds-Karp	BFS augmenting path	$O(VE^2)$
Base	Dinic	Level graph + blocking	$O(V^2E)$
Base	Push-Relabel	Preflow relabeling	$O(V^3)$
Node-Cap	Split transformation	Graph transform	$O(V + E)$
Node-Cap	Solve (split)	Solver on $2V, 2E$	$O(T_{\text{solver}})$
<i>Min-Cut Utility</i>			
Min-Cut	cut_edges	Extract partition	$O(E)$
Min-Cut	edges_some	BFS + filter	$O(E + T_s)$
Min-Cut	edges_every	Reachability	$O(V + E)$
Min-Cut	enumerate	Lattice ($2^{ \mathcal{F} }$)	$O(2^{ \mathcal{F} } \cdot E)^1$
<i>Structural</i>			
Struct	spof_nodes	BFS per candidate	$O(\mathcal{C} (V + E))$
Struct	enumerate_paths	DFS on DAG	$O(\mathcal{P})^2$
Struct	path_flow	Per-path scan	$O(\mathcal{P} \ell)$
Struct	redundancy	Menger (unit cap)	$O(E_c T_s)$
<i>Flow Sensitivity</i>			
Sens	critical_edges	Zero-cap solves	$O(E_{\text{sat}} T_s)$

Module	Function	Operation	Complexity
Sens	marginal_capacity	$+\delta$ solves	$O(E_{\text{sat}} T_s)$
Sens	marginal_range	Range evals	$O(E_{\text{some}} T_s)$
<i>Failure Impact</i>			
Fail	single_edge	Per-edge solves	$O(E_{\text{some}} T_s)$
Fail	k_edge	Per-combination	$O(\binom{ E_s }{k}T_s)^3$
Fail	degradation	Per-scenario	$O(S T_s)$
<i>Parametric Threshold</i>			
Param	degrade	Partition recursion	$O(\lg(c/\varepsilon)T_s)$
Param	upgrade	Doubling + recurse	$O(\lg^2(c/\varepsilon)T_s)$
Param	all_degrade	Per edge	$O(E_c \lg(c/\varepsilon)T_s)$

Table 7.2: Per-task computational complexity for the flow capacity toolkit. T_s denotes the cost of a single max-flow solve under the chosen algorithm.

7.7 Network Flow Output Structure

Field	Module	Description
max_flow	Base Solver	Maximum deliverable throughput F^* from source set to sink set.
flow	Base Solver	Final flow on each original edge.
residual_capacity	Base Solver	Spare capacity on each original edge after the solve.
node_flow	Base Solver	Through-flow at each node: outflow at sources, inflow at sinks, conserved flow at transit nodes.
sink_flow	Base Solver	Delivered flow broken down by individual sink.
mincut_S, mincut_T	Base Solver	Source- and sink-side min-cut node partitions.
mincut_capacity	Base Solver	Capacity of the reported minimum cut.
is_unbounded	Base Solver	Whether unbounded max-flow was detected.

Field	Module	Description
<code>saturated_edges</code>	Base Solver	Edges carrying flow at full capacity $f_{uv}^* = c_{uv}$; forms the candidate set for sensitivity and failure impact modules.
<code>node_cap_max_flow</code>	Node-Cap Base Solver	Maximum deliverable throughput F^* under node capacity constraints.
<code>node_cap_flow_drop</code>	Node-Cap Base Solver	Reduction in F^* relative to the edge-only baseline.
<code>saturated_nodes</code>	Node-Cap Base Solver	Nodes whose through-flow reaches the node capacity bound b_v under the node-capacitated solve.
<code>mincut_S_star_star</code>	Min-Cut Utility	S^{**} : extended source partition; nodes that cannot reach the sink in the residual graph.
<code>free_zone_size</code>	Min-Cut Utility	$ \mathcal{F} $: number of free-zone nodes assignable to either partition without affecting cut capacity. Exactly $2^{ \mathcal{F} }$ equivalent minimum cuts exist.
<code>edges_in_some</code>	Min-Cut Utility	E_{some} : edges appearing in at least one minimum cut.
<code>edges_in_every</code>	Min-Cut Utility	E_{every} : edges appearing in every minimum cut; structural single points of failure.
<code>mincut_enumeration</code>	Min-Cut Utility	Bounded enumeration of all valid minimum cuts in deterministic order, with completion flag <code>is_complete</code> and count.
<code>spof_nodes</code>	Structure Utility	Structural single points of failure nodes: $v \in \mathcal{C}$ where every s - t path must pass through v .
<code>enumerated_paths</code>	Structure Utility	Source-to-sink paths $\mathcal{P}$, bounded by <code>path_limit</code> .
<code>path_contributions</code>	Structure Utility	Per-path bottleneck flow $\phi(P) = \min_i f_{v_i v_{i+1}}^*$ ranked in descending order.
<code>critical_edges</code>	Flow Sensitivity	Saturated edges ranked by max-flow drop Δ_{uv}^* when set to zero capacity, in descending order.

Field	Module	Description
marginal_capacity	Flow Sensitivity	Saturated edges mapped to incremental max-flow gain μ_{uv} per unit capacity increase.
marginal_range	Flow Sensitivity	Total range of influence $\mathcal{B}_{uv} = F^*(G, c_\infty^{(uv)}) - F^*(G, c_0^{(uv)})$ for each edge.
fail_min_cut_edges	Failure Impact	Representative min-cut edge set: all edges (u, v) with $u \in S^*$ and $v \in T^*$.
single_edge_fail	Failure Impact	Edges in E_{some} ranked by throughput drop Δ_{uv} when set to zero capacity, in descending order.
k_edge_fail	Failure Impact	All k -edge combinations from E_{some} ranked by combined impact Δ_K , bounded by <code>combination_limit</code> .
degrade_results	Failure Impact	Max-flow and saturated edges for each capacity degradation scenario: either uniform scale factor α or edge-wise overrides.
degrade_thresholds	Parametric Threshold	For each candidate edge, the minimum capacity $c_{uv}^\dagger$ required to maintain target flow D_t , ranked by ascending degradation margin $c_{uv} - c_{uv}^\dagger$.
upgrade_thresholds	Parametric Threshold	For each candidate edge, the minimum capacity required for F^* to first reach D_t , with <code>upgrade_ineffective</code> flag where applicable.
param_target_flow	Parametric Threshold	Target service level D_t . Defaults to $0.9 \cdot F^*$ if not specified.
param_baseline_flow	Parametric Threshold	Baseline max flow F^* for reference.

Table 7.3: Unified output schema for network flow capacity analysis across all modules.

7.8 Case Study: Synthetic Benchmark Network

7.8.1 Benchmark Network Design

To demonstrate the full range of the flow toolkit, a synthetic benchmark was generated. The network is a multi-source, multi-sink capacitated DAG with 43 nodes, 107 directed edges. The benchmark simulates a distribution system that propagates resources across geographically dispersed locations. As shown in Figure 7.6, it is structured as an 8-layer hierarchy:

- **Origin nodes** (blue): five source nodes, over-provisioned so the supply zone does not form the binding bottleneck.
- **Hub nodes** (green): eight intermediate relay nodes with high fan-out capacities.
- **Processing nodes** (red, node-capacitated) and **bypass nodes** (orange, node-capacitated): the processing corridor carries the bulk of flow; the bypass is a lightweight alternative with tighter node constraints.
- **Trunk and secondary nodes** (purple, brown): distribute flow through a re-converging structure toward the gateway layer.
- **Dispatch gateways** (light blue, node-capacitated): two gateways assigned equal edge capacity of 17 by design.
- **Distribution combiner** (pink): aggregates all gateway flow before fan-out to the sinks.
- **Sink nodes** (grey): seven demand nodes, each with incoming edge capacity 6.

Based on this layered structure, edge and node capacity assignments are chosen to ensure that binding bottlenecks emerge at specific layers under the different analysis methods. Edge capacities in Layers 1–5 are scaled by a factor of 2 relative to the gateway layer. In this way, the gateway layer governs F^* under the edge-only solution. Similarly, to compute a measurable reduction in F^* , the processing node capacities are set to bind to the node-capacitated solve.

To explore competing design alternatives under controlled conditions, two configurations of the same system are presented to evaluate the effect of feeder reinforcement (Figures 7.6a and 7.6b). Both configurations of the network keep the same backbone topology, but represent different design objectives. Therefore, the comparative objective is not to maximize throughput alone, but to test whether robustness structure changes even when headline flow remains unchanged.

Config A proposes a symmetry-preserving design with balanced feeder-edge capacities and a limited direct bypass to sink 4. The capacity is set below the sink’s 6-unit limit such that the bypass delivers partial rather than full service. Config B’s asymmetric gateway structure is designed to produce non-trivial parametric upgrade thresholds and non-zero marginal-range values. This version has larger feeder capacities from secondary nodes into both gateways.

Table 7.4 summarises the key structural differences between the two. The expectation is that two designs may yield the same max-flow value while looking very different in terms of saturation pattern and min-cut landscape due to their differing feeder-capacity policies.

7.8.2 Comparative Results (Config A vs Config B)

The base flow solver run returns $F^* = 37.0$, with `mincut_capacity` = 37.0 for both networks, consistent with the expected maximum feasible flow for the benchmarked system. However, when optimising for a demand target or comparing two network design approaches, F^* alone is not always enough to inform decisions. As shown in Table 7.4, the two networks still differ in structure: Configuration A has 41 saturated edges versus 35 in Configuration B, and 4 valid minimum cuts versus 1. This is also reflected in free-zone size (2 in A, 0 in B). The same throughput under structurally different constraints motivates the use of the extended analysis modules in the toolkit.

Table 7.4: Configuration comparison summary.

Metric	Config A	Config B
F^*	37.0	37.0
<code>saturated_edges</code>	41	35
<code>free_zone_size</code>	2	0
<code>min_cuts</code>	4	1
<code>SP0F_nodes</code>	0	0
<code>upgrade_ineffective</code> (gateway edges)	true	false

From a design perspective, this reflects a trade-off between structural flexibility and tighter bottleneck localisation. In Configuration A, the free-zone nodes are {134, 135}, producing a wider cut lattice where E_{every} and E_{some} differ. The gateway edges $134 \rightarrow 136$ and $135 \rightarrow 136$ appear in E_{some} but not in E_{every} , so neither is a universal failure mode on its own. Cut 4 has only 3 crossing edges while Cut 1 has 13, both at capacity 37 (Table 7.9). In Configuration B, feeder-capacity reinforcement pins the cut to a unique partition: free-zone size collapses to 0 and the gateway edges move into E_{every} . The choice between configurations depends

on the optimisation target: Configuration A offers more structural redundancy and cut diversity, while Configuration B offers tighter bottleneck localisation and actionable upgrade thresholds.

Bypass edge and structural vulnerability. The direct bypass edge $132 \rightarrow 140$ improves `sink_4` service with capacity 3, below the sink-side limit of 6. The same edge raises redundancy scores for all secondary-to-gateway connections from 2 to 3, improving redundancy across the gateway-feeding layer, not only the directly connected nodes.

Without this edge, node 136 is the only SPOF node: every source-to-sink path passes through it and $F^*(G \setminus \{136\}) = 0$. Adding $132 \rightarrow 140$ gives $F^*(G \setminus \{136\}) = 3 > 0$, removing node 136 from the SPOF set, and both configurations report `spof_nodes` = $\emptyset$. However, only `sink_4` has an alternative path, so all other flow still passes through the combiner. Running the capacity degradation module with node 136 removed confirms this: F^* drops to 3, exposing the network’s continued vulnerability to combiner failure despite the absence of a formal SPOF classification.

Node-capacitated analysis. When node capacities are enabled, both configurations still return the same result: a 7-unit loss down to `node_cap_max_flow` = 30.0. The same six Layer 3 nodes {114, 115, 116, 117, 118, 119} saturate in both cases. This matters because it tells the system designer where to act: once these node limits are active, changing gateway-edge capacities does not change the outcome.

The flow loss is not shared evenly across demand nodes (Figure 7.9). `sink_1` (137) falls from 4.0 to 0.0, so it loses service completely. `sink_2` (138) falls from 6.0 to 3.0, while sinks 139–143 stay at their previous levels. Reporting total flow alone misses the main service issue.

To test a direct fix, a single-node upgrade sweep was run on nodes 114–119 (Table 7.5). Any one of these nodes can restore `sink_1` to its edge-only service level with $\Delta b_v = 7.0$, and this also returns total flow to $F^* = 37.0$ in both configurations. The increment is the same for all six nodes, but the final capacity targets are not: nodes 114/115 go to $b_v = 12.0$, nodes 116/117 go to $b_v = 14.0$, and nodes 118/119 go to $b_v = 10.0$. In practice, that means the choice should be made by delivery constraints such as cost, installation effort, and access, not by throughput effect.

Flow decomposition results. Flow decomposition shows that both configurations achieve the same total throughput ($F^* = 37.0$) with very similar route-level

Table 7.5: Node-upgrade sweep (Configurations A and B give identical results): minimum single-node capacity increase required to restore `sink_1` service under the node-capacitated solve.

Node	Current b_v	Required Δb_v	New b_v
114 (<code>processing_A</code>)	5.0	7.0	12.0
115 (<code>processing_B</code>)	5.0	7.0	12.0
116 (<code>processing_C</code>)	7.0	7.0	14.0
117 (<code>processing_D</code>)	7.0	7.0	14.0
118 (<code>bypass_E</code>)	3.0	7.0	10.0
119 (<code>bypass_F</code>)	3.0	7.0	10.0

allocation. Configuration A decomposes into 27 components and Configuration B into 25, so the difference is small and does not change the main delivery pattern. In both cases, the largest single component carries 3.0 units on a path bottlenecked at $(114 \rightarrow 120)$, and most remaining components are paths carrying 2.0 or 1.0 units distributed across the processing and bypass corridors.

This supports the earlier structural findings. The A/B edge-capacity changes alter which secondary-to-gateway edges appear as local path bottlenecks, but they do not create a different high-level flow-allocation picture. The dominant route family still passes through the same Layer 3 processing/bypass region and then through the combiner to sinks, with the direct bypass to sink 4 contributing only partial service. At decomposition level, the two designs differ in local edge pressure more than in overall route usage.

Sensitivity analysis. Sensitivity ranking shows a clear tiered pattern in Config A (Table 7.6). The two gateway-to-combiner edges $(134 \rightarrow 136)$ and $(135 \rightarrow 136)$ rank first and second with $\Delta^* = 17.0$ each, reducing flow from $F^* = 37.0$ to 20.0 when removed (about 46%). The next tier contains four feeder edges with $\Delta^* = 4.0$ and perturbed flow 33.0: $(128 \rightarrow 134)$, $(130 \rightarrow 135)$, $(131 \rightarrow 134)$, and $(133 \rightarrow 135)$. A third tier of five edges has $\Delta^* = 3.0$ and perturbed flow 34.0, including the direct bypass edge $(132 \rightarrow 140)$; this places the bypass in the same sensitivity tier as several secondary-to-gateway feeders. The remaining non-zero edges have $\Delta^* \in \{1.0, 2.0\}$ with perturbed flow 35.0 or 36.0. In total, 15 edges have non-zero single-edge impact in Configuration A.

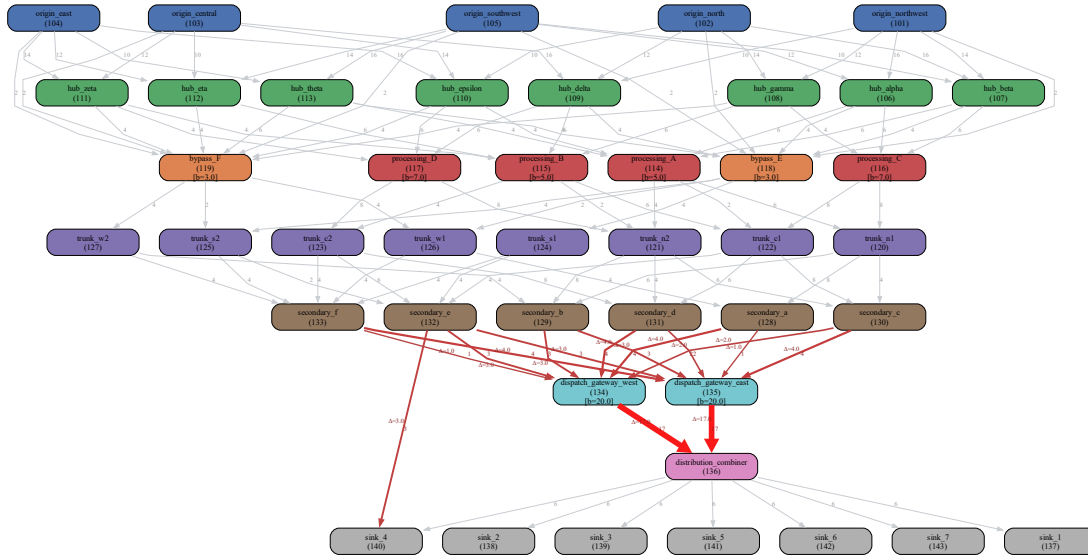
Configuration B is much more concentrated: only three edges have non-zero impact, $(134 \rightarrow 136)$, $(135 \rightarrow 136)$, and $(132 \rightarrow 140)$, while all other evaluated edges give $\Delta^* = 0$ (Table 7.7). This means the feeder-capacity reinforcement in B absorbs any single feeder-edge loss through rerouting, so feeder removals no longer reduce F^* individually. The non-zero set therefore collapses from 15 edges in A to 3 in B.

Table 7.6: Full ranked single-edge sensitivity results for Configuration A (all non-zero ΔF^* edges).

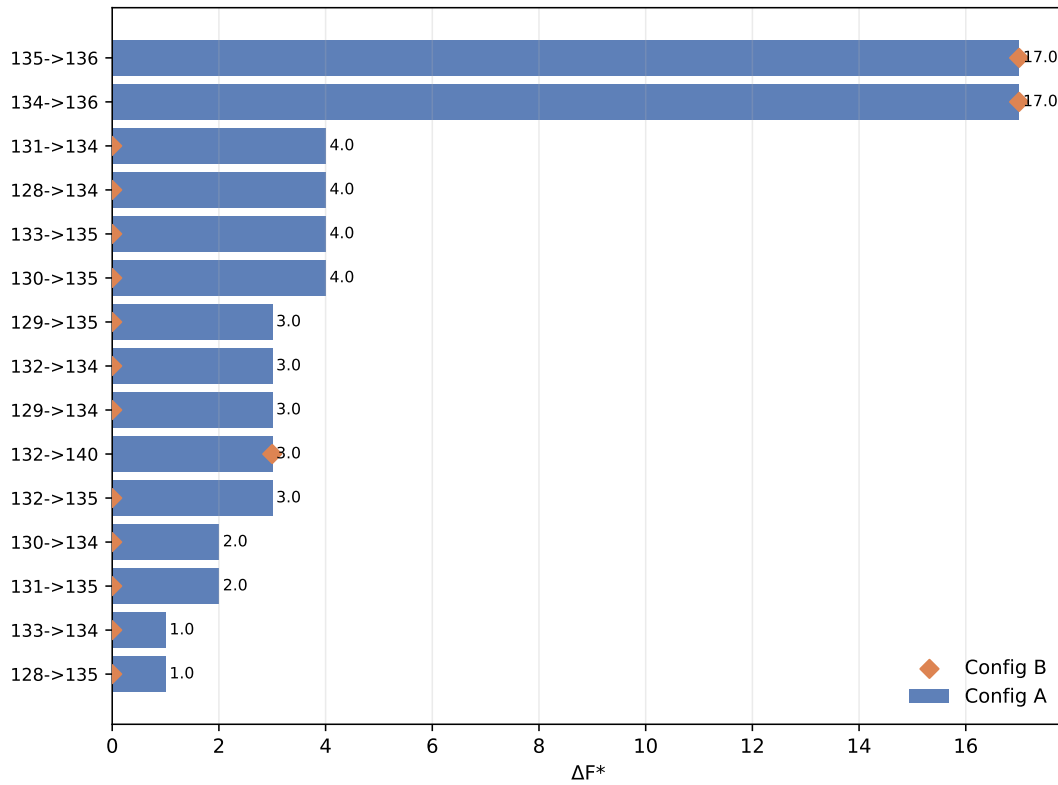
Rank	Label	ΔF^*	perturbed_flow
1	dispatch_gateway_west->distribution_combiner	17.0	20.0
2	dispatch_gateway_east->distribution_combiner	17.0	20.0
3	secondary_a->dispatch_gateway_west	4.0	33.0
4	secondary_c->dispatch_gateway_east	4.0	33.0
5	secondary_d->dispatch_gateway_west	4.0	33.0
6	secondary_f->dispatch_gateway_east	4.0	33.0
7	secondary_b->dispatch_gateway_west	3.0	34.0
8	secondary_b->dispatch_gateway_east	3.0	34.0
9	secondary_e->dispatch_gateway_west	3.0	34.0
10	secondary_e->dispatch_gateway_east	3.0	34.0
11	secondary_e->sink_4	3.0	34.0
12	secondary_c->dispatch_gateway_west	2.0	35.0
13	secondary_d->dispatch_gateway_east	2.0	35.0
14	secondary_a->dispatch_gateway_east	1.0	36.0
15	secondary_f->dispatch_gateway_west	1.0	36.0

Table 7.7: Full ranked single-edge sensitivity results for Configuration B (all non-zero ΔF^* edges).

Rank	Label	ΔF^*	perturbed_flow
1	dispatch_gateway_west->distribution_combiner	17.0	20.0
2	dispatch_gateway_east->distribution_combiner	17.0	20.0
3	secondary_e->sink_4	3.0	34.0



(a) Configuration A critical-edge overlay. Edge emphasis (colour/width) is proportional to non-zero ΔF^* .



(b) Ranked non-zero single-edge sensitivity for Configurations A and B.

Figure 7.5: Single-edge sensitivity in Config A and cross-configuration rank comparison (A vs B).

Failure impact. The two-edge failure ranking confirms that the dominant pair is simultaneous loss of $(134 \rightarrow 136)$ and $(135 \rightarrow 136)$, with drop 34.0 and perturbed flow 3.0. This is much larger than the next group (drop 21.0), and makes sense given the single-edge pattern shown in Figure 7.5. In Configuration B, the same gateway-pair dominance remains, while the non-zero failure set is more concentrated because feeder reinforcement absorbs most single-edge feeder losses through rerouting.

Degradation trajectory. The degradation trajectories are identical for Configurations A and B at all tested scale factors. The solved values are: $\alpha = 1.0 \rightarrow 37.0$, $\alpha = 0.9 \rightarrow 33.3$, $\alpha = 0.8 \rightarrow 29.6$, $\alpha = 0.7 \rightarrow 25.9$, $\alpha = 0.6 \rightarrow 22.2$, and $\alpha = 0.5 \rightarrow 18.5$. These points follow the linear relation

$$F^*(\alpha) = 37\alpha$$

over the tested interval.

The A/B curves that coincide point-for-point (Figure 7.8) indicate that the feeder-capacity differences that separate A and B under single-edge sensitivity do not produce a different global response under a uniformly scaled network-wide degradation.

Parametric thresholds. Parametric thresholds separate edge criticality from edge upgradeability. In Configuration A, the gateway edges $(134 \rightarrow 136)$ and $(135 \rightarrow 136)$ are upgrade-ineffective (`required_capacity` = ∞ , `ineffective`=true) even though they are high-impact in sensitivity ranking. In Configuration B, the same gateway edges become upgrade-effective with `required_capacity`=18.0 and `required_increase`=1.0. Degradation thresholds remain at 17.0 with zero margin in both configurations, so any capacity loss below current levels immediately reduces F^* .

This difference is explained by upstream supply. In Configuration A, supply into each gateway is already capped at the current gateway-edge capacity, so increasing gateway-edge capacity alone cannot raise throughput. In Configuration B, feeder reinforcement creates upstream headroom, so gateway-edge upgrades can increase F^* .

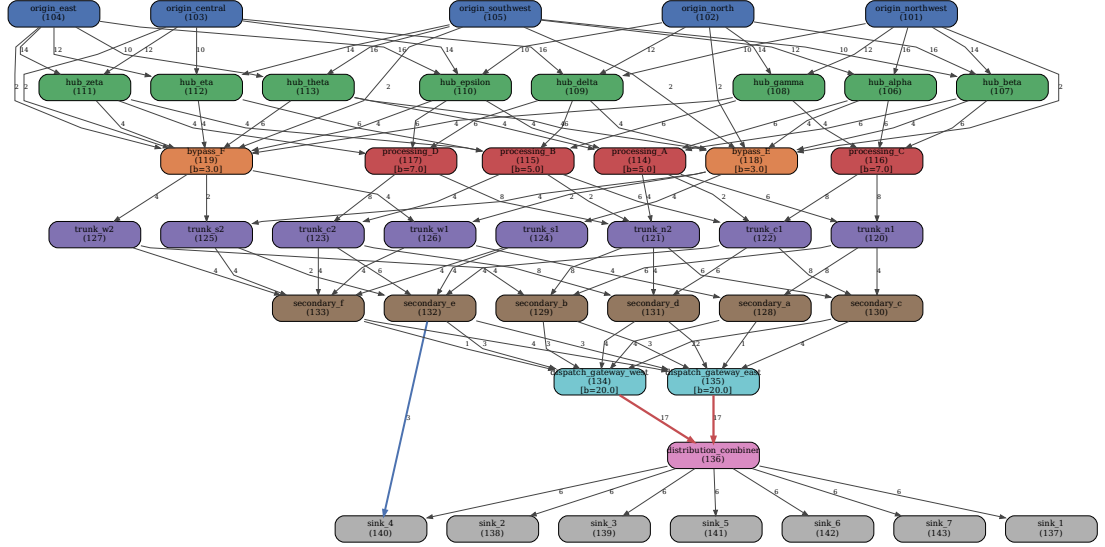
For the bypass edge $(132 \rightarrow 140)$, $\mathcal{B}_{uv}$ increases from 10.0 (A) to 16.0 (B), and the edge is upgrade-effective in both configurations with `required_increase`=1.0 (`required_capacity`=4.0). The main A/B contrast therefore remains at the gateway edges, not at bypass upgrade feasibility.

Table 7.8: Parametric threshold and marginal range comparison for key edges.

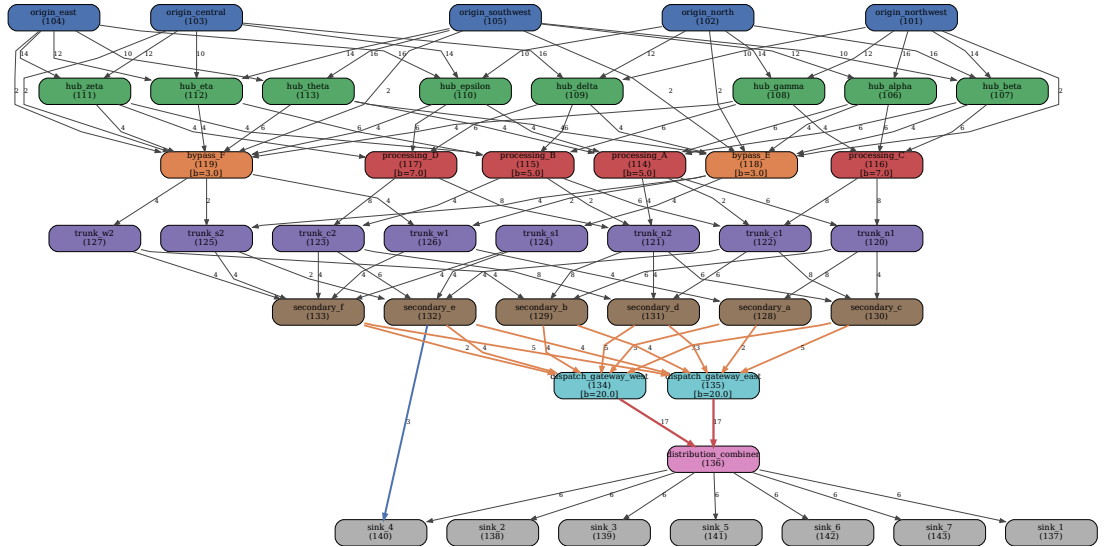
Edge	Config A		Config B	
	Upgrade	$\mathcal{B}_{uv}$	Upgrade	$\mathcal{B}_{uv}$
$134 \rightarrow 136$	ineffective	17.0	+1.0	23.0
$135 \rightarrow 136$	ineffective	17.0	+1.0	23.0
$132 \rightarrow 140$	+1.0	10.0	+1.0	16.0

Table 7.9: Min-cut enumeration for Config A.

Cut	Crossing edges
1	$(128 \rightarrow 134), (128 \rightarrow 135), (129 \rightarrow 134), (129 \rightarrow 135),$ $(130 \rightarrow 134), (130 \rightarrow 135), (131 \rightarrow 134), (131 \rightarrow 135),$ $(132 \rightarrow 134), (132 \rightarrow 135), (132 \rightarrow 140), (133 \rightarrow 134),$ $(133 \rightarrow 135)$
2	$(128 \rightarrow 135), (129 \rightarrow 135), (130 \rightarrow 135), (131 \rightarrow 135),$ $(132 \rightarrow 135), (132 \rightarrow 140), (133 \rightarrow 135), (134 \rightarrow 136)$
3	$(128 \rightarrow 134), (129 \rightarrow 134), (130 \rightarrow 134), (131 \rightarrow 134),$ $(132 \rightarrow 134), (132 \rightarrow 140), (133 \rightarrow 134), (135 \rightarrow 136)$
4	$(132 \rightarrow 140), (134 \rightarrow 136), (135 \rightarrow 136)$



(a) Configuration A: symmetric gateway design. Blue edge $132 \rightarrow 140$ is the direct bypass to `sink_4`.



(b) Configuration B: orange edges show capacity-boosted secondary-to-gateway feeders

Figure 7.6: Flagship benchmark DAG (43 nodes, 107 edges, 8 layers). Node colours: blue = sources, green = hubs, red = constrained nodes, orange = bypass nodes, purple = trunks, brown = secondary, pink = combiner, grey = sinks.

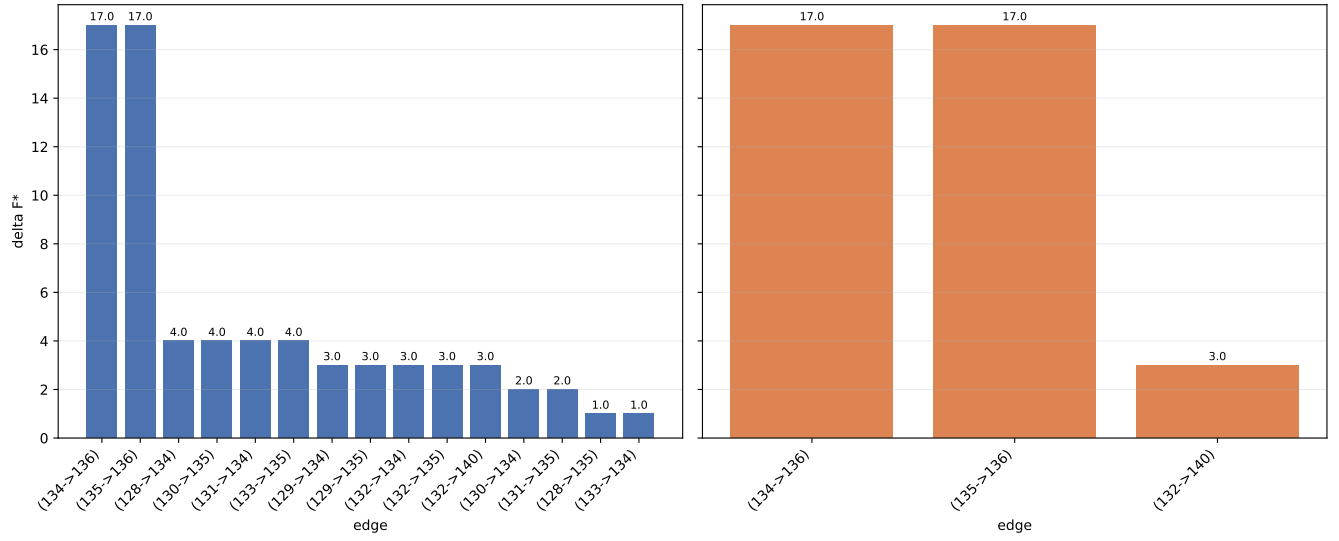


Figure 7.7: Sensitivity ranking for non-zero ΔF^* values in Configurations A and B.

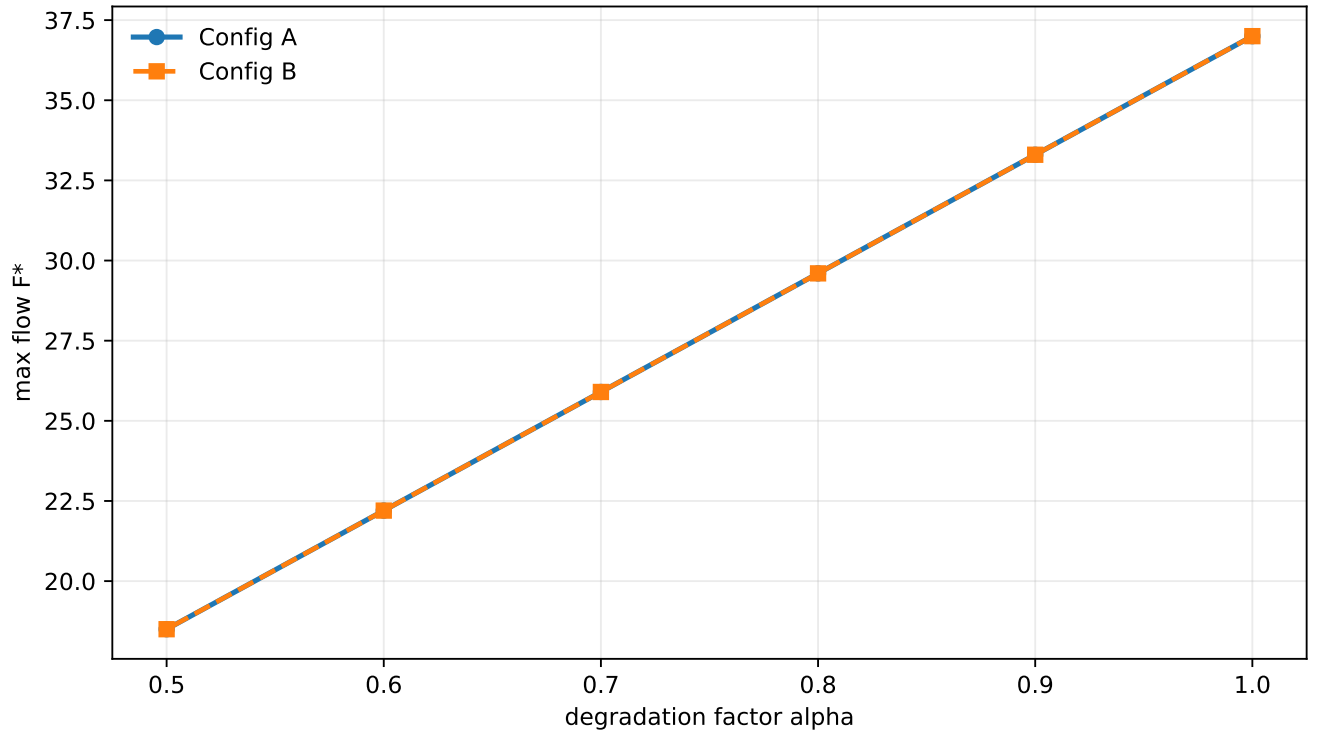


Figure 7.8: Degradation trajectory $F^*(\alpha)$ for Configurations A and B under uniform capacity scaling.

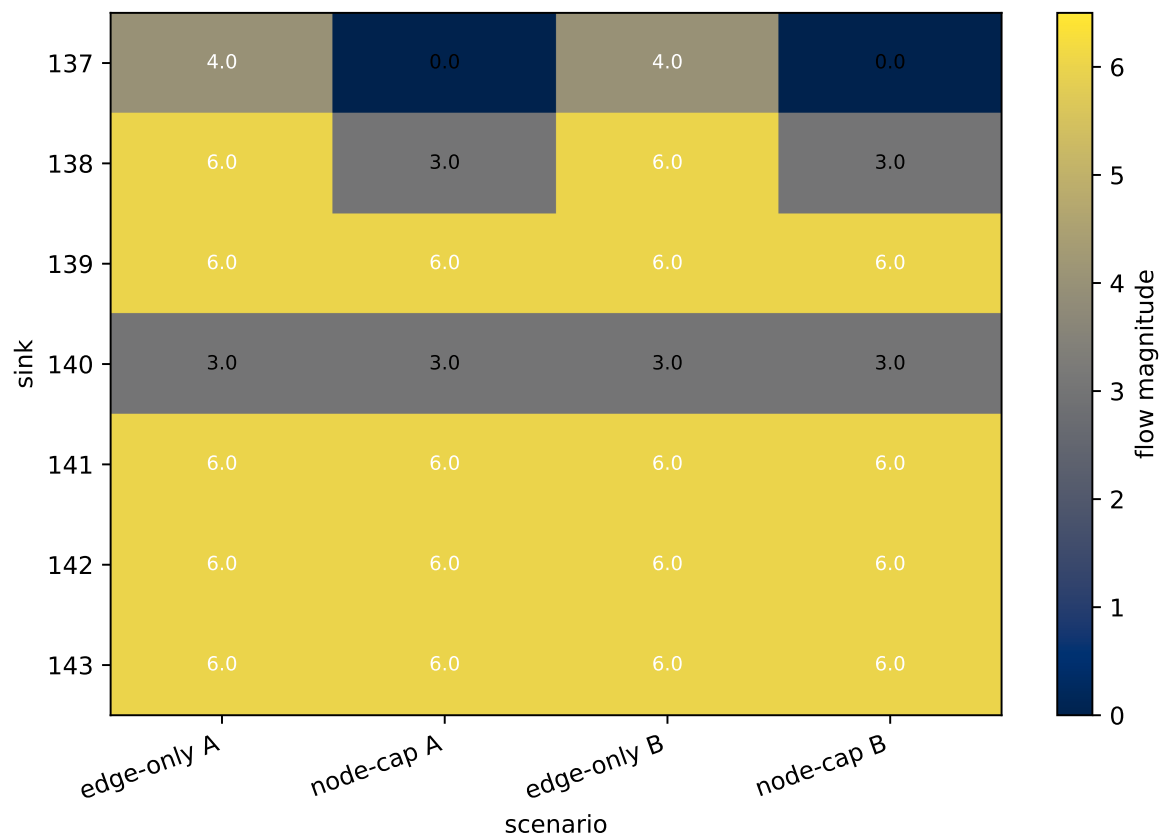


Figure 7.9: Sink-flow comparison for edge-only and node-capacitated solves across Configurations A and B.

Chapter 8

Information As Schedule and Cost: Critical Path Toolkit

8.1 Introduction

The Critical Path Toolkit consumes the unified graph object of Chapter 4 directly and involves no decomposition. Every analysis in it is built from linear-time passes over the layered iteration sets, so the Network Decomposition Module of Chapter 5 is bypassed entirely.

The quantities this toolkit propagates are the ones that accumulate along dependency paths. A duration, a cost, a risk factor and a load all compound as they travel, and a network carrying such a quantity poses the same pair of questions whatever the quantity is. One is extremal, asking for the best or worst value a complete chain of dependencies can produce. The other concerns room, asking how far an individual component can move before that extremal answer changes. The first is settled by a forward pass. The second is where the analysis actually lives, because room is what criticality, bottlenecks and priorities are made of, and it exists only where the arithmetic of the quantity can be run backwards.

8.2 Scheduling Analysis on Networks

The critical path method models a project as a network of activities and computes the longest path through it. The project cannot finish earlier than that path allows, the activities on it have no room to slip, and every other activity carries a float equal to how far it can slip before it starts to matter. The method and the float calculus date to the late 1950s [18], and the companion question of duration uncertainty was raised almost immediately: PERT replaced each fixed duration with a three-point estimate and a distributional assumption [26], and its descendants in modern practice replace the assumption with Monte Carlo sampling. Distribution-free treatment of the same uncertainty came much later. When each duration is known only as an interval, the forward quantities remain easy, but the backward ones change character entirely. Deciding whether an activity is possibly critical is NP-complete and float bounds are NP-hard [27], even on planar networks [59]. Exact methods exist for latest starting times and one of the two float bounds [60], the consolidated picture is given by Fortin, Zieliński, Dubois and Fargier [61], and the recognised tractable class is the series-parallel networks.

None of the machinery is specific to time. The forward recursion combines incoming values and compounds them along edges, and any quantity with those two operations propagates the same way. With combination as maximum and compounding as addition the recursion is the critical path method. With other pairs it is cost roll-up, risk scaling or route selection, and the algebra of such

systems is well established: max-plus linear systems theory treats the classical forward and backward passes as linear algebra over an idempotent semiring, with the backward pass arising as residuation [62]. The lesson of that algebra is a discipline. A forward recursion exists for any pair of operations, but a backward one is a property the pair either has or lacks, and an analysis that reports slack without checking has manufactured a number. Uncertainty sharpens the same discipline from another side, because the interval results above draw their line between easy and hard exactly where monotonicity is lost.

8.3 Analysis Modes

Whether the room question has an answer is a property of the operator pair, and it has one in exactly two situations. When combination is an order, maximum or minimum, a bound on the combined value binds every argument, and when compounding is monotone with a residual the bound inverts across each edge, so a constraint of the form finish by K travels backwards through the network the same way values travel forwards. That is residuation in an idempotent semiring [62], and it is what the classical backward pass has always been. Summation is different in kind. A total is not a deadline and no residuation exists, but a summed system is linear, and a linear map has an adjoint. The backward object of an accumulation analysis is therefore a sensitivity, and the room it measures is an allowance against a stated budget rather than a slack against a deadline. A pair with neither structure supports a forward recursion and nothing else, and any margin reported for it would be invention. A configuration of the toolkit, called a mode, is an operator pair together with whatever backward semantics its algebra supports, and Table 8.1 lists the four the framework provides.

Table 8.1: The shipped modes. Every row carries a defined backward semantics; the margin column names what the backward pass reports.

Mode	$\oplus$ / $\otimes$	Backward mechanism	Margin
LongestPath	max / +	residuation (min / −)	slack
ShortestPath	min / +	dual residuation (max / −)	margin over optimum
MaxScaling	max / $\times$	residuation (min / $\div$)	ratio slack
Accumulation	sum / +	adjoint (path multiplicities)	allowance

The four margins in the table are two objects, not four. The three order-based modes all report the same thing, the distance between the best complete path and the best path forced through the node in question, expressed as a difference for the additive pairs and as a ratio for the multiplicative one, and in each case

the zero set is the critical structure of the analysis. The allowance is the linear system's counterpart. Because a node's value reaches the target once per route, its influence on the total is a path count rather than a path choice, and the room it has under a budget is the remaining headroom shared across those routes. Running a scheduling backward pass over a summed quantity instead produces figures that answer no question at all, which is why no slack appears in the accumulation row. The water network of Section 8.7.1 shows what the allowance says in its place.

8.4 The Two Kernels

The backward pass is not a second algorithm. For the order-based modes one fold serves both directions. Read forwards over the layered iteration sets it computes, for every node v with parent set $\text{pa}(v)$, duration d_v and edge values w_{uv} ,

$$F_v = \left(\bigoplus_{u \in \text{pa}(v)} F_u \otimes w_{uv} \right) \otimes d_v,$$

the best value a chain of dependencies can deliver into each node, with sources seeded from the initial value. Read over the reversed graph the same fold computes

$$R_v = \bigoplus_{s \in \text{succ}(v)} (R_s \otimes d_s) \otimes w_{vs},$$

the best completion from each node onwards, with sinks seeded at the neutral element. The two arrays meet in the middle. Their compound $F_v \otimes R_v$ is the best complete path forced through v , the project value P is the best complete path free of any such constraint, and the margin of v is the gap between the two. Slack, in this reading, is the cost of insisting on a node. The textbook schedule quantities are recoveries from the same two arrays, early start as $F_v - d_v$ and late finish as $P - R_v$, with late start and total float following by the same subtractions. On the four-node network of Figure 8.1, with durations 1, 5, 2, 1 and no edge delays, the longest reading completes in 7 along $1 \rightarrow 2 \rightarrow 4$ while node 3 carries slack 3, and the shortest reading completes in 4 along $1 \rightarrow 3 \rightarrow 4$ while node 2 carries margin 3. The same inputs answer opposite questions with opposite critical chains.

Accumulation replaces the order by a sum, and the meeting in the middle changes meaning. The total at a target t expands to $\sum_v m_v d_v + \sum_{(u,v)} m_v w_{uv}$, where m_v counts the directed routes from v to the target, so a node influences the total not through one best path but once per route, and the reverse traversal that computes the counts, seeded with $m_t = 1$, is the adjoint of the forward map. The count is the derivative of the total with respect to the node's value, the product $m_v d_v$ is the node's share of the total, and under a budget B the room left to

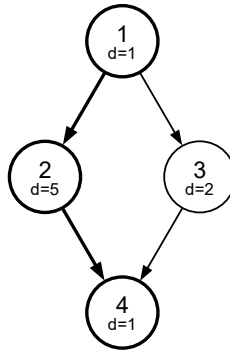


Figure 8.1: The running example. Node durations 1, 5, 2, 1; LongestPath is critical through nodes 1, 2, 4 and ShortestPath through 1, 3, 4.

the node is $(B - F_t)/m_v$, headroom divided across its routes. On the running example the total at node 4 is 10 and node 1 counts twice, once through each branch. Both kernels touch each edge a constant number of times, so every mode costs $O(|V| + |E|)$ whatever the operators are.

8.5 Interval Analysis

Uncertainty divides the toolkit’s quantities along a single line, and the line is monotonicity. Every forward quantity grows when any input grows, so with interval inputs its exact bounds are two crisp runs of the scalar kernel, one at the lower corner of the input box and one at the upper, and interval arithmetic never enters. Margins sit on the other side of the line. A margin is a difference of two monotone quantities that share their inputs, it is not monotone in anything, and the hardness results of Section 8.2 live precisely there. Running the scalar algorithms on an overloaded interval type would blur this line rather than respect it, ranking overlapping intervals through a partial order that cannot rank them and subtracting dependent quantities as though they were independent. The toolkit instead computes each interval quantity by the scheme its structure permits. Forward quantities are exact for the price of two runs. Margins come either as a sound enclosure derived from those exact bounds, declared conservative in the result it returns, or exactly, and an exact value and an enclosure are never allowed to look alike.

8.6 Exact Interval Floats: the Domination Split

Throughout this section the mode is LongestPath, durations d_u lie in intervals, delays are fixed, and $f_v = P - \text{through}_v \geq 0$ is the float of node v . An interval-valued delay reduces to this setting by subdividing its edge with a node that carries the delay as its duration. The question is the exact range of f_v over the box of duration configurations.

Proposition 8.1 (Corner sufficiency). *With all other durations fixed, f_v is a monotone function of any single duration d_u . Consequently the extremes of f_v over the box are attained at corner configurations.*

Proof. A path visits a node at most once, so as a function of $x = d_u$ every path length is either constant or affine with slope one. Hence $P(x) = \max(C_0, C_1 + x)$, where C_1 is the best path through u and C_0 the best avoiding it, and likewise $\text{through}_v(x) = \max(A_0, A_1 + x)$ over the paths through v . Each has one breakpoint with slope 0 then 1. If P 's breakpoint comes first their difference has slope pattern 0, +1, 0, otherwise 0, -1, 0, and in both cases f_v is monotone in x on the whole line. For the corner claim, push any coordinate to whichever endpoint does not worsen the objective, which monotonicity permits, and repeat per coordinate until the configuration is a corner. $\square$

Lemma 8.2 (Incomparable coordinates). *If no complete path contains both u and v , then f_v is nondecreasing in d_u , whatever the other durations are.*

Proof. No through- v path contains u , so through_v does not depend on d_u , while P is nondecreasing in it. $\square$

Lemma 8.3 (Dominated coordinates). *If every complete path through u also passes v , and in particular if $u = v$, then f_v is nonincreasing in d_u , whatever the other durations are.*

Proof. Suppose increasing d_u increases P . The new maximiser must contain u , hence v , hence it is a through- v path, so through_v rises to equal P and the float drops to zero. If P does not increase, the float cannot rise because through_v is nondecreasing in every duration. $\square$

The two lemmas cover every duration except those in the *bypass set*

$$H_v = \{u : \text{some complete path contains both } u \text{ and } v, \text{ and some complete path contains } u \text{ but not } v\}$$

the nodes that share a route with v yet can also route around it. Figure 8.2 shows the three classes around one node.

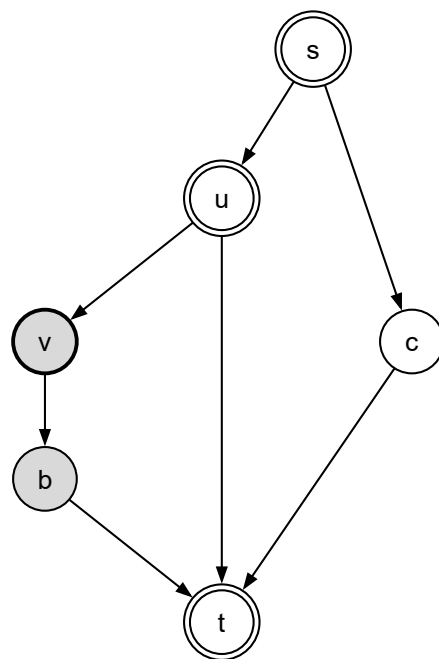


Figure 8.2: Duration classes relative to node v . Shaded nodes are dominated by v (every complete route through them passes v), the unfilled node is incomparable to it, and double-bordered nodes form the bypass set H_v : they share routes with v and can also route around it.

Theorem 8.4 (Exactness of the domination split). *The maximum of f_v over the box is attained at a corner with every incomparable duration at its upper bound, every dominated duration at its lower bound, and the bypass durations at some corner of H_v . The minimum is attained dually. Enumerating the $2^{|H_v|}$ bypass corners with the remaining durations pinned therefore yields the exact float range of v .*

Proof. By Proposition 8.1 an optimum lies at a corner. The three classes partition the durations. At an optimal corner, moving an incomparable duration to its upper bound cannot decrease the maximum by Lemma 8.2, and moving a dominated duration to its lower bound cannot decrease it by Lemma 8.3, so some optimal corner has the pinned pattern and lies within the enumerated set. $\square$

The cost is $2 \sum_v 2^{|H_v|}$ crisp propagations, one maximising and one minimising sweep per node, against 2^k for exhaustive enumeration of all k interval durations, and the exponent has moved from the size of the problem to the reconvergent width around each node. One exhaustive sweep serves every node simultaneously, so the implementation runs whichever of the two is cheaper and is never worse than exhaustive enumeration. Some instances must remain hard, because the general problem is NP-hard [27], and the split shows exactly where the hardness sits: in networks whose bypass sets are large.

The bypass set is a reconvergence measure, and this is where the toolkit touches the structural theme of Chapter 5 without ever calling its module. Both analyses pay for routes that divide and meet again, the decomposition module per join through the nodes it must fix, the split per node through H_v . The two sets index reconvergence at different granularities, neither contains the other in general, and neither computation uses the other.

8.7 Case Studies

Every number in this section is produced by the toolkit and certified against an independent computation. The certification corpus contains nine networks. On the seven where full path enumeration is affordable, forward values, through-values, margins and critical sets of every mode agree with a brute-force path oracle to within 3×10^{-14} , and on the remaining two the results pass invariant and sampling checks. Interval results are certified against exhaustive corner enumeration driven by the same path oracle, and the domination split agrees with exhaustive enumeration everywhere both can run.

8.7.1 One network, four questions

The wastewater treatment network of 32 nodes and 66 edges shows how much one set of inputs can say. Its stage processing times and pipeline delays fix the minimum release timeline at 45 units, carried by the chain $3 \rightarrow 11 \rightarrow 19 \rightarrow 27$ with a second chain $6 \rightarrow 14 \rightarrow 22 \rightarrow 30$ only 1.5 units behind (Table 8.2). Stage 6 alone carries slack 1 rather than 1.5, since its edge onto stage 11 lets it feed the critical chain as well. Read for the opposite extreme, the same inputs put the fastest achievable delivery at 15 units, and not along one route. The twelve margin-zero stages and the eighteen tight dependencies between them form a system of tied routes drawing on three of the eight sources, sharing no stage with the critical chain, and the nearest alternative stage sits 1.5 units behind the tie (Figure 8.3). The two extremal readings of one network differ not only in which stages they select but in the shape of what they select, a single thread against a redundant lattice.

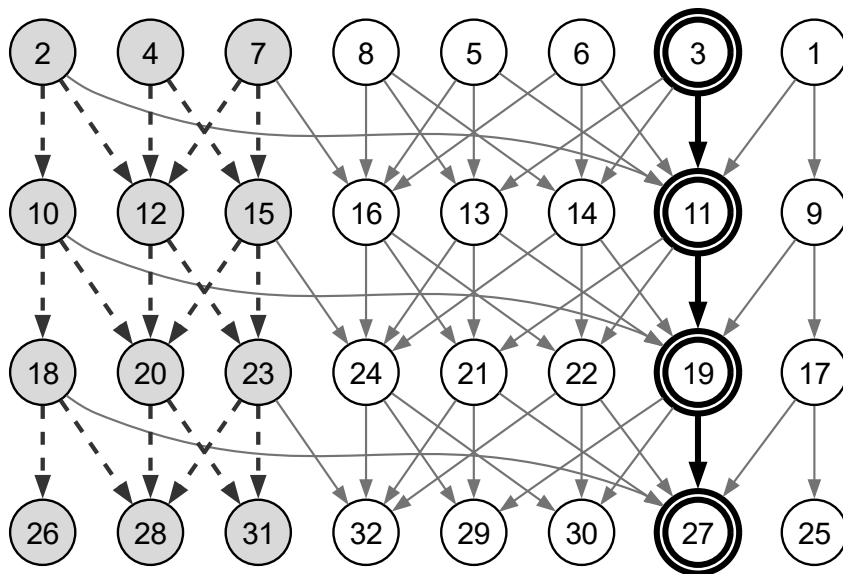


Figure 8.3: The water network read for both extremes. The bold double-bordered chain sets the minimum release timeline of 45 units. The shaded stages and dashed edges form the tied-fastest route system, every path through which completes in 15 units. The two readings share no stage.

Read as load rather than schedule, the same network delivers 480 units to its busiest outlet, and the stages that drive that figure are not the ones the completion times point at. The largest completions sit downstream where load gathers, but the leverage sits upstream at stages 3, 6 and 5, each reaching the

Table 8.2: LongestPath on the water network: completion 45 along the critical chain, with the second chain behind it.

Node	Completion	Slack
27	45.0	0
19	33.0	0
11	21.0	0
3	9.0	0
30	43.5	1.5
22	31.5	1.5
14	19.5	1.5
6	8.0	1.0

outlet along seven distinct routes, so a unit added at any of them arrives seven times. Under a budget ten percent above the current total those three stages can each grow by less than seven units, while stage 14, on two routes, has room for 24 (Table 8.3). Contribution and room order the stages differently, which is the information a capacity decision needs and which no scheduling slack could express.

Table 8.3: Accumulation on the water network under a budget of 528 units at outlet 27 (current total 480). Allowance is headroom divided by path multiplicity.

Node	Contribution	Multiplicity	Allowance
3	63	7	+6.9
6	56	7	+6.9
5	35	7	+6.9
1	30	5	+9.6
11	27	3	+16
8	24	6	+8
14	16	2	+24

Uncertainty then tests how much of the deterministic story survives. With every one of the 32 stage durations widened to ± 20 percent, exhaustive corner enumeration would require 2^{32} propagations, roughly 4.3×10^9 . The domination split required 2,269,328, a factor of 1,892 fewer, with the largest bypass set holding 18 of the 32 durations. The answer changes how the deterministic result must be read. Under full uncertainty no stage is necessarily critical, and exactly nine of the 32 are possibly critical. The single deterministic critical chain dissolves into a nine-candidate set (Figure 8.4), and 23 stages are provably incapable of becoming critical anywhere in the uncertainty box. A Monte Carlo attack on the same bounds, fifty thousand sampled configurations, produced no violation and

reached every bound at every node.

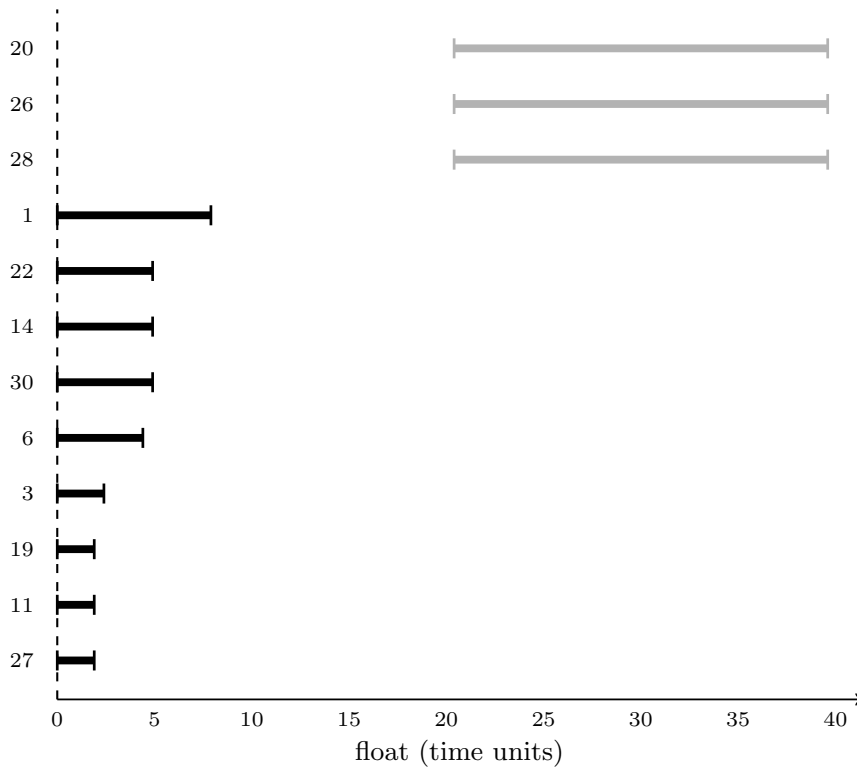


Figure 8.4: Exact float bounds on the water network with every duration at plus or minus 20 percent. Black bars are the nine possibly-critical stages, whose lower bounds reach zero. Grey bars are three of the 23 stages whose floats stay provably positive everywhere in the uncertainty box.

8.7.2 The most reliable route

The drone medical delivery network of 782 nodes, 2,789 edges, 33 launch sites and 323 destinations poses a multiplicative question, which route delivers with the highest end-to-end success. The factors are the study’s own component success rates, node priors between 0.75 and 0.95 and per-link probabilities between 0.70 and 0.95, so nothing in the scenario is invented. The best route in the network compounds to a success factor of 0.787, and it is a single direct leg from launch site 380 to destination 393. That brevity is structural rather than accidental. The strongest leg the data permits compounds its link and node factors to 0.9025, so every leg a route adds costs at least 9.8 percent of its success, and no amount of component quality along a longer route can repay that arithmetic. Route brevity is the currency of the multiplicative reading, which nothing in an additive analysis of the same network would reveal.

Read per destination rather than globally, the forward values become a service-level map, and the map is unequal (Figure 8.5). The five best-served destinations

Table 8.4: The five best-served destinations by single-route success. At the other end of the scale, 104 of the 323 destinations fall below 0.5 on even their best route.

Destination	Best-route success
393	0.787
325	0.771
209	0.747
248	0.742
125	0.734

sit within seven percent of the optimum (Table 8.4), while the weakest destination in the network can be reached with success 0.108 by its best route, and for 104 of the 323 destinations, roughly one in three, even the best single route succeeds less than half the time. No routing decision alone can serve a third of this network reliably, and where that matters the remedy is structural, stronger components on the thin corridors or genuinely redundant routes. Judging that redundancy is the neighbouring question, because a destination poorly served by its best single route may still be reliably reached when every route counts at once, which is the reachability reading of Chapter 6. The two readings separate route choice from system adequacy, and a delivery planner for this network needs both. Two thousand randomly sampled routes never exceeded the reported optimum and one reached it exactly, and the optimum itself is the product of three published input values, checkable by hand.

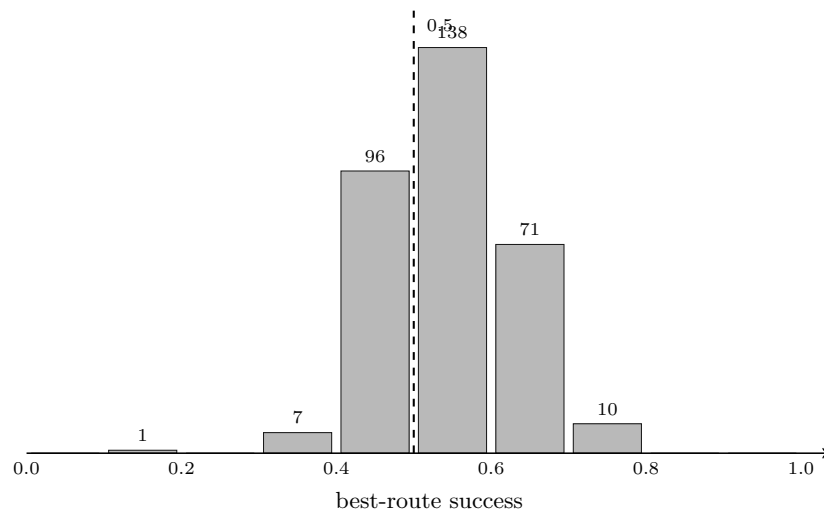


Figure 8.5: Best-route success across the 323 drone destinations. Left of the dashed line at 0.5 lie the 104 destinations that no single route serves reliably.

8.7.3 The boundary

Dense mesh reconvergence is where the split’s advantage disappears. On a five-by-five grid DAG with all 25 durations interval-valued, the bypass sets hold up to 23 of the 25 durations and the split would need more propagations than one shared exhaustive sweep, so the never-worse rule abandons the split there and the instance stands at the exhaustive boundary itself. The hardness results guarantee that some regime behaves this way, and it is the same regime in which the sets of nodes the probability toolkit must fix and enumerate explode. Mapping the boundary is part of the result rather than a qualification of it. Timings are reported only as measured. The water analyses run in 53 microseconds per pass for the additive modes and 7 for accumulation, the drone network in 1.8 milliseconds, and the exhaustive comparisons are quantified by run counts rather than by extrapolated wall-time.

8.8 Genericity

The boundary of the toolkit is drawn by the algebra rather than by the implementation. A new quantity joins by exhibiting its operator pair, and the pair earns its backward pass the way the shipped ones did, by carrying a residual or by being linear. The obligations are laws, verified on sampled values before a mode is accepted, so an extension can never silently acquire a backward pass its algebra does not support. The kernels are generic over the value they propagate, and interval support is a scheme wrapped around crisp runs rather than a property of a value type, which is what keeps its exactness claims provable.

The nearest extensions follow the existing arguments. The shortest-path dual of the domination split mirrors Section 8.6 and awaits only the line-by-line derivation. Interval-valued delays reduce to interval durations by subdividing their edges. Further out, neighbouring nodes have overlapping bypass sets and therefore overlapping corner evaluations, an economy of the kind the hash-consed store of Chapter 5 already practises for diamonds.

8.9 Summary

The toolkit reads one graph object in two directions. Forwards it computes the extremal value of any quantity that accumulates along dependency paths. Backwards it computes the room each component has before that value moves, wherever the quantity’s algebra defines such a reading, which is the condition every shipped mode satisfies and every future mode must. The classical critical

path method is the default instance rather than the design, and the sum family reports sensitivities and allowances because that is what its mathematics defines. Under interval uncertainty the forward quantities stay exact for the price of two crisp runs. The floats are exact through the domination split, whose correctness rests on three short proofs and whose cost grows with the reconvergent width around each node rather than with the number of uncertain inputs. That width stayed small enough on a real network to solve a 32-duration instance beyond the reach of exhaustive enumeration, and it does not stay small on dense meshes. Every number reported in this chapter is certified against an independent oracle, an exhaustive enumeration or a sampling attack.

Chapter 9

Package Implementation in Julia

Chapter in preparation. This chapter packages the modules of Chapters 4 to 8 as an installable Julia package: the public API, the type design that carries point, interval and probability-box values through one propagation, the test and validation harness, and the reproducibility story.

Chapter 10

The No-Code Web Interface

10.1 Introduction

Everything the framework can do, up to this point, it does in code. The analyses of Chapters 4 to 8 are invoked from a Julia session that loads a network, attaches inputs, and calls the modules, and Chapter 9 packages exactly that experience for a programmer. The people whose decisions these analyses serve are mostly not programmers. An asset engineer or a safety analyst understands the system, the failure modes, and what a reliability bound means for an intervention, without wanting to write a line of Julia to obtain one, and the gap between algorithmic availability and practical adoption is documented well enough to have a name, the last mile problem [63, 64].

Existing tools close parts of the gap and leave the whole open. Network visualisation environments such as Gephi [65] and Cytoscape [66] make graphs visual and explorable, and carry no quantitative reliability analysis, so they end where the questions of this thesis begin. The analytical capability lives in libraries, in Matlab [67], R [68], Julia [69], and Python [70], and a library is exactly the delivery form the excluded users cannot consume, because using one means writing code against it. A complete assessment of one system therefore tends to mean several tools, each with its own data format, stitched together by hand, and the stitching is itself a technical barrier.

Delivery architecture raises a separate obstacle. The accessible form of modern software is the web application, and the usual web application is a cloud service, which is precisely what industries with proprietary system data cannot accept. Sending a safety-critical network model to an external service raises custody and security questions [71] that in domains such as nuclear decommissioning are simply disqualifying. The local-first movement argues that software can keep the conveniences of the web while the user's data never leaves the user's machine [37], and that argument fits this framework's audience exactly.

The front end is the framework's answer to both obstacles at once, a no-code web interface that runs entirely on the user's machine. It puts visual network modelling and every analysis of the preceding chapters, with their native uncertainty forms, behind a browser interface that asks the user for files and clicks rather than code, while the computation and the data stay local. This chapter owns the architecture, the contract between the interface and the framework, the data-sovereignty commitment, and the design of the user experience. Chapter 11 then exercises the finished interface end to end on one network.

10.2 A Local Client and a Local Server

The front end is built as a client and a server that happen to share a machine (Figure 10.1). The server side is an analysis service, a Julia process that hosts the framework of Chapter 9 and exposes its analyses as HTTP endpoints bound to the local machine. The client side is a browser application, written in Angular with the network drawing done in d3, that drives those endpoints and renders their results. The two communicate over HTTP on localhost, so from the user’s seat the whole system is one tool opened in a browser tab, and no traffic leaves the machine.

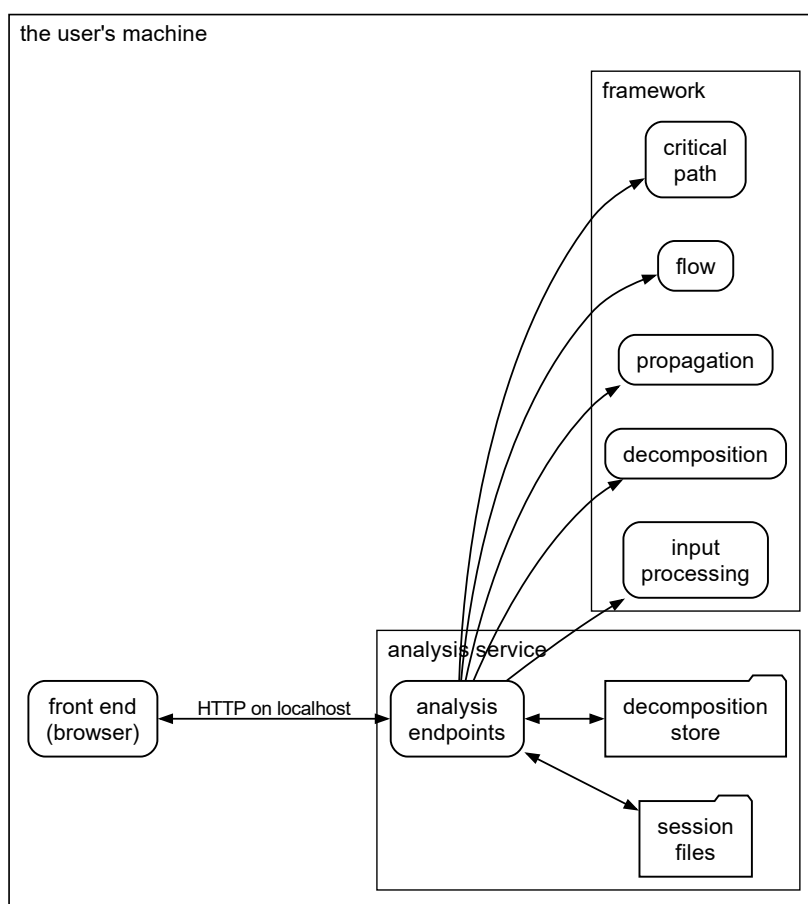


Figure 10.1: The local-first architecture. The browser front end drives the analysis service over HTTP on the local machine, and the service invokes the framework’s modules, keeping sessions and the decomposition store as files beside the user’s own.

Splitting a single-machine tool into a client and a server is a deliberate trade. The browser brings mature rendering, interaction, and accessibility machinery that no numerical language ecosystem matches, while Julia keeps the computa-

tion next to the package that implements it, so each side lives where it is strongest. The two sides can be developed, tested, and replaced independently, and because the boundary between them is ordinary HTTP, the same endpoints that serve the visual interface serve any scripted client, which keeps the service useful to programmers without a second code path. The boundary itself is written down as a machine-readable interface contract specifying every endpoint, request, and response shape, and both sides are built against that document rather than against each other's internals, which is what allows the interface and the framework to evolve at their own pace without silent drift.

10.3 The Contract

The service exposes one endpoint per analysis class, mirroring the module boundaries of the framework: network structure, diamond decomposition, probability propagation, flow, and critical path, alongside the housekeeping endpoints that manage uploads and sessions. An analysis request does not carry the network embedded in it. It names the user's files, and the service reads them through the input processing of Chapter 4, so the wire format for every input is the file format that chapter already defined, the edge list or adjacency file for structure and the JSON contracts for analysis inputs. The interface invents no second format, the files a user uploads are byte for byte the files the package reads, and the value-form dispatch is the same too, with the service detecting deterministic, interval, or probability-box inputs from the declared data type exactly as input processing does. One convenience softens the file requirements rather than the contract. When analysis inputs arrive without a structure file, the service reconstructs the edge list from the edge keys the inputs themselves carry, so a self-consistent set of analysis files is enough to work with.

Results travel back as JSON keyed by node and edge, and a value never loses its form in transit. A deterministic belief crosses the boundary as a number, an interval as its pair of bounds, and a probability box as a typed summary carrying its bounding curves and moments at the stated discretisation level, so the client always knows what kind of quantity it is rendering and cannot silently collapse an uncertain result into a point. Each analysis endpoint accepts the options its module defines and nothing else. The flow endpoint takes the solver choice among the three of Chapter 7, the failure-combination size, the enumeration limits, the target service level, and degradation scenarios, and its inputs are deterministic capacities because that is the capacity contract of Chapter 4. The critical path endpoint takes the value maps and runs the analyses of Chapter 8, and the propagation endpoint accepts any of the three value forms and can return the

decomposition detail alongside the beliefs when the user wants to see the structure behind a number.

Work the user has already paid for is kept. A session is nothing more than a folder holding the uploaded files and a plain JSON record of what has been computed, so analyses persist across visits, a session can be reopened, and deleting one is deleting a folder. The decomposition structure of Chapter 5 gets the same treatment at a finer grain, because identification is the expensive preprocessing step that every propagation reuses. The service computes the diamond store once, holds it in memory, and persists it beside the session, keyed by the content of the network and prior files so that editing an input invalidates exactly the store it affects. A propagation request that follows a decomposition request, or a second propagation under a different scenario, finds the structure already in place. The store that Chapter 5 built to keep the algorithms from repeating themselves here keeps the user's sessions from repeating themselves.

10.4 Data Sovereignty

Nothing described above leaves the user's machine, and that is a design commitment rather than a deployment accident. There is no account to create, no telemetry, and no remote service in the path. The browser talks to a process on the same computer, and every artefact of a session, the uploaded network, the analysis inputs, the computed results, and the cached decomposition, exists as ordinary files in one folder that the user can inspect, archive to controlled storage, or destroy, with nothing held anywhere else. The web technologies are used for what they are good at, rendering and interaction, without importing the custody model that usually comes with them.

For the industries this framework grew out of, the commitment is not a nicety. A decommissioning plan's dependency network, a defence logistics topology, or a utility's failure data are exactly the models an organisation cannot place in external custody, whatever the convenience [71], and tools that require it simply do not get adopted there. Local-first software keeps the usability argument and drops the custody problem [37]. For this framework's audience the practical consequence is plain enough to state in one line. Adopting the interface requires no security review of a third party, because there is no third party.

10.5 The Interface

The interface arranges the framework's capabilities as a guided pipeline (Figure 10.2), and each step unlocks when the data it needs exists. A user cannot

reach a propagation view before a network is loaded, or a comparison view before there are results to compare, so the interface never presents a dead control, and a first-time user is steered through the same order this thesis follows, structure first, then analysis, then comparison. Familiar users are not forced through ceremony, since any unlocked step is reachable directly, and the interface preserves each view's state, its filters, selections, and open tabs, across navigation, so moving between views to cross-reference results costs nothing.

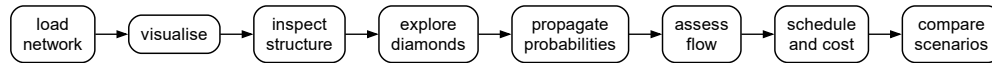
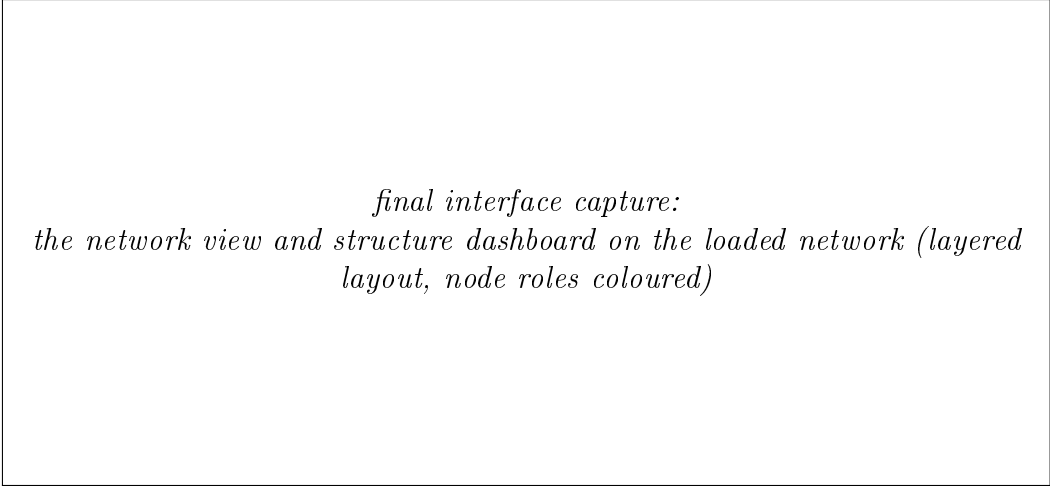


Figure 10.2: The guided pipeline. Each step unlocks when its prerequisite data exists, and any unlocked step is reachable directly.

A network enters as files, either picked individually or as a whole folder that the interface sorts by naming convention into the structure file and the analysis inputs. The same convention carries scenarios, since a network folder may hold several input sets, one per value form or operating case, and the interface treats each as a scenario of the same network, so one upload supports the whole comparative workflow. What the user then sees first is the network itself (Figure 10.3). The drawing is built from the unified graph object, laid out layer by layer along the iteration sets of Chapter 4 so that information visibly flows down the page in the order the algorithms process it, with the node roles of the model chapter, sources, sinks, forks, and joins, distinguished at a glance, and a force-directed alternative available when the layered view is not the natural reading. A structure dashboard beside it reports what input processing derived, the roles, the layers, and the connectivity, which is frequently where a modelling error in the input files announces itself, before any analysis has been run.

The decomposition of Chapter 5 is not hidden inside the propagation. The diamonds view lists the identified structures, and each diamond opens into a detail view that exploits the self-similarity that chapter established. A stored diamond is a graph object of the same form as the network, so the interface can present one as a network in its own right and run any analysis on it in isolation, with its local sources overridable, which turns the decomposition from an internal accelerator into an instrument for localised investigation. A reconvergence that dominates the network's uncertainty can be studied on its own, at its own scale, without leaving the tool.

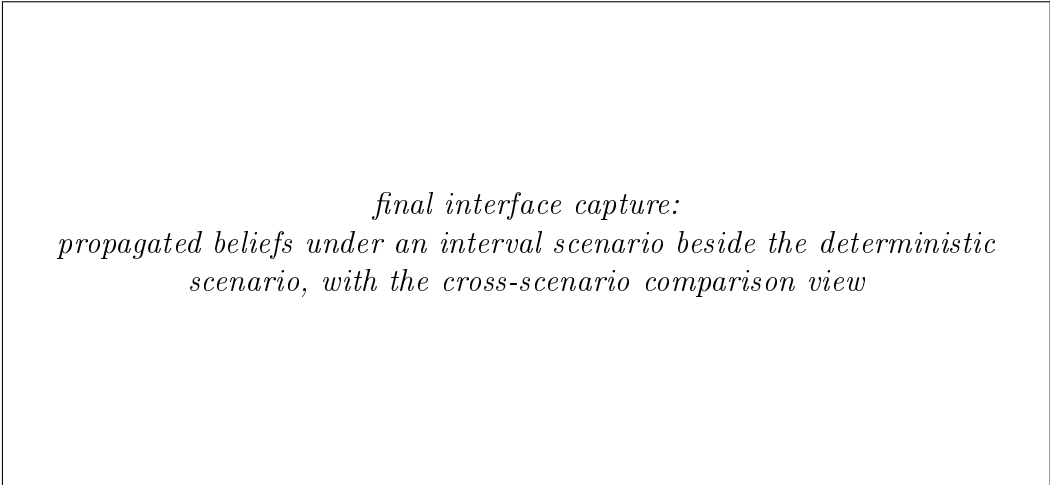
The analysis views deliver the toolkits of Chapters 6 to 8, and their common design rule is honesty about the form of a value. A propagated belief under a deterministic scenario is shown as a number, under an interval scenario as a range,



*final interface capture:
the network view and structure dashboard on the loaded network (layered
layout, node roles coloured)*

Figure 10.3: The network as the interface presents it, the interactive layered drawing and the structure dashboard derived by input processing.

and under a probability-box scenario as a band, with width statistics summarising where the network’s uncertainty concentrates, and at no point does the interface average, midpoint, or otherwise flatten an uncertain result into a false precision (Figure 10.4). The flow views report the throughput, bottleneck, sensitivity, and threshold results of Chapter 7, the scheduling views the time and cost analyses of Chapter 8, and a cross-scenario profile view sets the scenarios of one network side by side, which is where the comparative questions, what the interval inputs do to the answer, which design carries the risk, get answered without exporting anything to a spreadsheet. In-app documentation explains each view and the input formats in the user’s terms, so the interface carries its own manual.



*final interface capture:
propagated beliefs under an interval scenario beside the deterministic
scenario, with the cross-scenario comparison view*

Figure 10.4: Propagation results as the interface presents them, ranges shown as ranges, with scenarios of the same network compared side by side.

The boundaries of the design are as deliberate as its features. The interface is a single-user local tool, with no collaboration layer, because its custody commit-

ment is the point. It exposes the framework’s analyses and does not author new ones, since extending the framework is a programmer’s task and belongs to the package of Chapter 9. And it adds no analytical capability of its own, because everything it shows is produced by the modules the preceding chapters validated, which is precisely what makes its results trustworthy: the interface is a window, not a second implementation.

10.6 Summary

The front end closes the framework’s last mile. A local analysis service exposes the modules of Chapters 4 to 8 behind a documented contract in which the user’s files are the currency, inputs keep the formats input processing defined, results keep their value forms, and paid-for work persists in sessions and a content-keyed decomposition store. A browser front end turns that contract into a guided, no-code workflow, from upload through structure, decomposition, and the three toolkits to cross-scenario comparison, rendering uncertain results as the ranges and bands they are. Everything runs on the user’s machine and every artefact is an ordinary file the user owns, which is what makes the tool adoptable in the proprietary-data industries the framework serves. Chapter 11 puts the whole interface to work on a single network, end to end.

Chapter 11

An End-to-End Framework

Application: An Integrated Case Study

Chapter in preparation. One network, carried end to end through the finished no-code interface of Chapter 10: upload and structure, decomposition, all three toolkits under their native uncertainty forms, and cross-scenario comparison, shown through interface captures.

Chapter 12

Conclusions

Chapter in preparation. What the framework establishes, where exact analysis ends and why, and the work the three problem statements of Chapter 3 leave open.

Appendices

Appendix A

Case-Study Network Specifications

Appendix in preparation. See the content note in the source file for the intended structure. Cross-reference target: `app:network-specs`.

Appendix B

Full Numerical Results

Appendix in preparation. See the content note in the source file for the intended structure. Cross-reference target: `app:full-results`.

Bibliography

- [1] D.-H. Huang, C.-F. Huang and Y.-K. Lin, ‘Exact project reliability for a multi-state project network subject to time and budget constraints,’ *Reliability Engineering & System Safety*, vol. 195, p. 106 744, 2020. DOI: [10.1016/j.ress.2019.106744](https://doi.org/10.1016/j.ress.2019.106744)
- [2] Y. Tong, ‘New approaches for modeling and reliability assessment of infrastructure flow networks,’ Ph.D. dissertation, Georgia Institute of Technology, 2021.
- [3] L. Thurner et al., ‘Pandapower—an open-source python tool for convenient modeling, analysis, and optimization of electric power systems,’ *IEEE Transactions on Power Systems*, vol. 33, no. 6, pp. 6510–6521, 2018. DOI: [10.1109/TPWRS.2018.2829021](https://doi.org/10.1109/TPWRS.2018.2829021)
- [4] R. Billinton and R. N. Allan, ‘Network modelling and evaluation of simple systems,’ in *Reliability Evaluation of Engineering Systems: Concepts and Techniques*, Boston, MA: Springer US, 1992.
- [5] L. Xing and S. V. Amari, ‘Fault tree analysis,’ in *Handbook of Performability Engineering*, London: Springer London, 2008.
- [6] W. Liu and J. Li, ‘An improved cut-based recursive decomposition algorithm for reliability analysis of networks,’ *Earthquake Engineering and Engineering Vibration*, vol. 11, no. 1, pp. 1–10, 2012.
- [7] Y. Kim and W.-H. Kang, ‘Network reliability analysis of complex systems using a non-simulation-based method,’ *Reliability Engineering & System Safety*, vol. 110, pp. 80–88, 2013.
- [8] G. S. Fishman, ‘A comparison of four Monte Carlo methods for estimating the probability of s–t connectedness,’ *IEEE Transactions on Reliability*, vol. 35, no. 2, pp. 145–155, 1986.
- [9] R. E. Bryant, ‘Graph-based algorithms for Boolean function manipulation,’ *IEEE Transactions on Computers*, vol. C-35, no. 8, pp. 677–691, 1986. DOI: [10.1109/TC.1986.1676819](https://doi.org/10.1109/TC.1986.1676819)

- [10] A. Rauzy, ‘New algorithms for fault trees analysis,’ *Reliability Engineering & System Safety*, vol. 40, no. 3, pp. 203–211, 1993.
- [11] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. San Francisco, CA: Morgan Kaufmann, 1988.
- [12] S. L. Lauritzen and D. J. Spiegelhalter, ‘Local computations with probabilities on graphical structures and their application to expert systems,’ *Journal of the Royal Statistical Society, Series B*, vol. 50, no. 2, pp. 157–224, 1988.
- [13] A. Satyanarayana and R. K. Wood, ‘A linear-time algorithm for computing K-terminal reliability in series-parallel networks,’ *SIAM Journal on Computing*, vol. 14, no. 4, pp. 818–832, 1985. DOI: [10.1137/0214057](https://doi.org/10.1137/0214057)
- [14] A. Satyanarayana and M. K. Chang, ‘Network reliability and the factoring theorem,’ *Networks*, vol. 13, no. 1, pp. 107–120, 1983. DOI: [10.1002/net.3230130107](https://doi.org/10.1002/net.3230130107)
- [15] R. K. Wood, ‘A factoring algorithm using polygon-to-chain reductions for computing K-terminal network reliability,’ *Networks*, vol. 15, no. 2, pp. 173–190, 1985. DOI: [10.1002/net.3230150204](https://doi.org/10.1002/net.3230150204)
- [16] A. K. Goharshady and F. Mohammadi, ‘An efficient algorithm for computing network reliability in small treewidth,’ *Reliability Engineering & System Safety*, vol. 193, p. 106665, 2020. DOI: [10.1016/j.ress.2019.106665](https://doi.org/10.1016/j.ress.2019.106665)
- [17] J.-C. Filliâtre and S. Conchon, ‘Type-safe modular hash-consing,’ in *Proceedings of the ACM SIGPLAN Workshop on ML*, 2006, pp. 12–19. DOI: [10.1145/1159876.1159880](https://doi.org/10.1145/1159876.1159880)
- [18] J. E. Kelley and M. R. Walker, ‘Critical-path planning and scheduling,’ in *Proceedings of the Eastern Joint IRE-AIEE-ACM Computer Conference*, Boston, MA, 1959, pp. 160–173. DOI: [10.1145/1460299.1460318](https://doi.org/10.1145/1460299.1460318)
- [19] L. R. Ford and D. R. Fulkerson, ‘Maximal flow through a network,’ *Canadian Journal of Mathematics*, vol. 8, pp. 399–404, 1956. DOI: [10.4153/CJM-1956-045-5](https://doi.org/10.4153/CJM-1956-045-5)
- [20] L. A. Zadeh, ‘Fuzzy sets,’ *Information and Control*, vol. 8, no. 3, pp. 338–353, 1965.
- [21] G. Shafer, *A Mathematical Theory of Evidence*. Princeton, NJ: Princeton University Press, 1976.
- [22] S. Ferson, V. Kreinovich, L. Ginzburg, D. S. Myers and K. Sentz, ‘Constructing probability boxes and Dempster-Shafer structures,’ Sandia National Laboratories, Albuquerque, NM, Tech. Rep. SAND2002-4015, 2003.

- [23] R. C. Williamson and T. Downs, ‘Probabilistic arithmetic I: Numerical methods for calculating convolutions and dependency bounds,’ *International Journal of Approximate Reasoning*, vol. 4, no. 2, pp. 89–158, 1990.
- [24] D. Bertsimas, E. Nasrabadi and S. Stiller, ‘Robust and adaptive network flows,’ *Operations Research*, vol. 61, no. 5, pp. 1218–1242, 2013. DOI: [10.1287/opre.2013.1200](https://doi.org/10.1287/opre.2013.1200)
- [25] P. Doulliez and E. Jamoulle, ‘Transportation networks with random arc capacities,’ *Revue française d’automatique, informatique, recherche opérationnelle. Recherche opérationnelle*, vol. 6, no. V3, pp. 45–59, 1972. [Online]. Available: https://www.numdam.org/item/R0_1972__6_3_45_0/
- [26] D. G. Malcolm, J. H. Roseboom, C. E. Clark and W. Fazar, ‘Application of a technique for research and development program evaluation,’ *Operations Research*, vol. 7, no. 5, pp. 646–669, 1959. DOI: [10.1287/opre.7.5.646](https://doi.org/10.1287/opre.7.5.646)
- [27] S. Chanas and P. Zieliński, ‘The computational complexity of the criticality problems in a network with interval activity times,’ *European Journal of Operational Research*, vol. 136, no. 3, pp. 541–550, 2002. DOI: [10.1016/S0377-2217\(01\)00048-0](https://doi.org/10.1016/S0377-2217(01)00048-0)
- [28] Esri, *ArcGIS utility network: Trace types*, <https://doc.esri.com/en/arcgis-pro/latest/help/data/utility-network/utility-network-trace-types.html>, 2024.
- [29] Hottinger Brüel & Kjær (ReliaSoft). ‘BlockSim: System reliability, availability, and maintainability analysis.’
- [30] Isograph Ltd. ‘Availability Workbench.’
- [31] ITEM Software. ‘ITEM ToolKit: Reliability analysis and safety software tools.’
- [32] S. Balakrishnan and B. Cassottana, ‘InfraRisk: An open-source simulation platform for resilience analysis in interconnected power–water–transport networks,’ *Sustainable Cities and Society*, vol. 83, p. 103963, 2022. DOI: [10.1016/j.scs.2022.103963](https://doi.org/10.1016/j.scs.2022.103963)
- [33] A. Abendroth, ‘The democratization of software engineering: Evolution, definition, and the future of low-code and no-code platforms,’ *Management Review Quarterly*, vol. 67, 2026. DOI: [10.1007/s11301-026-00603-2](https://doi.org/10.1007/s11301-026-00603-2)
- [34] SimScale GmbH, *SimScale: AI-native engineering simulation software in the cloud*, <https://www.simscale.com/>, 2026.

- [35] M. Daud, F. M. Ugliotti and A. Osello, ‘Comprehensive analysis of the use of web-gis for natural hazard management: A systematic review,’ *Sustainability*, vol. 16, no. 10, p. 4238, 2024. DOI: [10.3390/su16104238](https://doi.org/10.3390/su16104238)
- [36] M. P. Repetto, M. Burlando, G. Solari, P. De Gaetano, M. Pizzo and M. Tizzi, ‘A web-based GIS platform for the safe management and risk assessment of complex structural and infrastructural systems exposed to wind,’ *Advances in Engineering Software*, vol. 117, pp. 29–45, 2018. DOI: [10.1016/j.advengsoft.2017.03.002](https://doi.org/10.1016/j.advengsoft.2017.03.002)
- [37] M. Kleppmann, A. Wiggins, P. van Hardenberg and M. McGranaghan, ‘Local-first software: You own your data, in spite of the cloud,’ in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, ser. Onward! 2019, ACM, 2019, pp. 154–178. DOI: [10.1145/3359591.3359737](https://doi.org/10.1145/3359591.3359737)
- [38] The MathWorks, Inc., *Graph and network algorithms*, <https://www.mathworks.com/help/matlab/graph-and-network-algorithms.html>, 2026.
- [39] G. Csárdi and T. Nepusz, ‘The igraph software package for complex network research,’ *InterJournal, Complex Systems*, p. 1695, 2006.
- [40] L. J. M. Aslett, *ReliabilityTheory: Tools for structural reliability analysis*, <https://cran.r-project.org/package=ReliabilityTheory>, 2024.
- [41] petgraph contributors, *Petgraph: Graph data structure library for Rust*, <https://github.com/petgraph/petgraph>, 2026.
- [42] A. A. Hagberg, D. A. Schult and P. J. Swart, ‘Exploring network structure, dynamics, and function using NetworkX,’ in *Proceedings of the 7th Python in Science Conference (SciPy2008)*, G. Varoquaux, T. Vaught and J. Millman, Eds., Pasadena, CA, 2008, pp. 11–15. DOI: [10.25080/TCWV9851](https://doi.org/10.25080/TCWV9851)
- [43] V. Gaur, O. P. Yadav, G. Soni and A. P. S. Rathore, ‘A literature review on network reliability analysis and its engineering applications,’ *Proceedings of the Institution of Mechanical Engineers, Part O: Journal of Risk and Reliability*, vol. 235, no. 2, pp. 167–181, 2021.
- [44] J. Bezanson, A. Edelman, S. Karpinski and V. B. Shah, ‘Julia: A fresh approach to numerical computing,’ *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017. DOI: [10.1137/141000671](https://doi.org/10.1137/141000671)
- [45] M. Ouyang, ‘Review on modeling and simulation of interdependent critical infrastructure systems,’ *Reliability Engineering & System Safety*, vol. 121, pp. 43–60, 2014. DOI: [10.1016/j.ress.2013.06.040](https://doi.org/10.1016/j.ress.2013.06.040)

- [46] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley and S. Havlin, ‘Catastrophic cascade of failures in interdependent networks,’ *Nature*, vol. 464, no. 7291, pp. 1025–1028, 2010. DOI: [10.1038/nature08932](https://doi.org/10.1038/nature08932)
- [47] H. Pérez-Rosés, ‘Sixty years of network reliability,’ *Mathematics in Computer Science*, vol. 12, pp. 275–293, 2018. DOI: [10.1007/s11786-018-0345-5](https://doi.org/10.1007/s11786-018-0345-5)
- [48] E. F. Moore and C. E. Shannon, ‘Reliable circuits using less reliable relays,’ *Journal of the Franklin Institute*, vol. 262, no. 3, pp. 191–208, 1956. DOI: [10.1016/0016-0032\(56\)90559-2](https://doi.org/10.1016/0016-0032(56)90559-2)
- [49] L. B. Page and J. E. Perry, ‘A practical implementation of the factoring theorem for network reliability,’ *IEEE Transactions on Reliability*, vol. 37, no. 3, pp. 259–267, 1988. DOI: [10.1109/24.3752](https://doi.org/10.1109/24.3752)
- [50] N. Robertson and P. D. Seymour, ‘Graph minors. II. Algorithmic aspects of tree-width,’ *Journal of Algorithms*, vol. 7, no. 3, pp. 309–322, 1986. DOI: [10.1016/0196-6774\(86\)90023-4](https://doi.org/10.1016/0196-6774(86)90023-4)
- [51] A. P. Ershov, ‘On programming of arithmetic operations,’ *Communications of the ACM*, vol. 1, no. 8, pp. 3–6, 1958, The origin of the hash-consing technique. DOI: [10.1145/368892.368907](https://doi.org/10.1145/368892.368907)
- [52] R. E. Bryant and C. Meinel, ‘Ordered binary decision diagrams,’ in *Logic Synthesis and Verification*, Boston, MA: Springer, 2002.
- [53] Y. Tong and I. Tien, ‘Probability propagation method for reliability assessment of acyclic directed networks,’ *ASCE-ASME Journal of Risk and Uncertainty in Engineering Systems, Part A: Civil Engineering*, vol. 5, no. 3, p. 04019011, 2019.
- [54] L. G. Valiant, ‘The complexity of enumeration and reliability problems,’ *SIAM Journal on Computing*, vol. 8, no. 3, pp. 410–421, 1979. DOI: [10.1137/0208032](https://doi.org/10.1137/0208032)
- [55] M. O. Ball, ‘Computational complexity of network reliability analysis: An overview,’ *IEEE Transactions on Reliability*, vol. R-35, no. 3, pp. 230–239, 1986. DOI: [10.1109/TR.1986.4335422](https://doi.org/10.1109/TR.1986.4335422)
- [56] D. Jones et al., ‘Conceptual design of a medical drone logistics network for Scotland,’ *International Journal of Logistics Research and Applications*, 2025, Submitted.
- [57] L. R. Ford and D. R. Fulkerson, *Flows in Networks*. Princeton, NJ: Princeton University Press, 1962.

- [58] Z. W. Birnbaum, ‘On the importance of different components in a multicomponent system,’ in *Multivariate Analysis II*, P. R. Krishnaiah, Ed., New York: Academic Press, 1969, pp. 581–592.
- [59] S. Chanas and P. Zieliński, ‘On the hardness of evaluating criticality of activities in a planar network with duration intervals,’ *Operations Research Letters*, vol. 31, no. 1, pp. 53–59, 2003.
- [60] D. Dubois, H. Fargier and J. Fortin, ‘Computational methods for determining the latest starting times and floats of tasks in interval-valued activity networks,’ *Journal of Intelligent Manufacturing*, vol. 16, pp. 407–421, 2005. DOI: [10.1007/s10845-005-1654-5](https://doi.org/10.1007/s10845-005-1654-5)
- [61] J. Fortin, P. Zieliński, D. Dubois and H. Fargier, ‘Criticality analysis of activity networks under interval uncertainty,’ *Journal of Scheduling*, vol. 13, pp. 609–627, 2010. DOI: [10.1007/s10951-010-0163-3](https://doi.org/10.1007/s10951-010-0163-3)
- [62] F. Baccelli, G. Cohen, G. J. Olsder and J.-P. Quadrat, *Synchronization and Linearity: An Algebra for Discrete Event Systems*. Chichester: John Wiley & Sons, 1992.
- [63] J. E. Hannay, C. MacLeod, J. Singer, H. P. Langtangen, D. Pfahl and G. Wilson, ‘How do scientists develop and use scientific software?’ In *Proceedings of the 2009 ICSE Workshop on Software Engineering for Computational Science and Engineering*, IEEE, 2009, pp. 1–8. DOI: [10.1109/SECSE.2009.5069155](https://doi.org/10.1109/SECSE.2009.5069155)
- [64] P. Prabhu et al., ‘A survey of the practice of computational science,’ in *Proceedings of the 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC ’11, New York, NY: ACM, 2011, pp. 1–12. DOI: [10.1145/2063348.2063374](https://doi.org/10.1145/2063348.2063374)
- [65] M. Bastian, S. Heymann and M. Jacomy, ‘Gephi: An open source software for exploring and manipulating networks,’ in *Proceedings of the International AAAI Conference on Web and Social Media*, vol. 3, 2009, pp. 361–362. DOI: [10.1609/icwsm.v3i1.13937](https://doi.org/10.1609/icwsm.v3i1.13937)
- [66] P. Shannon et al., ‘Cytoscape: A software environment for integrated models of biomolecular interaction networks,’ *Genome Research*, vol. 13, no. 11, pp. 2498–2504, 2003. DOI: [10.1101/gr.1239303](https://doi.org/10.1101/gr.1239303)
- [67] E. Patelli, H. George-Williams, J. Sadeghi, R. Rocchetta, M. Broggi and M. de Angelis, ‘OpenCossan 2.0: An efficient computational toolbox for risk, reliability and resilience analysis,’ in *Proceedings of the Joint ICVRAM ISUMA UNCERTAINTIES Conference*, A. T. Beck, G. F. M. de Souza and M. A. Trindade, Eds., 2018.

- [68] S. Ferson, *Pba.r: Probability bounds analysis in R*, <https://github.com/ScottFerson/pba.r>, 2019.
- [69] A. Gray, S. Ferson and E. Patelli, *ProbabilityBoundsAnalysis.jl: Arithmetic with sets of distributions*, Proceedings of JuliaCon, 2021. [Online]. Available: <https://github.com/Institute-for-Risk-and-Uncertainty/ProbabilityBoundsAnalysis.jl>
- [70] N. Gray, S. Ferson, M. De Angelis, A. Gray and F. Baumont de Oliveira, ‘Probability bounds analysis for Python,’ *Software Impacts*, vol. 12, p. 100 246, 2022. DOI: [10.1016/j.simpa.2022.100246](https://doi.org/10.1016/j.simpa.2022.100246)
- [71] K. Hashizume, D. G. Rosado, E. Fernández-Medina and E. B. Fernández, ‘An analysis of security issues for cloud computing,’ *Journal of Internet Services and Applications*, vol. 4, no. 1, pp. 1–13, 2013. DOI: [10.1186/1869-0238-4-5](https://doi.org/10.1186/1869-0238-4-5)